

Componentes/Discount/STDRuleDiscount.PRW

984 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static nTotDa1      := 0 //Armazena o total dos produtos contidos na tabela DA1
5 Static lDescGrupo   := .F.
6 Static lFrmDesc     := .F.
7 Static lDscTotSPg   := .F.//Desconto Varejo no Total sem Grupo e sem forma de pagamento
8
9
10 //-----
11 /*{Protheus.doc} STDtotRlDi
12 Function Busca Regras de Desconto do Tipo Total
13
14 @param  cCliCode      Codigo do Cliente
15 @param  cCliStore     Loja do Cliente
16 @param  lPayRule      Indica se Busca Regras do Tipo Pagamento ou Tipo Total
17 @return aRet[1] MEI_CODREG      1 Codigo da Regra
18 @return aRet[2] MEJ_PRINUM      2 Prioridade da Regra
19 @return aRet[3] MEI_ACUMUL      3 Indica se a Regra é acumulativa
20 @return aRet[4] MEI_DESCRI      4 Descrição da Regra
21 @obs      Verifica condicoes de Filial, Hora/Data/Semana e Cliente. Nao verifica o Range de Valores
22 @sample
23 */
24 //-----
25 Function STDtotRlDi( cCliCode , cCliStore , lPayRule, lPgto )
26
27 Local aArea      := GetArea()      // Guarda Alias Corrente
28 Local aRet       := {}             // Retorna regras validas
29 Local cPriority   := ""             // Prioridade da regra (MEJ_PRINUM)
30
31 Default cCliCode := ""
32 Default cCliStore := ""
33 Default lPayRule := .F.
34 Default lPgto    := .F.
35
36 ParamType 0 var cCliCode As Character Default ""
37 ParamType 1 var cCliStore As Character Default ""
38 ParamType 1 var lPayRule As Logical Default .F.
39
40 DbSelectArea("MEI")
41 DbSetOrder(3) //MEI_FILIAL+MEI_TPIMPD
42 If DbSeek(xFilial("MEI") + "T")
43     While !(MEI->(Eof())) .AND. (AllTrim(MEI->MEI_TPIMPD) == "T")
44
45         If STDValRule( cCliCode , cCliStore ) // Validacoes gerais
46
47             cPriority := STDPriority( MEI->MEI_CODREG ) // Busca prioridade (MEJ_PRINUM)
48             AADD( aRet , { MEI->MEI_CODREG, cPriority , MEI->MEI_ACUMUL , MEI->MEI_DESCRI, MEI->MEI_CODTAB } )
49
50         EndIf
51
52     MEI->(DbSkip())

```

```

53
54     EndDo
55 EndIf
56
57 RestArea( aArea )
58
59 Return aRet
60
61 //-----
62 /*/{Protheus.doc} STDItemRldi
63 Function Busca Regras de Desconto do Tipo Item
64
65 @param  cItemCode      Codigo do Item
66 @param  nValueItem     Valor do Item
67 @param  cTypeProd      Tipo Saldão ou mostruário
68 @param  cCliCode       Codigo do cliente
69 @param  cCliStore      Loja do Cliente
70 @return aRet[1] MEI_CODREG      1 Codigo da Regra
71 @return aRet[2] MEJ_PRINUM      2 Prioridade da Regra
72 @return aRet[3] MEI_ACUMUL      3 Indica se a Regra é acumulativa
73 @return aRet[4] MEI_DESCRI      4 Descrição da Regra
74 @obs    Verifica condicoes de Filial, Hora/Data/Semana e Cliente. Nao verifica o Range de Valores
75 @sample
76 /*/
77 //-----
78 Function STDItemRldi( cItemCode , nValueItem , cTypeProd , cCliCode , ;
79                     cCliStore, cTypeCall )
80
81 Local aArea      := GetArea()      // Guarda Alias Corrente
82 Local aRet       := {}             // Retorna regras validas
83 Local aRange     := {}             // Armazena Range de valores
84 Local lCategory  := .F.            // Indica se existe categoria para o produto
85 Local cCategory  := ""             // Armazena categoria do produto
86 Local nI         := 0              // Variavel contador
87 Local aAreaSB1   := SB1->(GetArea()) // Salva a posição da SB1
88
89 Default cItemCode := ""
90 Default nValueItem := 0
91 Default cTypeProd  := ""
92 Default cCliCode   := ""
93 Default cCliStore  := ""
94 Default cTypeCall  := "I"          // "I" = Item, "T" = Total
95
96 PosSB1(cItemCode) // Posiciona no produto
97
98 /*/
99     Se existir categoria no produto, vai pesquisar nas regras por categoria
100 /*/
101 DbSelectArea("ACV")
102 DbSetOrder(5) //ACV_FILIAL+ACV_CODPRO+ACV_CATEGO
103 If DbSeek(xFilial("ACV") + alltrim(cItemCode))
104     cCategory := ACV->ACV_CATEGO

```

```

105     lCategory      := .T.
106     LjGrvLog("Regra de desconto", "STDItemRlDi - Categoria: " + cCategory,)
107 Else
108     ACV->( DbSetOrder(2) ) // ACV_FILIAL+ACV_GRUPO+ACV_CODPRO+ACV_CATEG
109     If !Empty(SB1->B1_GRUPO) .AND. DbSeek(xFilial("ACV") + alltrim(SB1->B1_GRUPO))
110         cCategory      := ACV->ACV_CATEG
111         lCategory      := .T.
112         LjGrvLog("Regra de desconto", "STDItemRlDi - Categoria: " + cCategory,)
113     EndIf
114 EndIf
115
116 RestArea(aAreaSB1)
117
118 /*/
119     Busca regra por Categoria
120 /*/
121 If lCategory
122     DbSelectArea("MB8")
123     DbSetOrder(3)//MB8_FILIAL+MB8_CATEG
124     If DbSeek(xFilial("MB8") + cCategory ) //MB8_FILIAL+MB8_CATEG
125         While !MB8->(Eof()) .AND. AllTrim(MB8->MB8_CATEG) == AllTrim(cCategory)
126             DbSelectArea("MEI")
127             DbSetOrder(1)//MEI_FILIAL+MEI_CODREG
128             If DbSeek(xFilial("MEI") + MB8->MB8_CODREG )
129
130                 If (MEI->MEI_TPIMPD == cTyPeCall) .And. (MEI->MEI_ATIVA = "1") //só irá consultar regras ATIVAS
131
132                 If STDValRule( cCliCode , cCliStore ) // Validacao generica por Data, Ativa, Cliente, Grupo de cliente, Filial e Prioridade
133
134                     If STDValMoSa( MEI->MEI_CODREG , cTypeProd ) // Validacao especifica item (Mostruario/Saldao)
135
136                         cPriority := STDPriority( MEI->MEI_CODREG ) // Busca prioridade (MEJ_PRINUM)
137
138                         AADD( aRet , { MEI->MEI_CODREG ,; // 01-Codigo da Regra de Desconto
139                                     cPriority ,; // 02-Prioridade da Regra de Desconto
140                                     MEI->MEI_ACUMUL ,; // 03-Acumulativa (Sim/Nao)
141                                     MEI->MEI_DESCRI ,; // 04-Descrição da regra
142                                     MB8->MB8_DESCPR ,; // 05-Percentual de Desconto
143                                     MB8->MB8_DESCVL ,; // 06-Valor de Desconto
144                                     "" ,; // 07-Codigo do Produto
145                                     "PRO" ,; // 08-Indica se é uma regra especifica por produto
146                                     MB8->MB8_TPREGR ,; // 09-Tipo de Regra
147                                     MB8->MB8_REFGRD ,; // 10-Referencia do Grid (Item da Regra)
148                                     Nil ,; // 11-Posicao nao usada
149                                     lCategory ,; // 12-Indica que existe regra por categoria do produto
150                                     MB8->MB8_QTDPRO ,; // 13-Qtd. Venda para aplicacao da regra
151                                     MB8->(Recno())} ) // 14-Recno do Registro da tabela
152                         LjGrvLog("Regra de desconto", "STDItemRlDi - Regra por Categoria - Regra: " + MEI->MEI_CODREG,)
153
154                     EndIf
155                 EndIf
156             Endif

```

```

157         EndIf
158         MB8->(DbSkip())
159     EndDo
160 EndIf
161 EndIf
162
163 /*/
164     Busca regra por produto
165 /*/
166 DbSelectArea("MB8")
167 DbSelectArea("MEI")
168 MB8->(DbSetOrder(2)) //MB8_FILIAL+MB8_CODPRO
169 If MB8->(DbSeek(xFilial("MB8") + cItemCode )) //MB8_FILIAL+MB8_CODPRO
170     While MB8->(EOF()) .AND. AllTrim(MB8->MB8_CODPRO) == AllTrim(cItemCode)
171         MEI->(DbSetOrder(1))//MEI_FILIAL+MEI_CODREG
172         If MEI->(DbSeek(xFilial("MEI")+ MB8->MB8_CODREG ))
173
174             If (MEI->MEI_TPIMPD == cTypeCall) .And. (MEI->MEI_ATIVA = "1") //só irá consultar regras ATIVAS
175
176                 If STDValRule( cCliCode , cCliStore ) // Validacao generica
177
178                     If STDValMoSa( MEI->MEI_CODREG , cTypeProd ) // Validacao especifica item (Mostruario/Saldao)
179
180                         cPriority := STDPriority( MEI->MEI_CODREG ) // Busca prioridade (MEJ_PRINUM)
181
182                         AADD( aRet , { MEI->MEI_CODREG ,; // 01-Codigo da Regra de Desconto
183                                     cPriority ,; // 02-Prioridade da Regra de Desconto
184                                     MEI->MEI_ACUMUL ,; // 03-Acumulativa (Sim/Nao)
185                                     MEI->MEI_DESCRI ,; // 04-Descrição da regra
186                                     MB8->MB8_DESCPR ,; // 05-Percentual de Desconto
187                                     MB8->MB8_DESCVL ,; // 06-Valor de Desconto
188                                     MB8->MB8_CODPRO ,; // 07-Codigo do Produto
189                                     "PRO" ,; // 08-Indica se é uma regra especifica por produto
190                                     MB8->MB8_TPREGR ,; // 09-Tipo de Regra
191                                     MB8->MB8_REFGRD ,; // 10-Referencia do Grid (Item da Regra)
192                                     Nil ,; // 11-Posicao nao usada
193                                     lCategory ,; // 12-Indica que existe regra por categoria do produto
194                                     MB8->MB8_QTDPRO ,; // 13-Qtd. Venda para aplicacao da regra
195                                     MB8->(Recno()) } ) // 14-Recno do Registro da tabela
196                         LjGrvLog("Regra de desconto", "STDItemRlDi - Regra por Produto - Regra: " + MEI->MEI_CODREG,)
197
198                     EndIf
199                 EndIf
200             EndIf
201         EndIf
202         MB8->(DbSkip())
203     EndDo
204 EndIf
205
206 /*/
207     Busca regra por Range(Todos os produtos desde que pertença ao Range)
208 /*/

```

```

209 DbSelectArea("MEI")
210 MEI->(DbSetOrder(3)) //MEI_FILIAL+MEI_TPIMPD
211 If MEI->(DbSeek(xFilial("MEI") + "I"))
212     While !MEI->(Eof())
213
214         If MEI->MEI_ATIVA = "1" .And. (MEI->MEI_TPIMPD == cTyPeCall)
215             /*/
216             Busca Range para a regra.
217             /*/
218             aRange := STDItEranG( MEI->MEI_CODREG , nValueItem )
219
220             If Len(aRange) > 0
221
222                 If STDValRule( cCliCode , cCliStore ) // Validacao generica
223
224                     cPriority := STDPriority( MEI->MEI_CODREG ) // Busca prioridade (MEJ_PRINUM)
225
226                     For nI := 1 to len(aRange)
227                         AADD( aRet , { MEI->MEI_CODREG ,; // 01-Codigo da Regra de Desconto
228                                     cPriority ,; // 02-Prioridade da Regra de Desconto
229                                     MEI->MEI_ACUMUL ,; // 03-Acumulativa (Sim/Nao)
230                                     MEI->MEI_DESCRI ,; // 04-Descrição da regra
231                                     aRange[nI][3] ,; // 05-Percentual de Desconto
232                                     aRange[nI][4] ,; // 06-Valor de Desconto
233                                     Nil ,; // 07-Codigo do Produto
234                                     "TOT" ,; // 08-Indica se é uma regra especifica por produto
235                                     aRange[nI][5] ,; // 09-Tipo de Regra
236                                     aRange[nI][6] ,; // 10-Referencia do Grid (Item da Regra)
237                                     aRange[nI][2] ,; // 11-Valor Final da Faixa
238                                     lCategory ,; // 12-Indica que existe regra por categoria do produto
239                                     0 ,; // 13-Qtd. Venda para aplicacao da regra
240                                     0 } ) // 14-Recno do Registro da tabela
241                     Next nI
242
243                 EndIf
244             EndIf
245         Endif
246         MEI->(DbSkip())
247     EndDo
248 EndIf
249
250 RestArea( aArea )
251
252 Return aRet
253
254
255 //-----
256 /*/{Protheus.doc} STDValidPer
257 Function Valida o período da regra.
258
259 @param dDatDe          Data inicial da regra
260 @param dDatAte          Data final da regra

```

```

261 @param   cCodReg           Codigo da regra
262
263 @return   lRet              Retorna se a regra é válida
264 @obs      Verifica condicoes de Data, Hora e Dia da Semana
265 @sample
266 /*/
267 //-----
268 Static Function STDValidPer( dDatDe, dDatAte, cCodReg )
269
270 Local aArea      := GetArea()           // Posicionamento atual
271 Local lRet       := .F.                 // Retorno da Funcao
272 Local cTime      := Time()              // Horario atual da chamada da funcao para validar com horario da regra
273 Local dData      := Date()              // Armazena a data atual para validar a data da regra
274 Local cDiaSem    := CDOW(dData)         // Converte data para um dia da semana
275 Local aHora      := {}                  // Armazena a hora da regra para validar horario
276
277 Default dDatDe   := CTOD(" ")
278 Default dDatAte  := CTOD(" ")
279 Default cCodReg  := ""
280
281 ParamType 0 var  dDatDe      As Date      Default CTOD(" ")
282 ParamType 1 var  dDatAte     As Date      Default CTOD(" ")
283 ParamType 2 var  cCodReg     As Character  Default ""
284
285 /*/
286     Valida se a data informada e permitida para aplicar a regra de acordo com a data atual
287 /*/
288 If (dData >= dDatDe) .And. (dData <= dDatAte)
289     DbSelectArea("MB7")
290     DbSetOrder(1) //MB7_FILIAL+MB7_CODREG
291     If DbSeek(xFilial("MB7") + cCodReg)
292         Do Case
293             Case Upper(cDiaSem) == "SUNDAY"           //Domingo
294                 aHora := {AllTrim(MB7->MB7_HRDOMI), AllTrim(MB7->MB7_HRDOMF)}
295             Case Upper(cDiaSem) == "MONDAY"           //Segunda
296                 aHora := {AllTrim(MB7->MB7_HRSEGI), AllTrim(MB7->MB7_HRSEGF)}
297             Case Upper(cDiaSem) == "TUESDAY"          //Terca
298                 aHora := {AllTrim(MB7->MB7_HRTERI), AllTrim(MB7->MB7_HRTERF)}
299             Case Upper(cDiaSem) == "WEDNESDAY"        //Quarta
300                 aHora := {AllTrim(MB7->MB7_HRQUAI), AllTrim(MB7->MB7_HRQUAF)}
301             Case Upper(cDiaSem) == "THURSDAY"         //Quinta
302                 aHora := {AllTrim(MB7->MB7_HRQUII), AllTrim(MB7->MB7_HRQUIF)}
303             Case Upper(cDiaSem) == "FRIDAY"           //Sexta
304                 aHora := {AllTrim(MB7->MB7_HRSEXI), AllTrim(MB7->MB7_HRSEXF)}
305             Case Upper(cDiaSem) == "SATURDAY"         //Sabado
306                 aHora := {AllTrim(MB7->MB7_HRSABI), AllTrim(MB7->MB7_HRSABF)}
307         End
308     EndIf
309     MB7->(DbCloseArea())
310 EndIf
311
312

```

```

313 // Valida se a hora da regra e permitida para ser aplicada de acordo
314 // com a data atual.
315
316 If Len(aHora) < 1
317     lRet := .F.
318 Else
319     If cTime > aHora[1] .And. cTime < aHora[2]
320         lRet := .T.
321     EndIf
322 EndIf
323
324 RestArea(aArea)
325
326 Return lRet
327
328
329 //-----
330 /*{Protheus.doc} STBValPagto
331 Function Valida se a Regra é do tipo Pagamento
332
333 @param    cCodRule      Código da regra de desconto
334 @return   lRet          Retorna se a regra é do tipo pagamento
335 @obs      As condicoes foram retiradas porque elas
336           nao validavam com exatidao, ou seja, so
337           validava se o codigo da regra existia na
338           MB4 e nao a Forma de pagto selecionada
339           era igual. 04/08/2014
340 @sample
341 */
342 //-----
343 Static Function STBValPagto( cCodRule, lPgto )
344
345 Local aArea          := GetArea()          // Guarda Alias corrente
346 Local lRet           := .F.                // Retorno da Funcao
347
348 Default cCodRule     := ""
349 Default lPgto        := .F.
350
351 ParamType 0 var cCodRule As Character Default ""
352
353 /*
354     Admite como sendo por PAGAMENTO se tiver encontrado registro
355     referente a aba/guia de NEGOCIACAO do cadastro da regra.
356 */
357
358 DbSelectArea("MB2")
359 DbSetOrder(1) //MB2_FILIAL+MB2_CODREG+MB2_REFGRD
360 If DbSeek(xFilial("MB2") + cCodRule)
361     If MB2->MB2_PGT0 == "1">//Considera Forma de pagamento
362         If MB2->MB2_TPREGR == "2" .Or. MB2->MB2_TPREGR == "3">//Brinde ou Bonificacao
363             If lPgto
364                 lRet := .T. //Regra por Total apos Finalizar a Venda

```

```

365         Else
366             lRet := .F. //Porque a validacao da forma de pagto é feita no OnClik do objeto no momento de selecionar a Forma de pagamento
367         Endif
368     Else
369         if isInCallStack("STBPromPgto") .OR. isInCallStack("STIAddNewPan")
370             lRet := .T. //retona .T. para considerar a regra por forma de pagamento,mas so se vier do click da forma de pagamento
371         EndIf
372     Endif
373 Else
374     lRet := .T.
375 Endif
376 Endif
377
378 MB2->(DbCloseArea())
379
380 RestArea( aArea )
381
382 Return lRet
383
384
385 //-----
386 /*{Protheus.doc} STDValRule
387 Function Validacao da Regra de Desconto
388
389 @param  cCliCode      Código do cliente
390 @param  cCliStore     Loja do cliente
391 @return lRet          Retorna se a regra é válida
392 @obs    Verifica condicoes de Filial, Hora/Data/Semana e Cliente. Nao verifica o Range de Valores
393 @sample
394 */
395 //-----
396 Function STDValRule( cCliCode , cCliStore )
397
398 Local aArea          := GetArea()          // Alias Corrente
399 Local lRet           := .T.                // Retorna se a regra eh valida
400 Local aFilMB3        := {}                // Filiais da regra de desconto
401 Local lAI0_CLIFUN    := SA1->(ColumnPos("AI0_CLIFUN")) > 0 //Importante: esta validação deve ser mantida, pois este campo existe no compatibilizador da 11.80 e não
                        existe no dicionário da 12
402 Local cCliFun        := "" //Cliente Funcionario?
403 Local lAI00_CLIFU    := AI0->(ColumnPos("AI00_CLIFUN")) > 0 //Foi protegido este campo pois este programa vai subir na release 12.1.16 e campo na posterior
404
405 Default cCliCode     := ""
406 Default cCliStore    := ""
407
408 ParamType 0 var  cCliCode    As Character    Default ""
409 ParamType 1 var  cCliStore   As Character    Default ""
410
411 /*/
412     ATIVA
413 /*/
414 If lRet
415     If (MEI->MEI_ATIVA <> "1")

```

```

416         lRet := .F.
417     EndIf
418 EndIf
419 /*/
420     CLIENTE LOJA
421 /*/
422 If lRet
423     If !Empty(AllTrim( MEI->MEI_CODCLI + MEI->MEI_LOJA )) .AND. ( AllTrim( MEI->MEI_CODCLI + MEI->MEI_LOJA ) <> AllTrim( cCliCode + cCliStore ) )
424         lRet := .F.
425     EndIf
426 EndIf
427
428 /*/
429     GRUPO CLIENTE
430 /*/
431 If lRet
432     If !Empty(AllTrim(MEI->MEI_GRPVEN)) .AND. !lDescGrupo
433         DbSelectArea("SA1")
434         DbSetOrder(1)
435         If DbSeek(xFilial("SA1")+ cCliCode + cCliStore)
436             If SA1->A1_GRPVEN <> MEI->MEI_GRPVEN
437                 lRet := .F.
438                 lDescGrupo := .F.
439             Else
440                 If !Empty(SA1->A1_GRPVEN) .AND. AllTrim(SA1->A1_GRPVEN) == AllTrim(MEI->MEI_GRPVEN)
441                     lDescGrupo := .T.
442                 EndIf
443             Endif
444         Endif
445     Endif
446 EndIf
447
448 /*/
449     Se a regra for por desconto de funcionario, porem no cad de cliente nao estiver como desconto de funcionario nao se aplica a regra
450 /*/
451
452 If lRet
453     If (MEI->MEI_DESFUN == "S")
454         If lAI0_CLIFU
455             cCliFun := GetAdvFVal("AI0","AI0_CLIFUN",xFilial("AI0")+cCliCode + cCliStore,1,"2")
456         EndIf
457
458         If Empty(cCliFun) .AND. lA1_CLIFUN
459             cCliFun := GetAdvFVal("SA1","A1_CLIFUN",xFilial("SA1")+cCliCode + cCliStore,1,"2")
460         EndIf
461         If !(cCliFun == "1")
462             lRet := .F.
463         EndIf
464     EndIf
465 EndIf
466 EndIf
467

```

```

468 /*/
469     DIA, DATA E HORA
470 /*/
471 If lRet
472     If !(STDValidPer( MEI->MEI_DATDE , MEI->MEI_DATATE , MEI->MEI_CODREG ))
473         lRet := .F.
474     EndIf
475 EndIf
476
477 /*/
478     FILIAL
479     Valida tambem Grupo de Filiais
480 /*/
481 If lRet
482
483     DbSelectArea("MB3")
484     DbSetOrder(3) // MB3_FILIAL+MB3_CODREG+MB3_TIPO+MB3_CODGRU+MB3_CODEMP+MB3_CODFIL
485
486     If DbSeek( xFilial("MB3") + MEI->MEI_CODREG )
487
488         While MB3->( !Eof() .And. MB3_FILIAL+MB3_CODREG == xFilial("MB3")+MEI->MEI_CODREG )
489
490             // 1. Tipo Filial
491             // 2. Tipo Grupo de Filiais
492
493             IF MB3->MB3_TIPO == "1"
494
495                 If MB3->MB3_CODEMP == SubStr(cNumEmp,1,2)
496
497                     aAdd( aFilMB3 , MB3->MB3_CODFIL )
498
499                 Endif
500
501             ElseIf MB3->MB3_TIPO == "2"
502
503                 // Explode o grupo de filiais
504                 SAU->(dbSetOrder(1))
505                 IF SAU->( dbSeek( xFilial("SAU") + MB3->MB3_CODGRU ) )
506
507                     While SAU->( !EOF() .And. SAU->AU_FILIAL+RTrim(SAU->AU_CODGRUP) == xFilial("SAU")+RTrim(MB3->MB3_CODGRU) )
508
509                         If Ascan( aFilMB3 , SAU->AU_CODFIL ) == 0
510                             aAdd( aFilMB3 , SAU->AU_CODFIL )
511                         Endif
512                         SAU->( DbSkip() )
513
514                     End
515
516                 Endif
517
518             Endif
519

```

```

520         MB3->( DbSkip() )
521
522     End
523
524
525     // Verifica se a filial esta entre as filiais permitidas
526     If Ascan( aFilMB3 , AllTrim( FWCodFil() ) ) == 0
527         lRet := .F.
528     Endif
529
530     Else
531         lRet := .F.
532     EndIf
533
534 EndIf
535
536 /*/
537     PRIORIDADE
538 /*/
539 If lRet
540     DbSelectArea("MEJ")
541     DbSetOrder(2) //MEJ_FILIAL+MEJ_CODREG
542     If !(DbSeek(xFilial("MEJ") + MEI->MEI_CODREG))
543         lRet := .F.
544     EndIf
545 EndIf
546
547 RestArea( aArea )
548
549 Return lRet
550
551
552 //-----
553 /*/{Protheus.doc} STDRangRul
554 Function Busca o(s) Range(s) da Regra de Desconto
555
556 @param    cCodRule      Código da regra de desconto
557 @return   aRet[1] nMB2_VLRINI      1 Valor inicial
558 @return   aRet[2] nMB2_VLRFIM      2 Valor final
559 @return   aRet[3] nMB2_PDESC       3 Percentual de desconto
560 @return   aRet[4] nMB2_VLRFX       4 Valor de desconto
561 @obs
562 @sample
563 /*/
564 //-----
565 Function STDRangRul( cCodRule, cTipo )
566
567 Local aArea      := GetArea()      // Guarda Alias corrente
568 Local aRet       := {}             // Armazena o(s) Range(s) da Regra de Desconto
569
570 Default cCodRule := ""
571 Default cTipo    := ""

```

```

572
573 ParamType 0 var cCodRule As Character Default ""
574
575 DbSelectArea("MB2")
576 DbSetOrder(1) //MB2_FILIAL+MB2_CODREG+MB2_REFGRD
577 If DbSeek(xFilial("MB2") + cCodRule)
578     While !MB2->(Eof()) .AND. (MB2->MB2_CODREG == cCodRule)
579         If Empty(cTipo)
580             AADD( aRet, { MB2->MB2_VLRINI , MB2->MB2_VLRFIM , MB2->MB2_PDESC , MB2->MB2_VLRFX, ""/*MB2->MB2_IDPROD*/,MB2->MB2_TPREG,MB2_REFGRD } )
581         Else
582             If MB2->MB2_TPREG == cTipo .Or. MB2->MB2_TPREG <> "1"// Desconto não é contemplado para manter o legado ja existe
583             AADD( aRet, { MB2->MB2_VLRINI , MB2->MB2_VLRFIM , MB2->MB2_PDESC , MB2->MB2_VLRFX, ""/*MB2->MB2_IDPROD*/,MB2->MB2_TPREG,MB2_REFGRD } )
584             Endif
585         Endif
586     MB2->(DbSkip())
587 EndDo
588 EndIf
589
590 RestArea( aArea )
591
592 Return aRet
593
594
595 //-----
596 /*{Protheus.doc} STDItRang
597 Function Busca Range que possa ser aplicado para o valor do Item
598
599 @param cCodRule      Código da regra de desconto
600 @param nValueItem    Valor do Item
601 @return aRet[1] nMB2_VLRINI      1 Valor inicial
602 @return aRet[2] nMB2_VLRFIM      2 Valor final
603 @return aRet[3] nMB2_PDESC       3 Percentual de desconto
604 @return aRet[4] nMB2_VLRFX       4 Valor de desconto
605 @obs
606 @sample
607 */
608 //-----
609 Function STDItRang( cCodRule , nValueItem )
610
611 Local aArea      := GetArea()           // Alias Corrente
612 Local aRet       := {}                  // Retorno funcao
613
614 Default cCodRule := ""
615 Default nValueItem := 0
616
617 ParamType 0 var cCodRule As Character Default ""
618 ParamType 0 var nValueItem As Numeric Default 0
619
620 DbSelectArea("MB2")
621 DbSetOrder(1) //MB2_FILIAL+MB2_CODREG+MB2_REFGRD
622 If DbSeek(xFilial("MB2") + cCodRule )
623     While !MB2->(Eof()) .AND. (MB2->MB2_CODREG == cCodRule)

```

```

624      If ( MB2->MB2_VLRINI < nValueItem ) .AND. ( MB2->MB2_VLRFIM > nValueItem ) // Valida Range
625          AADD( aRet, { MB2->MB2_VLRINI , MB2->MB2_VLRFIM , MB2->MB2_PDESC , MB2->MB2_VLRFIX, MB2->MB2_TPREGR, MB2->MB2_REFGRD } )
626      EndIf
627      MB2->(DbSkip())
628  EndDo
629 EndIf
630
631 RestArea( aArea )
632
633 Return aRet
634
635
636 //-----
637 /*{Protheus.doc} STDPayForms
638 Function Busca a(s) Forma(s) de pagamento da Regra de Desconto
639
640 @param  cCodRule      Código da regra de desconto
641 @param  cForma        Forma de pagamento selecionada na venda
642 @return aRet          Retorna as formas de pagamento cadastradas para a regra de desconto
643 @obs
644 @sample
645 */
646 //-----
647 Function STDPayForms( cCodRule, cForma )
648
649 Local aArea      := GetArea()    // Armazena alias corrente
650 Local aRet       := {}          // Retorno funcao
651 Local nI         := 0           // Contador
652
653 Default cCodRule := ""          // Código da regra de desconto varejo
654 Default cForma   := ""          // Forma de pagamento selecionada na venda
655
656 ParamType 0 Var  cCodRule As Character Default ""
657
658 lFrmDesc := .F. // Setamos falso pois ela é chamada em diversas vezes e estava apresentando .T. de forma errada.
659
660 DbSelectArea("MB4")
661 DbSetOrder(1)//MB4_FILIAL+MB4_CODREG+MB4_CODFRM
662 If DbSeek(xFilial("MB4") + cCodRule)
663     While !MB4->(Eof()) .AND. ( MB4->MB4_CODREG == cCodRule )
664         AADD( aRet , AllTrim(MB4->MB4_CODFRM) )
665         DbSkip()
666     EndDo
667 EndIf
668
669 RestArea(aArea)
670
671 If !Empty(aRet)
672     For nI := 1 To Len(aRet)
673         If AllTrim(aRet[nI]) == AllTrim(cForma)
674             lFrmDesc := .T.
675         EndIf

```

```

676     Next
677 EndIf
678
679 Return  aRet
680
681
682 //-----
683 /*{Protheus.doc} STDPropForm
684 Function Verifica se a Regra de Desconto é proporcional ou não
685
686 @param  cCodRule      Código da regra de desconto
687 @return lRet          Retorna as formas de pagamento cadastradas para a Regra de Desconto
688 @obs
689 @sample
690 */
691 //-----
692 Function STDPropForm( cCodRule )
693
694 Local aArea          := GetArea()    // Armazena alias corrente
695 Local lRet           := .F.         // Retorno funcao
696
697 Default cCodRule     := ""
698
699 ParamType 0 Var  cCodRule As Character Default ""
700
701 DbSelectArea("MB4")
702 DbSetOrder(1)//MB4_FILIAL+MB4_CODREG+MB4_CODFRM
703 If DbSeek(xFilial("MB4") + cCodRule )
704     lRet := ( MB4->MB4_PROPOR == "1" )
705 EndIf
706
707 RestArea(aArea)
708
709 Return  lRet
710
711
712 //-----
713 /*{Protheus.doc} STDValMoSa
714 Function Valida se a Regra de Desconto é válida para Saldão/Mostruário
715
716 @param  cCodRule      Código da regra de desconto
717 @param  cTypeProd     Tipo Saçdão ou Mostruário
718 @return lRet          Retorna se a regra é válida para produto saldão/mostruário
719 @obs
720 @sample
721 */
722 //-----
723 Static Function STDValMoSa( cCodRule , cTypeProd )
724
725 Local lRet           := .T.         // Retorno da funcao
726 Local aArea          := GetArea()    // Armazena Alias Corrente
727 Local aAreaMEI       := MEI->(GetArea()) // Armazena Area MEI

```

```

728
729 Default cCodRule      := ""
730 Default cTypeProd     := ""
731
732 ParamType 0 Var  cCodRule As Character Default ""
733 ParamType 0 Var  cTypeProd As Character Default ""
734
735 DbSelectArea("MEI")
736 DbSetOrder(1)//MEI_FILIAL+MEI_CODREG
737 If DbSeek(xFilial("MEI") + cCodRule )
738     If ((MEI->MEI_PRDMOS == "S") .And. cTypeProd <> "S") .Or. ((MEI->MEI_PRDMOS == "M") .AND. cTypeProd <> "M")
739         lRet := .F.
740     EndIf
741 EndIf
742
743 RestArea(aAreaMEI)
744 RestArea(aArea)
745
746 Return lRet
747
748
749 //-----
750 /*{Protheus.doc} STDPriority
751 Function Busca Prioridade da Regra
752
753 @param  cCodRule      Código da regra de desconto
754 @return cPriority      Retorna prioridade da regra
755 @obs
756 @sample
757 */
758 //-----
759 Static Function STDPriority( cCodRule )
760
761 Local aArea      := GetArea()           // Armazena alias corrente
762 Local aAreaMEJ   := MEJ->(GetArea())    // Armazena Posicao MEJ
763 Local cPriority   := ""                 // Retorno funcao
764
765 Default cCodRule := ""
766
767 ParamType 0 Var  cCodRule As Character Default ""
768
769 If !Empty( cCodRule )
770     DbSelectArea("MEJ")
771     DbSetOrder(2)//MEJ_FILIAL+MEJ_CODREG
772     If DbSeek( xFilial("MEJ") + cCodRule )
773         cPriority := MEJ->MEJ_PRINUM
774     EndIf
775 EndIf
776
777 RestArea(aAreaMEJ)
778 RestArea(aArea)
779

```

```

780 Return cPriority
781
782
783 //-----
784 /*{Protheus.doc} STDNextSale
785 Function Verifica se a Regra é Desconto para P'xima venda
786
787 @param   cCodRule      Código da regra de desconto
788 @return  lRet          Retorna se é desconto na próxima venda
789 @obs
790 @sample
791 */
792 //-----
793 Function STDNextSale( cCodRule )
794
795 Local aArea      := GetArea()           // Armazena alias corrente
796 Local aAreaMEI   := MEI->(GetArea()) // Armazena area MEI
797 Local lRet       := .F.                 // Retorna se a regra eh para desconto na proxima venda
798
799 Default cCodRule := ""
800
801 ParamType 0 Var cCodRule As Character Default ""
802
803 DbSelectArea("MEI")
804 DbSetOrder(1)//MEI_FILIAL+MEI_CODREG
805 If DbSeek(xFilial("MEI") + cCodRule )
806     lRet := MEI->MEI_DESCPR == "2"
807 EndIf
808
809 RestArea( aAreaMEI )
810 RestArea( aArea )
811
812 Return lRet
813
814
815 //-----
816 /*{Protheus.doc} STDRuleDate
817 Function Busca a Data Final de Validade da Regra
818
819 @param   cCodRule      Código da regra de desconto
820 @return  dRet          Retorna se é desconto na próxima venda
821 @obs
822 @sample
823 */
824 //-----
825 Function STDRuleDate( cCodRule )
826
827 Local aArea      := GetArea()           // Armazena alias corrente
828 Local aAreaMEI   := MEI->(GetArea()) // Armazena area MEI
829 Local dRet       := CToD("")           // Data de validade da Regra de Desconto
830
831 Default cCodRule := ""

```

```

832
833 ParamType 0 Var  cCodRule As Character Default ""
834
835 DbSelectArea("MEI")
836 DbSetOrder(1)//MEI_FILIAL+MEI_CODREG
837 If DbSeek(xFilial("MEI") + cCodRule )
838     dRet := MEI->MEI_DATATE
839 EndIf
840
841 RestArea( aAreaMEI )
842 RestArea( aArea )
843
844 Return dRet
845
846 //-----
847 /*{Protheus.doc} STDProdDA1
848 Funcao responsavel por verificar todos os produtos que existem em determinada
849 tabela de preco
850
851 @param  cTabPrc          Código da tabela de preco
852 @return
853 @obs
854 @sample
855 */
856 //-----
857 Function STDProdDA1(cTabPrc, oCesta)
858
859 Local nI := 0 //Variavel de loop
860 Local nTotal := 0 //Valor total para conceder o desconto
861
862 For nI := 1 To oCesta:Length()
863     oCesta:GoLine(nI)
864     If !oCesta:IsDeleted(nI) .AND. AllTrim(oCesta:GetValue("L2_CODTAB")) == AllTrim(cTabPrc)
865         nTotal += oCesta:GetValue("L2_VLRITEM")
866     EndIf
867 Next nI
868 nTotDal := nTotal
869
870 Return nTotal
871
872 //-----
873 /*{Protheus.doc} STDGetDA1
874 Funcao responsavel em retornar o valor total dos produtos para condecer o
875 desconto
876
877 @param
878 @return  nTotDal - Total dos produtos
879 @obs
880 @sample
881 */
882 //-----
883 Function STDGetDA1()

```

```

884 Return nTotDa1
885
886
887 //-----
888 /*{Protheus.doc} STDAdmFn
889 Função para retornar Administradora Financeira (MB6)
890 envolvido na Regra de Desconto Varejo utilizada
891
892 @param   cCodReg - Código da Regra de Desconto
893 @return  array com administradora financeira (MB6)
894 @obs
895 @sample
896 */
897 //-----
898 Function STDAdmFn(cCodReg)
899 Local aArea := GetArea()
900 Local aRet := {}
901
902 Default cCodReg := ""
903
904 DbSelectArea("MB6")
905 MB6->( DbSetOrder(1) ) //MB6_FILIAL+MB6_CODREG+MB6_ADMFIN
906 If MB6->( DbSeek(xFilial("MB6") + cCodReg) )
907     While MB6->(!Eof()) .AND. MB6->MB6_FILIAL + MB6->MB6_CODREG == xFilial("MB6") + cCodReg
908         AADD(aRet, MB6->MB6_ADMFIN )
909         MB6->( DbSkip() )
910     EndDo
911 EndIf
912
913 RestArea(aArea)
914 Return aRet
915
916 /*{Protheus.doc} STDGetGrp
917 Retorna o conteúdo da variavel lDescGrupo
918 @type Function
919 @param
920 @return lDescGrupo, logico, variavel que contém o conteúdo .T. se for desconto por grupo de cliente ou .F. para quando não é desconto por grupo.
921 */
922 Function STDGetGrp()
923 Return lDescGrupo
924
925 /*{Protheus.doc} STDSetGrp
926 Seta o valor para a variavel lDescGrupo limpando ao final de cada venda
927 @type Function
928 @param lValue, logico, seta um default para a variavel lDescGrupo
929 @return
930 */
931 Function STDSetGrp(lValue)
932 Default lValue := .F.
933 lDescGrupo := lValue
934 Return
935

```

```

936 /*/{Protheus.doc} STDGetFrm
937 Função para retornar se a forma de pagamento Existe dentro da regra de desconto
938 @return lFrmDesc, logico, variavel que contém o conteúdo .T. se existir e .F. se não existir.
939 @obs
940 @sample
941 */
942 //-----
943
944 Function STDGetFrm()
945 Return lFrmDesc
946
947 /*/{Protheus.doc} STDSetFrm
948 Função para setar se a forma de pagamento Existe dentro da regra de desconto
949 na variavel lFrmDesc limpando ao final de cada venda
950 @param lValue, logico, seta um default para a variavel
951 @return lValue, logico
952 @obs
953 @sample
954 */
955 //-----
956
957 Function STDSetFrm(lValue)
958 Default lValue := .F.
959 lFrmDesc := lValue
960 Return
961
962
963 //lDscTotSPg
964
965 /*/{Protheus.doc} STDGetDTSPg
966     Regra de Desconto Varejo no Total Sem Forma de Pagamento e Sem Grupo de Cliente
967     @type Function
968     @return lDscTotSPg, lógico, Regra de Desconto Varejo no Total Sem Forma de Pagamento e Sem Grupo de Cliente
969     @example
970     (examples)
971     @see (links_or_references)
972     */
973 Function STDGetDTSPg()
974 Return lDscTotSPg
975
976 /*/{Protheus.doc} STDSetDTSPg
977     Regra de Desconto Varejo no Total Sem Forma de Pagamento e Sem Grupo de Cliente
978     @type Function
979     @param lValue, lógico, Regra de Desconto Varejo no Total Sem Forma de Pagamento e Sem Grupo de Cliente
980     */
981 Function STDSetDTSPg(lValue)
982 Default lValue := .F.
983 lDscTotSPg:= lValue
984 Return

```

Componentes/Discount/STWDiscountTotalSale.PRW

55 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STWTotDisc
6 Function Desconto no Total da Venda
7
8 @param  nValDesc      Valor do Desconto
9 @param  cType         Tipo do Desconto. Se P - Percentual ou V - Valor.
10 @param  cOrigin       Define a procedência do desconto
11 @return lRet         Retorna se efetuou desconto no total
12 @obs
13 @sample
14 */
15 //-----
16 Function STWTotDisc( nValDesc , cType , cOrigin , lAccumulate, aDescOrc)
17
18 Local      lRet          := .T.                // Retorno se efetuou desconto
19
20 Default    nValDesc      := 0                  // Valor do desconto
21 Default    cType         := ""                 // Tipo do Desconto. Se P - Percentual ou V - Valor.
22 Default    cOrigin       := ""                 // Define a procedência do desconto
23 Default    lAccumulate   := .F.                 // Acumula com descontos anteriores
24 Default    aDescOrc      := {0,0}              // Desconto no orçamento (Caso Exista)
25
26 ParamType 0 Var nValDesc AS Numeric   Default 0
27 ParamType 1 Var cType   AS Character  Default ""
28 ParamType 2 Var cOrigin AS Character  Default ""
29
30 If lRet
31   lRet := STBValDisc( nValDesc + Iif(cType == "V",aDescOrc[1],aDescOrc[2]) , cType , cOrigin, aDescOrc) // Valida o valor do desconto
32 EndIf
33
34 // TODO: Zerar descontos ao dar esc na tela pois o desconto sendo concomitante ja foi pra impressora
35 // If nVlrDesc > 0 .AND. LjAnalisaLeg(14)[1]
36 // analisaLeg(22) verificar se eh a mesma coisa!!!!
37
38 If lRet
39   STBTotDiscApply( nValDesc , cType , cOrigin , lAccumulate )
40
41   // Indica que foi concedido Desconto no Total da Venda pelo PDV
42   STBSetDiscTotPDV(.T.)
43
44   /*
45   Se um desconto no total for aplicado, o Caixa poderá alterar as parcelas,
46   já que o valor da Venda foi alterado
47   */
48   If FindFunction("STIGetPayR0") .AND. STIGetPayR0()
49     STISetPayR0(-1) //permite editar os pagamentos
50   EndIf
51 EndIf
52

```

```
53 STFSHOWMESSAGE("TotDesc")
54
55 Return lRet
```

Componentes/Discount/STWItemDiscount.PRW

381 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWITEMDISCOUNT.CH"
5
6 static lAtivMotor := ExistFunc("PshAtivMTP") .AND. PshAtivMTP()
7
8 //-----
9 /*{Protheus.doc} STWItemDiscount
10 Função desconto no item
11
12 @param nItemLine      Numero do item na venda
13 @param nValDisc       Valor do Desconto
14 @param cTypeDisc      Tipo do desconto. P - Percentual , V - Valor
15 @param cTypeItem      Tipo do Item. "KIT" - Item de KIT , "IMOP" - Item de Importação de Orçamento
16 @param lItemFiscal    Indica se o Item é fiscal. Impresso em cupom fiscal
17 @param cTime          Configuração se desconto antes ou depois de registrar o item
18 @param nQuantProd     numerico, Quantidade do produto na venda
19 @return lRet          Retorna se aplicou desconto
20 @obs
21 @sample
22 */
23 //-----
24 Function STWItemDiscount( nItemLine , nValDisc , cTypeDisc , cTypeItem , ;
25                          lItemFiscal , cTime , nQuantProd )
26
27 Local lRet          := .F.      // Retorna se Aplicou desconto
28 Local lContinue     := .T.      // Fluxo função
29 Local aRuleDiscount := {}       // Armazena retorno regra de desconto por item
30 Local aDiscount     := {}       // Armazena Desconto via Usuário
31 Local cConfig       := ""       // Configuração se utiliza desconto via usuário, via regra ou ambos
32 Local cCodSup       := ""       // Código do Supervisor que liberou o desconto
33 Local lCRdesItTt    := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMV("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
34
35 Default nItemLine    := 0
36 Default nValDisc     := 0
37 Default cTypeDisc    := ""
38 Default cTypeItem    := ""
39 Default lItemFiscal  := .F.
40 Default cTime        := ""
41 Default nQuantProd   := 0
42
43 ParamType 0 Var nItemLine As Numeric Default 0
44 ParamType 1 Var nValDisc As Numeric Default 0
45 ParamType 2 Var cTypeDisc As Character Default ""
46 ParamType 3 Var cTypeItem As Character Default ""
47 ParamType 4 Var lItemFiscal As Logical Default .F.
48 ParamType 5 Var cTime As Character Default ""
49
50 If !lAtivMotor .Or. (nValDisc == 0 .And. STBGetItDiscount())[1] == 0)

```

```

51  /*/
52      Item de importação de orçamento
53  /*/
54  If Upper(cTypeItem) == "IMP"
55
56      /*/
57          Validações
58      /*/
59      If ( nValDisc <= 0 ) .OR. ( cTypeDisc <> "V" ) .OR. ( nItemLine <= 0 )
60          lContinue := .F.
61      EndIf
62
63
64      /*/
65          Para importação de orçamento, é passado tipo de desconto por Valor,
66          Por isso faço um TaxAlt direto
67      /*/
68      If lContinue
69          STBTaxAlt( "IT_DESCONTO" , nValDisc , nItemLine )
70          lRet := .T.
71      EndIf
72
73  /*/
74      Item tipo KIT
75  /*/
76  ElseIf AllTrim(Upper(cTypeItem)) $ "KIT|BON"
77
78      lRet := STBIDApply(      nItemLine      , nValDisc      , cTypeDisc      , lItemFiscal      , ;
79                          "A"                                )
80  Else
81
82      /*/
83          Get da configuração de desconto no item:
84          "U" - Usuário
85          "R" - Regra de Desconto
86          "A" - Ambos
87      /*/
88      cConfig := STDIDUseDiscountFrom()
89
90      Do Case
91
92          /*/
93              Apenas via usuário
94          /*/
95          Case cConfig == "U"
96
97              //Busca desconto do usuario
98              aDiscount := STBGetItDiscount()
99
100              If aDiscount[1] > 0 //se usuario digitou desconto, considera ele
101                  lRet := STBIDUser( nItemLine , cTime , lItemFiscal )
102              Else

```

```

103         //se nao digitou desconto , aplica regra de desconto varejo.
104         aRuleDiscount := STWRuleItem( nItemLine )
105
106         If aRuleDiscount[LRULEAPPLY]
107
108             lRet := STBIDApply( nItemLine , aRuleDiscount[NDESCTOTAL] , "V" , lItemFiscal , "A" )
109
110             If lRet
111
112                 STWRsnDesc( nItemLine , STR0001 , STR0002 ) // "REGRA DE DESCONTO"
113
114             EndIf
115
116         Else
117
118             lRet := .F. // Desconto nenhum foi aplicado
119
120         EndIf
121
122     EndIf
123
124     /*/
125     Apenas via regra de desconto
126     /*/
127     Case cConfig == "R"
128
129         aRuleDiscount := STWRuleItem( nItemLine )
130
131         If aRuleDiscount[LRULEAPPLY]
132
133             lRet := STBIDApply( nItemLine , aRuleDiscount[NDESCTOTAL] , "V" , lItemFiscal , "A" )
134
135             If lRet
136
137                 STWRsnDesc( nItemLine , STR0001 , STR0002 ) // "REGRA DE DESCONTO"
138
139             EndIf
140
141         Else
142
143             lRet := .F. // Desconto nenhum foi aplicado
144
145         EndIf
146
147     /*/
148     Ambos
149     /*/
150     Case cConfig == "A"
151
152         If !lCRdesItTt .OR. ( lCRdesItTt .AND. nQuantProd > 0 )
153             /*/
154             Aplica primeiro a de Regra de desconto, depois a de usuário, no formato desconto sobre desconto

```

```

155      /*/
156      aRuleDiscount := STWRuleItem( nItemLine, , , nQuantProd )
157
158  EndIf
159
160  If (ValType(aRuleDiscount) == "A" .AND. Len(aRuleDiscount) > 0 ) .AND. aRuleDiscount[LRULEAPPLY]
161
162      /*/
163      Get do desconto dado via usuário
164      /*/
165      aDiscount := STBGetItDiscount()
166
167      If Empty(cTime)
168
169          cTime := STDIDTime()
170
171      EndIf
172
173      /*/
174      Se vier desconto via usuário,
175      Validar o desconto de usuário em relação ao novo total*.
176      Total descontado da Regra de Desconto
177      /*/
178      lRet := STBIDValid( nItemLine , aDiscount[1] , aDiscount[2] , .T. , ;
179                      cTime , aRuleDiscount[NNEWTOTAL] )
180
181      If lRet
182
183          lRet := STBIDBothApply( nItemLine , aRuleDiscount , aDiscount , lItemFiscal , cTime )
184
185      Else
186
187          If STBTaxRet( nItemLine , "IT_DESCONTO" ) <= 0 // Caso não tenha sido aplicado desconto nenhm, aplica o desconto de Regra apenas.
188              lRet := STBIDApply( nItemLine , aRuleDiscount[NDESCTOTAL] , "V" , lItemFiscal , "A" )
189              If lRet
190                  If ( aDiscount[1] > 0 ) .AND. ( aDiscount[2] $ "V|P" ) // Indica intenção de desconto via usuário
191                      STFMessage("STItemDiscount","ALERT",STR0003) //"Foi aplicado apenas desconto via Regra de Deconto"
192                  EndIf
193              EndIf
194          EndIf
195
196          If ( aDiscount[1] > 0 ) .AND. ( aDiscount[2] $ "V|P" ) // Indica intenção de desconto via usuário
197              STFMessage("STItemDiscount","ALERT",STR0004) //"Desconto não aplicado"
198          EndIf
199
200          STFShowMessage("STItemDiscount")
201
202      EndIf
203
204  Else
205      lRet := STBIDUser( nItemLine , cTime , lItemFiscal, @cCodSup )
206  EndIf

```

```

207
208     EndCase
209
210     EndIf
211
212     If lRet .AND. Upper(cTypeItem) != "IMP"
213         If cConfig <> "R" // Para regra de desconto já chamado(sem interface) logo após aplicar a regra
214             STWRsnDesc( nItemLine , , , , cCodSup ) // Chamada Motivo de Desconto
215         EndIf
216     EndIf
217 Else
218     lRet := .F.
219     STFMessage("STFMTPRO","POPUP", STR0008 )// "Não é Possível Aplicar Desconto Manual, o Motor de Promoções está ativado"
220     STFShowMessage("STFMTPRO")
221 EndIf
222
223 Return lRet
224
225
226 //-----
227 /*{Protheus.doc} STWIDAfter
228 Avalia desconto aplicado via usuário depois do item
229
230 @param  nDiscount      Valor para desconto
231 @param  cTypeDisc      Tipo de desconto, se "V" - Valor ou "P" - Percentual
232 @return lRet           Retorna se Efetuiu o Desconto
233 @obs
234 @sample
235 */
236 //-----
237 Function STWIDAfter( nDiscount , cTypeDisc )
238
239 Local lRet           := .F.           // Retorna se efetuou o desconto
240 Local cTime          := ""           // Configuração se desconto antes ou depois de registrar o item
241 Local cConfig        := ""           // Configuração de desconto se via usuário, regra ou ambos
242 Local nItemLine      := 0           // Linha do item
243 Local lItemFiscal    := .F.         // Indica se o item deve ser registrado em impressora fiscal
244
245 Default nDiscount    := 0
246 Default cTypeDisc    := ""
247
248 ParamType 0 Var  nDiscount      As Numeric      Default 0
249 ParamType 1 Var  cTypeDisc     As Character    Default ""
250
251 /*/
252     Get configuração se desconto antes ou depois de registrar o item
253 /*/
254 cTime := STDIDTime()
255
256 /*/
257     Get da Configuração de desconto se via usuário, regra ou ambos
258 /*/

```

```

259 cConfig := STDIDUseDiscountFrom()
260
261 If cConfig <> "R" // Via Regra de Desconto
262
263     If cTime <> "D" // Desconto depois de registrado o Item
264
265         If nDiscount >= 0 .AND. !Empty(cTypeDisc) // Indica intenção de desconto
266
267             STFMessage("STWIDAAfter","ALERT", STR0005) //"A configuração nao permite desconto após o item"
268
269         EndIf
270
271     Else
272
273         nItemLine := STDPBLength("SL2") // Ultimo item
274
275         If nItemLine > 0
276
277             /*/
278             Set Desconto
279             /*/
280             STBSetItDiscount( nDiscount , cTypeDisc )
281
282             lItemFiscal := STDGPBasket( "SL2" , "L2_FISCAL" , nItemLine )
283
284             lRet := STWItemDiscount( nItemLine , Nil , Nil , "" , lItemFiscal , "D" )
285
286         EndIf
287
288     EndIf
289
290 Else
291
292     If nDiscount >= 0 .AND. !Empty(cTypeDisc) // Indica intenção de desconto
293
294         STFMessage("STWIDAAfter","ALERT", STR0006) //"A configuração não permite desconto via usuário"
295
296     EndIf
297
298 EndIf
299
300 STFShowMessage("STWIDAAfter")
301
302 /*/
303 Set default desconto usuário
304 /*/
305 STBDefItDiscount()
306
307 Return lRet
308
309
310 //-----

```

```

311 /*{Protheus.doc} STWIDBefore
312 Avalia desconto aplicado via usuário antes/junto do item
313
314 @param  nDiscount      Valor para desconto
315 @param  cTypeDisc      Tipo de desconto, se "V" - Valor ou "P" - Percentual
316 @return Nil
317 @obs
318 @sample
319 */
320 //-----
321 Function STWIDBefore( nDiscount , cTypeDisc, cTime )
322
323 //Local cTime      := ""          // Configuração se desconto antes ou depois de registrar o item
324 Local cConfig      := ""          // Configuração de desconto se via usuário, regra ou ambos
325
326 Default nDiscount  := 0
327 Default cTypeDisc   := ""
328 Default cTime       := "" // Configuração se desconto antes ou depois de registrar o item
329
330 ParamType 0 Var  nDiscount      As Numeric      Default 0
331 ParamType 1 Var  cTypeDisc      As Character    Default ""
332 ParamType 2 Var  cTime          As Character    Default ""
333
334 /*/
335   Get configuração se desconto antes ou depois de registrar o item
336 /*/
337 If Empty(cTime)
338   cTime := STDIDTime()
339 EndIf
340
341 /*/
342   Get da Configuração de desconto se via usuário, regra ou ambos
343 /*/
344 cConfig := STDIDUseDiscountFrom()
345
346 If cConfig <> "R" // Via Regra de Desconto
347
348   If cTime <> "A" // Antes|Junto com registro de Item
349
350     If nDiscount > 0 .AND. !Empty(cTypeDisc) // Indica intenção de desconto
351
352       STFMessage("STWIDBefore","ALERT", STR0007) //"A configuração não permite desconto antes do item"
353
354     EndIf
355
356   Else
357
358     /*/
359       Set Desconto
360     /*/
361     STBSetItDiscount( nDiscount , cTypeDisc )
362

```

```
363      /*/  
364      Após o set, o desconto deve ser efetuado via chamado do registro de item  
365      /*/  
366  
367      EndIf  
368  
369  Else  
370  
371      If nDiscount > 0 .AND. !Empty(cTypeDisc) // Indica intenção de desconto  
372  
373          STFMessage("STWIDBefore","ALERT", STR0006) //"A configuração não permite desconto via usuário"  
374  
375      EndIf  
376  
377  EndIf  
378  
379  STFShowMessage("STWIDBefore")  
380  
381  Return Nil
```

Componentes/Discount/STWNextSaleDiscount.PRW

136 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWNEXTSALEDISCOUNT.CH"
4
5 //-----
6 /*{Protheus.doc} STWNextSalD
7 Function Desconto na Próxima Venda
8
9 @param cType          Tipo. I - Item , T - Total
10 @param nItemLine      Numero do Item
11 @param cItemCode      Código do produto
12 @param nValue         Valor. Para I - Valor Total do Item , Para T - Valor Total
13 @param nAmount        Quantidade
14 @param cCodReg        Código da Regra
15 @param nPerDiscount   Valor percentual do Desconto
16 @param nValDiscount   Valor do Desconto
17 @return lRet          Retorna se foi concedido desconto na próxima venda
18 @obs
19 @sample
20 */
21 //-----
22 Function STWNextSalD( cType , nItemLine , cItemCode , nValue , ;
23                     nAmount , cCodReg , nPerDiscount , nValDiscount )
24
25 Local lRet      := .T.      // Foi concedido o desconto na próxima venda?
26 Local nDiscount := 0        // Valor do desconto da Regra a ser gerada NCC
27 Local dEndDate  := CTOD("") // Data de validade da Regra. Mesma da NCC a ser gerada
28
29 Default cType      := ""
30 Default nItemLine  := 0
31 Default cItemCode  := ""
32 Default nValue     := 0
33 Default nAmount    := 1
34 Default cCodReg    := ""
35 Default nPerDiscount := 0
36 Default nValDiscount := 0
37
38 ParamType 0 var cType      As Character Default ""
39 ParamType 1 var nItemLine  As Numeric   Default 0
40 ParamType 2 var cItemCode  As Character Default ""
41 ParamType 3 var nValue     As Numeric   Default 0
42 ParamType 4 var nAmount    As Numeric   Default 1
43 ParamType 5 var cCodReg    As Character Default ""
44 ParamType 6 var nPerDiscount As Numeric   Default 0
45 ParamType 7 var nValDiscount As Numeric   Default 0
46
47 /*/
48     Verifica se a regra eh para desconto na proxima venda
49 /*/
50 lRet := STDNextSale( cCodReg )
51
52 If lRet

```

```

53
54   dEndDate := STDRuleDate( cCodReg ) // Data de Validade Regra/NCC a gerar
55
56   Do Case
57
58       Case cType == "I"
59
60           If Empty(cItemCode)
61               cItemCode := STBTaxRet( nItemLine , "IT_PRODUTO" )
62           EndIf
63
64           If Empty(nValue)
65               nValue := STBTaxRet( nItemLine , "IT_TOTAL" )
66           EndIf
67
68           If Empty(nAmount)
69               nAmount := STBTaxRet( nItemLine , "IT_QUANT" )
70           EndIf
71
72       /*
73       Tratamento para buscar o valor do Desconto. ( Valor que sera gerada NCC )
74       */
75       Do Case
76           Case nPerDiscount > 0 // PERCENTUAL
77
78               nDiscount := STBArred( nValue * ( nPerDiscount / 100 ) )
79
80           Case nValDiscount > 0 // VALOR
81
82               nDiscount := nValDiscount * nAmount
83
84           Otherwise
85
86               lRet := .F.
87       EndCase
88
89       Case cType == "T" // TOTAL
90
91           If Empty(nValue)
92               nValue := STFSaleTotal()
93           EndIf
94
95       /*
96       Tratamento para buscar o valor do Desconto. ( Valor que sera gerada NCC )
97       */
98       Do Case
99           Case nPerDiscount > 0 // PERCENTUAL
100
101               nDiscount := nValue * ( nPerDiscount / 100 )
102
103           Case nValDiscount > 0 // VALOR
104

```

```

105             nDiscount := nValDiscount
106
107             Otherwise
108
109                 lRet := .F.
110             EndCase
111
112             Otherwise
113                 lRet := .F.
114             EndCase
115         EndCase
116
117     EndIf
118
119     lRet := lRet .AND. ( nDiscount > 0 )
120
121     If lRet
122         // Chamada armazenar no model o desconto na proxima venda
123         If STDNxtSalAdd( cType      , nItemLine , cItemCode , nValue , ;
124                         nDiscount , dEndDate )
125             STFMessage("STNxtSalD","ALERT", STR0001 ) //"Adicionado Desconto na Próxima venda"
126             STFMessage("STNxtSalD","ALERT", STR0002 ) // "Ao Finalizar a venda será gerada NCC"
127             STFMessage("STNxtSalD","ALERT", STR0003 + cValToChar(nDiscount) ) //"Valor: "
128             STFMessage("STNxtSalD","ALERT", STR0004 + DToC(dEndDate) ) //"Validade: "
129         Else
130             lRet := .F.
131         EndIf
132     EndIf
133
134     STFShowMessage("STNxtSalD")
135
136     Return lRet

```

Componentes/Discount/STWReasonForDiscount.PRW

99 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWREASONFORDISCOUNT.CH"
4
5 //-----
6 /*{Protheus.doc} STWRsnDesc
7 Function Motivo de Desconto
8
9 @param nItemLine      Linha do Item na Venda (Passar 99999 para Desconto no Total)
10 @param cMotCode       Código do Motivo de Desconto
11 @param cObs           String obs
12 @param cItemCode      Código do Produto
13 @return lRet          Retorna se efetuou Motivo de Desconto
14 @obs
15 @sample
16 */
17 //-----
18 Function STWRsnDesc( nItemLine , cMotCode , cObs , cItemCode ,
19                    cCodSup )
20
21 Local lRet          := .T.                // Retorna se efetuou Motivo de Desconto10%2
22 Local cType         := ""                // Tipo de desconto. Se ITEM ou TOTAL
23 Local cRFDFromRD    := SuperGetMV("MV_LJMDRD", "", "") // Código do Motivo de Desconto via Regra de Desconto
24
25 Default nItemLine   := 0                // Linha do Item na Venda
26 Default cMotCode    := ""                // Código do Motivo de Desconto
27 Default cObs        := ""                // String obs
28 Default cItemCode   := ""                // Código do Produto
29 Default cCodSup     := ""                // Código do Supervisor que liberou o desconto
30
31 // Verifica se utiliza motivo de desconto. Existindo algum registro no cadastro utiliza.
32 lRet := STDRsnDesc()
33
34 // Verifica se o item e valido
35 If lRet
36     If nItemLine <= 0
37
38         lRet := .F.
39         STFMessage("STRsnDesc","STOP", STR0001) //"Nao é possível atribuir Motivo de Desconto para item 0"
40
41     EndIf
42 EndIf
43
44 If lRet
45
46     // Se nao foi passado o motivo de desconto, chama tela.
47     If Empty(cMotCode)
48
49         /*/
50         Get do motivo informado via interface
51         /*/
52         cMotCode := STIGetReason()

```

```

53      cObs      := STIGetObsReason()
54      lRet := !(Empty(cMotCode))
55
56  Else
57
58      If cMotCode == "REGRA DE DESCONTO"
59
60          /*/
61          Validar se cadastra Motivo de Desconto por Regra de desconto
62          /*/
63          If !Empty(cRFDFromRD)
64
65              If STDRFDFromRD(cRFDFromRD) // Verifica se utiliza
66                  cMotCode := cRFDFromRD
67              Else
68                  lRet := .F.
69              EndIf
70
71          Else
72              lRet := .F.
73          EndIf
74
75      EndIf
76
77  EndIf
78
79  If lRet
80
81      If nItemLine == 99999
82          cType = "VALOR"
83      Else
84          cType = "ITEM"
85          If Empty( cItemCode )
86              cItemCode := STBTaxRet( nItemLine , "IT_PRODUTO" )
87          EndIf
88      EndIf
89
90      STDAddReason(   nItemLine   , cMotCode , cObs , cItemCode , ;
91                    cType       , cCodSup   )
92
93  EndIf
94
95 EndIf
96
97 STFShowMessage("STRsnDesc")
98
99 Return lRet

```

Componentes/Discount/STWRuleDiscountItem.PRW

88 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4
5 //-----
6 /*{Protheus.doc} STWRuleItem
7 Function Regra de Desconto Item
8
9 @param  nItemLine      Número do Item
10 @param  cItemCode      Codigo Produto
11 @param  nValueItem     Valor
12 @param  nAmount        Quantidade
13 @param  cTypeProd      Tipo do Produto Mostruario/Saldao
14 @param  cCliCode       Código do Cliente
15 @param  cCliStore      Codigo da Loja
16 @return aRet           Retorna informações da Regra
17 @obs
18 @sample
19 */
20 //-----
21 Function STWRuleItem( nItemLine , cItemCode , nValueItem , nAmount , ;
22                     cTypeProd , cCliCode , cCliStore )
23
24 Local aArea      := GetArea()           // Armazena area corrente
25 Local aRet       := {}                 // Retorno funcao
26 Local nItemTotal := 0                  // Valor Unitário * Quantidade
27
28 Default nItemLine := 0
29 Default cItemCode := ""
30 Default nValueItem := 0
31 Default nAmount   := 0
32 Default cTypeProd := ""
33 Default cCliCode  := ""
34 Default cCliStore := ""
35
36 ParamType 0 Var nItemLine As Numeric      Default 0
37 ParamType 1 Var cItemCode As Character    Default ""
38 ParamType 2 Var nValueItem As Numeric     Default 0
39 ParamType 3 Var nAmount   As Numeric     Default 0
40 ParamType 4 Var cTypeProd As Character    Default ""
41 ParamType 5 Var cCliCode  As Character    Default ""
42 ParamType 6 Var cCliStore As Character    Default ""
43
44 /*/
45     Get das variáveis
46 /*/
47 If Empty(cItemCode)
48     cItemCode := STBTaxRet( nItemLine , "IT_PRODUTO" )
49 EndIf
50
51 If Empty(nValueItem)
52     nValueItem := STBTaxRet( nItemLine , "IT_PRCUNI" )

```

```

53 EndIf
54
55 If Empty(nAmount)
56     nAmount := STBTaxRet( nItemLine , "IT_QUANT" )
57 EndIf
58
59 If Empty(nItemTotal)
60     nItemTotal := STBTaxRet( nItemLine , "IT_TOTAL" )
61 EndIf
62
63 If Empty(cTypeProd)
64     // TODO: Pegar se o produto é mostruário/Saldão. Nao foi feito ainda nessa fase
65 EndIf
66
67 If Empty(cCliCode)
68     cCliCode := STDGPBasket( "SL1" , "L1_CLIENTE" )
69 EndIf
70
71 If Empty(cCliStore)
72     cCliStore := STDGPBasket( "SL1" , "L1_LOJA" )
73 EndIf
74
75 If SuperGetMv("MV_LJRGDES",,.F.) // "Nova" Regra de Desconto ( Regra do LOJA )
76
77     aRet := STBItemRLDi( nItemLine , cItemCode , nValueItem , nAmount , ;
78                         cTypeProd , cCliCode , cCliStore )
79
80 Else
81
82     AADD( aRet , .F. )
83
84 EndIf
85
86 RestArea( aArea )
87
88 Return aRet

```

Componentes/Discount/STWRuleDiscountPay.PRW

60 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STWPayRlDi
6 Function Regra de Desconto Total
7
8 @param  nValueTotal      Valor total da venda
9 @param  nValueBalancePay Valor pagamento
10 @param  cFormPay         Forma de pagamento
11 @param  aAllPay          Array de Pagamentos
12 @param  cCliCode        Codigo Cliente
13 @param  cCliStore        Loja Cliente
14 @return aRet[1]          LRULETOTAP          1 // Alguma regra foi aplicada?
15 @return aRet[2]          NNEWTOTAL           2 // Valor total depois da aplicada(s) a(s) regra(s)
16 @return aRet[3]          NPERDESCTO         3 // Percentual de desconto total aplicado em relacao ao valor total antigo
17 @return aRet[4]          NOLDTOTAL           4 // Valor total antes da aplicada(s) a(s) regra(s)
18 @obs
19 @sample
20 */
21 //-----
22 Function STWPayRlDi(nValueTotal , nValueBalancePay , cFormPay , aAllPay , ;
23                   cCliCode , cCliStore )
24
25 Local aRet          := {}
26
27 Default nValueTotal      := 0
28 Default nValueBalancePay := 0
29 Default cFormPay         := ""
30 Default aAllPay          := {}
31 Default cCliCode        := ""
32 Default cCliStore        := ""
33
34 ParamType 0 Var nValueTotal      AS Numeric      Default 0
35 ParamType 1 Var nValueBalancePay AS Numeric      Default 0
36 ParamType 2 Var cFormPay         AS Character    Default ""
37 ParamType 3 Var aAllPay          AS Array        Default {}
38 ParamType 4 Var cCliCode        AS Character    Default ""
39 ParamType 5 Var cCliStore        AS Character    Default ""
40
41 // Valida total
42 If nValueTotal == 0
43     nValueTotal := STFSaleTotal()
44 EndIf
45
46 If SuperGetMv("MV_LJRGDES",,.F.) // "Nova" Regra de Desconto
47     aRet := STBPayRlDi( nValueTotal , nValueBalancePay , cFormPay , aAllPay , ;
48                       cCliCode , cCliStore )
49
50     If aRet[LRULEAPPLY]
51         STWRsnDesc( 99999 , "REGRA DE DESCONTO" , "Regra de Desconto - TOTAL" ) // Motivo de Desconto
52     EndIf

```

```
53 EndIf
54
55 If Len(aRet) == 0
56     // Retorno padrao
57     aRet := { .F. , nValueTotal , 0 , 0 , nValueTotal }
58 EndIf
59
60 Return aRet
```

Componentes/Discount/STWRuleDiscountTotal.PRW

80 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWRULEDISCOUNTTOTAL.CH"
5
6 //-----
7 /*{Protheus.doc} STWRuleTotal
8 Function Regra de Desconto Total
9
10 @param nValueTotal      Valor total da venda
11 @param cCliCode         Codigo Cliente
12 @param cCliStore        Loja Cliente
13 @return aRet[1]         1 // Alguma regra foi aplicada?
14 @return aRet[2]         2 // Valor total depois da aplicada(s) a(s) regra(s)
15 @return aRet[3]         3 // Percentual de desconto total aplicado em relacao ao valor total antigo
16 @return aRet[4]         4 // Valor total antes da aplicada(s) a(s) regra(s)
17 @obs
18 @sample
19 */
20 //-----
21 Function STWRuleTotal( nValueTotal , cCliCode , cCliStore )
22
23 Local aRet          := {}                      // Retorno funcao
24 Local lRet          := .F.                    // retorno do desconto por forma de pagamento
25 Local cMvCondPad    := Iif (ExistFunc("STBCondPad"), STBCondPad(), SuperGetMv('MV_CONDPAD', .F., '')) //Condicao de pagamento padrao
26 Local aArea         := {}
27 Local cForma        := ""
28
29 Default nValueTotal := 0
30 Default cCliCode    := ""
31 Default cCliStore   := ""
32
33 ParamType 0 var nValueTotal As Numeric Default 0
34 ParamType 1 var cCliCode    As Character Default ""
35 ParamType 2 var cCliStore   As Character Default ""
36
37 If nValueTotal == 0
38     nValueTotal := STFSaleTotal()
39 EndIf
40
41 If Empty(cCliCode)
42     cCliCode := STDGPBasket( "SL1" , "L1_CLIENTE" )
43 EndIf
44
45 If Empty(cCliStore)
46     cCliStore := STDGPBasket( "SL1" , "L1_LOJA" )
47 EndIf
48
49 If SuperGetMv("MV_LJRGDES", .F.) // "Nova" Regra de Desconto
50     If Empty( STDGPBasket( "SL1" , "L1_NUMORIG" ) ) // Para importação de orçamento o desconto deve vir gravado
51
52         /* Verifica a forma padrao, usada na entrada do primeiro pagamento */

```

```

53      aArea      := GetArea()
54      DbSelectArea("SE4")
55      SE4->(DbSetOrder(1))
56      If SE4->(DbSeek(xFilial("SE4")+ cMvCondPad))
57          cForma := Alltrim(SE4->E4_FORMA)
58      EndIf
59
60      RestArea(aArea)
61
62      If !Empty(cForma) .And. STBPromPgto(cForma)
63          aRet := STBTotRlDi( nValueTotal , cCliCode , cCliStore, .T., cForma )
64          lRet := aRet[LRULEAPPLY] //alguma regra foi aplicada ?
65      EndIf
66
67      If lRet //alguma regra foi aplicada ?
68          STWTotDisc( aRet[NPERDESCTO] , "V" , "RULEDISC")
69
70          STWRsnDesc( 99999 , "REGRA DE DESCONTO" , "Regra de Desconto - TOTAL" ) // Motivo de Desconto
71      EndIf
72  EndIf
73 EndIf
74
75 If Len(aRet) == 0
76     // Retorno padrao
77     aRet := { .F. , nValueTotal , 0 , 0 , nValueTotal }
78 EndIf
79
80 Return aRet

```

Componentes/ExtendedWarranty/STBExtWarranty.prw

204 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBFINSERVICE.CH"
5
6
7 //-----
8 /*{Protheus.doc} STBIsGarEst
9 Verifica se o produto é do tipo Garantia Estendida
10
11 @param   cProductCode      Codigo do Produto
12 @return  lRet              Retorno da validação
13 @obs
14 @sample
15 */
16 //-----
17 Function STBIsGarEst( cProductCode )
18
19 Local      lRet              := .F.                // Retorna se é Garantia Estendida
20 Local      aProduct          := {}                // Array do Produto
21 Local      cGarEstType       := SuperGetMv("MV_LJTPGAR",,"GE") // Define o tipo do produto de Garantia Estendida
22
23 Default    cProductCode      := ""                // Codigo do Produto
24
25 If ExistFunc("STDFindItem")
26   aProduct := STDFindItem( cProductCode )
27 Else
28   aProduct := {}
29 EndIf
30
31 If !EMPTY(aProduct)
32   lRet := aProduct[ITEM_TIP0] == cGarEstType
33 EndIf
34
35 Return lRet
36
37
38 //-----
39 /*{Protheus.doc} Adm_Item_ExtWarranty
40 Classe para Armazenamento de Informacoes de Itens Garantia Estendida
41
42 @param
43 @return
44 @obs
45 @sample
46 */
47 //-----
48 CLASS Adm_Item_ExtWarranty
49 /*
50   Declaracao das propriedades da Classe
51 */
52 DATA aRecords As Array //Propriedade para armazenamento das informacoes

```

```

53
54 /*
55     Declaracao dos Metodos da Classe
56 */
57 METHOD New() CONSTRUCTOR           //Metodo Construtor
58 METHOD SetGarEst(nItemLine, cCodItem) //Metodo para inserir informacoes no objeto
59 METHOD GetGarEst(nItemServFin)      //Metodo para pegar informacoes no objeto
60 METHOD ClearGarEst()                //Metodo para limpar informacoes no objeto
61 METHOD DelGarEst()                  //Metodo para marcar item como deletado no objeto
62 METHOD SelGarEst()                  //Metodo para marcar item como selecionado no objeto
63
64 ENDCLASS
65
66 /*
67     Criacao do construtor para atribuir os valores default
68     para as propriedades e retorna Self
69 */
70 METHOD New() CLASS Adm_Item_ExtWarranty
71
72 Self:aRecords := {} //Inicializa propriedade com tipo array
73
74 Return Self
75
76 /*
77     Metodo para adicionar informacoes no objeto
78 */
79 METHOD SetGarEst(cCodGarEst, nPriceGarEst, cCodItem, nItemLine,;
80               cTypeItem, nPriceItem, cSerieItem) CLASS Adm_Item_ExtWarranty
81
82 Local lHabilita := .F.
83
84 Default      cCodGarEst      := "" //Codigo do Produto Servico Financeiro
85 Default      nPriceGarEst    := 0  //Valor do Produto Servico Financeiro
86 Default      cCodItem       := "" //Codigo do Produto Vendido
87 Default      nItemLine      := 0  //Numero de Item do Produto Vendido
88 Default      cTypeItem      := "" //Tipo do Item - Utilizado para importacao de Orcamento
89 Default      nPriceItem     := 0  //Valor total do produto vinculado à Garantia Estendida
90 Default      cSerieItem     := 0  //Série do produto vinculado à Garantia Estendida
91
92 //Tratamento para importacao Garantia Estendida
93 If cTypeItem == "IMP"
94     lHabilita := .T.
95 EndIf
96
97 //Verifica se produto é uma Garantia
98 If ExistFunc("STDDescProd") .AND. STBIsGarEst( cCodGarEst )
99     //Verifica se produto ja existe no objeto
100     If aScan(Self:aRecords,{|xVar| xVar[1] == cCodGarEst .AND. xVar[3] == nPriceGarEst .AND. xVar[4] == cCodItem .AND.;
101         xVar[6] == nItemLine .AND. xVar[7] == cTypeItem}) == 0
102
103         //Adiciona item no objeto
104         aAdd(Self:aRecords, ARRAY(11))

```

```

105     Self:aRecords[Len(Self:aRecords), 1] := cCodGarEst      //Codigo do Produto Servico Financeiro
106     Self:aRecords[Len(Self:aRecords), 2] := STDDescProd(cCodGarEst) //Descricao do Produto Servico Financeiro
107     Self:aRecords[Len(Self:aRecords), 3] := nPriceGarEst      //Valor do Servico Financeiro
108     Self:aRecords[Len(Self:aRecords), 4] := cCodItem          //Codigo do Produto Vendido
109     Self:aRecords[Len(Self:aRecords), 5] := STDDescProd(cCodItem) //Descricao do Produto Vendido
110     Self:aRecords[Len(Self:aRecords), 6] := nItemLine         //Numero do Item do Produto Vendido
111     Self:aRecords[Len(Self:aRecords), 7] := cTypeItem         //Tipo do Item - Utilizado na importacao de orcamento
112     Self:aRecords[Len(Self:aRecords), 8] := .F.              //Define se item esta deletado
113     Self:aRecords[Len(Self:aRecords), 9] := lHabilita         //Define se item selecionado para finalizacao de venda
114     Self:aRecords[Len(Self:aRecords),10] := nPriceItem        //Valor Total do Produto vinculado à Garantia Estendida
115     Self:aRecords[Len(Self:aRecords),11] := cSerieItem        //Série do Produto vinculado à Garantia Estendida
116   EndIf
117 EndIf
118
119 Return Nil
120
121 /*
122   Metodo para pegar informacoes do objeto
123   Retorna posicao do array de acordo com o Item vendido (nItemLine)
124 */
125 METHOD GetGarEst(nItemLine) CLASS Adm_Item_ExtWarranty
126
127 Local aRet := {}
128 Local nI   := 0
129
130 Default   nItemLine := 0    //Numero de Item do Produto Vendido
131
132 ParamType 0 Var nItemLine AS Numeric Default 0
133
134 //Busca itens de servico conforme parametro
135 For nI := 1 To Len(Self:aRecords)
136
137   If nItemLine == 0 .OR. Self:aRecords[nI, 6] == nItemLine
138     aAdd( aRet, aClone(Self:aRecords[nI]) )
139   EndIf
140
141 Next nI
142
143 Return aRet
144
145 /*
146   Metodo para limpar informacoes do objeto
147   Retorna NIL
148 */
149 METHOD ClearGarEst() CLASS Adm_Item_ExtWarranty
150
151 Self:aRecords := {} //Limpa propriedade com tipo array
152
153 Return Nil
154
155 /*
156   Metodo para marcar item como deletado no objeto

```

```

157 */
158 METHOD DelGarEst(cCodGarEst, cCodItem, nItemLine) CLASS Adm_Item_ExtWarranty
159
160 Local nI := 0 //Contador
161
162 Default      cCodGarEst      := "" //Codigo do Produto Garantia Estendida
163 Default      cCodItem        := "" //Codigo do Produto Vendido
164 Default      nItemLine       := 0 //Numero de Item do Produto Vendido
165
166 /* Busca itens de GE conforme parametro */
167 For nI := 1 To Len(Self:aRecords)
168     /* Se item Garantia Estendida avulso, deleta o mesmo */
169     If STBIsGarEst( cCodGarEst )
170         If AllTrim(Self:aRecords[nI, 1]) == AllTrim(cCodGarEst)
171             Self:aRecords[nI, 8] := .T.
172         EndIf
173     Else
174         /* Senao deleta as Garantias Estendidas vinculadas */
175         If AllTrim(Self:aRecords[nI, 4]) == AllTrim(cCodItem) .AND. Self:aRecords[nI, 6] == nItemLine
176             Self:aRecords[nI, 8] := .T.
177         EndIf
178     EndIf
179 Next nI
180
181 Return Nil
182
183 /*
184     Metodo para marcar selecao de item
185 */
186 METHOD SelGarEst(cCodGarEst, cCodItem, nItemLine, nValServ) CLASS Adm_Item_ExtWarranty
187
188 Local nI := 0 //Contador
189
190 Default      cCodGarEst      := "" //Codigo do Produto Garantia Estendida
191 Default      cCodItem        := "" //Codigo do Produto Vendido
192 Default      nItemLine       := 0 //Numero de Item do Produto Vendido
193
194 /* Busca itens de servico conforme parametro */
195 For nI := 1 To Len(Self:aRecords)
196     /* Seleciona Garantia Estendida vinculada */
197     If AllTrim(Self:aRecords[nI, 1]) == AllTrim(cCodGarEst) .AND. AllTrim(Self:aRecords[nI, 4]) == AllTrim(cCodItem) .AND.;
198         Self:aRecords[nI, 6] == nItemLine .AND. Self:aRecords[nI, 3] == nValServ
199
200         Self:aRecords[nI, 9] := .T.
201     EndIf
202 Next nI
203
204 Return Nil

```

Componentes/ExtendedWarranty/STDExtWarranty.prw

74 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STDFINSERVICE.CH"
5
6
7 //-----
8 /*{Protheus.doc} STDPrintGarEst
9 Function para Buscar alimentar Rdmake dos Relatorios Gerenciais - Garantia Estendida
10
11 @param   aSelItem      Array com o item selecionado de Garantia Estendida
12 @param   aRelSF        Array que contera os itens selecionados para impressao
13 @return
14 @obs
15 @sample
16 */
17 //-----
18 Function STDPrintGarEst(aSelItem, aRelGE)
19
20 Default aSelItem      := {}
21 Default aRelGE        := {}
22
23 If Len(aSelItem) >= 11      //Se houver total do item vinculado à garantia
24
25     /* Array com informacoes para impressao */
26     aAdd(aRelGE,,
27         {aSelItem[4],;          //Código de Produto Comum
28         aSelItem[5],;          //Descrição do Produto Comum
29         Str(a410Arred(aSelItem[10] * 1,"L2_VLRITEM"), TamSx3("L2_VLRITEM")[1],TamSx3("L2_VLRITEM")[2]),; //Preço do Produto Comum
30         aSelItem[11],;         //Série do produto comum
31         aSelItem[1],;          //Código da Garantia Utilizada
32         aSelItem[2],;          //Descrição da Garantia Utilizada
33         Str(a410Arred(aSelItem[3] * 1,"L2_VLRITEM"), TamSx3("L2_VLRITEM")[1],TamSx3("L2_VLRITEM")[2]),; //Valor da Garantia Utilizada
34         })
35 Else
36     LjGrvLog("STDPrintGarEst","Array aSelItem é menor que 11 elementos")
37 EndIf
38
39 Return Nil
40
41
42 //-----
43 /*{Protheus.doc} STDGeProdVin
44 Function para retornar o código do produto vinculado, e o seu valor, a partir do STDGPBasket.
45
46 @param   cProdGar      Produto Garantia
47 @return   cProduto      Produto Comum que vai utilizar aquele Produto Garantia
48 @return   nVlrItem      Valor do produto comum que será impresso no cupom gerencial
49 @return   cSerie        Série do Produto Comum que vai utilizar aquele Produto Garantia
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STDGEProdVin(cProdGar,cItemProdVin)
55
56 Local   nX           := 0           //Contador
57 Local   cProduto     := ""          //Produto Garantia
58 Local   nVlrItem     := 0           //Valor do Item
59 Local   cSerie       := ""          //Serie do Produto
60
61 For nX := 1 To STDPBLength("SL2")
62
63     //Pesquisa, pelo Produto Garantia (cProdGar), qual produto comum é vinculado ao produto garantia (L2_GARANT).
64     If cProdGar == STDGPBasket( "SL2" , "L2_GARANT" , nX ) .AND.;
65         cItemProdVin == STDGPBasket( "SL2" , "L2_ITEM" , nX )
66         cProduto     := STDGPBasket( "SL2" , "L2_PRODUTO" , nX )
67         nVlrItem     := STDGPBasket( "SL2" , "L2_VLRITEM" , nX )
68         cSerie       := STDGPBasket( "SL2" , "L2_NSERIE" , nX )
69     Loop
70 EndIf
71
72 Next nX //Retorno dados do produto comum (vinculado)
73
74 Return {cProduto, nVlrItem, cSerie}

```

Componentes/ExtendedWarranty/STWExtWarranty.prw

121 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWFINDITEM.CH"
5
6 Static oGarEstItens := nil //Objeto para armazenamento de informacoes de Itens Servicos Financeiros
7
8
9 //-----
10 /*{Protheus.doc} STWItemGarEst
11 Function Armazena, Retorna ou Limpa Item Garantia Estendida em Array
12
13 @param nTipoProc      Tipo do Processo (1=Set - 2=Get - 3=Clear)
14 @param cCodGarEst      Produto Garantia
15 @param nPriceGarEst    Valor do Produto Garantia
16 @param cCodItem        Produto da Venda
17 @param nItemLine       Item do Produto da Venda
18 @param cTypeItem       Tipo do Item - Usado na importacao de orcamento
19 @return lRet           Garantia Estendida Avulso
20 @obs
21 @sample
22 */
23 //-----
24 Function STWItemGarEst( nTipoProc      , cCodGarEst      , nPriceGarEst  , cCodItem,,
25                       nItemLine       , cTypeItem      , nPriceItem   , cSerieItem   )
26
27 Local xRet              := Nil // Retorno da função
28
29 Default nTipoProc       := 1    //Tipo do Processo (1=Set - 2=Get - 3=Clear - 4=Del - 5=Sel)
30 Default cCodGarEst      := ""    //Produto Servico
31 Default nPriceGarEst    := 0     //Valor do Produto Servico
32 Default cCodItem        := ""    //Produto da Venda
33 Default nItemLine       := 0     //Item do Produto da Venda
34 Default cTypeItem       := ""    //Tipo do Item - Usado na importacao de orcamento
35 Default nPriceItem      := 0     //Valor do Produto da Venda
36 Default cSerieItem      := ""    //Série do Produto da Venda
37
38 /*
39   Inicializando objeto
40 */
41 If oGarEstItens = nil
42   oGarEstItens := Adm_Item_ExtWarranty():New()
43 EndIf
44
45 /*
46   Armazena Item Garantia Estendida
47 */
48 If Valtype(oGarEstItens) <> "U" //Valida se objeto foi instanciado
49   If nTipoProc == 1
50     xRet := oGarEstItens:SetGarEst(cCodGarEst, nPriceGarEst, cCodItem, nItemLine,,
51                                   cTypeItem, nPriceItem, cSerieItem)
52   ElseIf nTipoProc == 2

```

```

53     xRet := oGarEstItens:GetGarEst(nItemLine)
54     ElseIf nTipoProc == 3
55         xRet := oGarEstItens:ClearGarEst()
56     ElseIf nTipoProc == 4
57         xRet := oGarEstItens:DelGarEst(cCodGarEst, cCodItem, nItemLine)
58     ElseIf nTipoProc == 5
59         xRet := oGarEstItens:SelGarEst(cCodGarEst, cCodItem, nItemLine, nPriceGarEst)
60     EndIf
61 EndIf
62
63 Return xRet
64
65
66 //-----
67 /*{Protheus.doc} STWPrintGarEst
68 Funcao para impressao de relatorios gerenciais Garantia Estendida
69
70 @param
71 @return
72 @obs
73 @sample
74 */
75 //-----
76 Function STWPrintGarEst()
77
78 Local aRelGE      := {}           //Array para passagem de parâmetro de função rdMake
79 Local nVias       := 2           //Número de vias
80 Local nCont       := 0           //Contador
81 Local cTexto      := ""          //Texto
82 Local cImpRGer    := SuperGetMV("MV_RELGART",,"U_L0JR600A")
83 Local aGarItens   := STWItemGarEst(2) //Tipo do Processo (1=Set - 2=Get - 3=Clear - 4=Del - 5=Sel)
84 Local cNomeCli    := POSICIONE("SA1", 1, xFilial("SA1") + STDGPBasket( "SL1" , "L1_CLIENTE" ) + STDGPBasket( "SL1" , "L1_LOJA" ), "A1_NOME")//Retorna o nome do Cliente
85
86 If ExistFunc("STDPrintGarEst")
87     For nCont := 1 To Len(aGarItens)
88         /* Garantia Estendida habilitada e selecionada */
89         If !aGarItens[nCont][8] .And. aGarItens[nCont][9]
90             /* Alimenta Rdmake dos Relatorios */
91             STDPrintGarEst(aGarItens[nCont], @aRelGE)
92         EndIf
93     Next nCont
94 EndIf
95
96 /*
97     Efetua a impressao
98 */
99
100 If FindFunction(cImpRGer) //Chamar função RDMAKE
101     For nCont := 1 To Len(aRelGE)
102         cTexto := chr(16) +;
103             &cImpRGer.(aRelGE[nCont][1],; //Código do produto comum
104                 aRelGE[nCont][2],; //Descrição do produto comum

```

```
105          aRelGE[nCont][3],;          //Valor do produto comum
106          aRelGE[nCont][4],;          //Número da série do produto comum
107          aRelGE[nCont][5],;          //Código do produto garantia
108          aRelGE[nCont][6],;          //Descrição do produto garantia
109          aRelGE[nCont][7],;          //Valor do produto garantia
110          cNomeCli,;                  //Nome do cliente
111          1,;                          //Número da moeda
112          .F.,;                        //Se é um serviço financeiro
113          .T.)                        //Se é um POS PDV
114      //Relatorio da conferencia de caixa
115      STWManagReportPrint( cTexto ,nVias )
116  Next nCont
117 Else
118     LjGrvLog("STWPrintGarEst","Função "+cImpRGer+" não encontrada, portanto, não será impresso o relatório gerencial no cupom.")
119 EndIf
120
121 Return Nil
```

Componentes/FinancialService/STBFinService.prw

407 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBFINSERVICE.CH"
5
6 //-----
7 /*/{Protheus.doc} STBIsBulkServ
8 Verifica se o produto serviço é avulso..
9
10 @param  cProductCode      Codigo do Produto
11 @return  lRet              Retorno da validação
12 @obs
13 @sample
14 */
15 //-----
16 Function STBIsBulkServ( cProductCode )
17
18 Local      lRet              := .F.              // Retorna Serviço Avulso?
19 Local      aProductCode     := {}               // Array do Produto de Serviço
20
21 Default    cProductCode     := ""               // Codigo do Produto
22
23 ParamType 0 Var cProductCode AS Character      Default ""
24
25 aProductCode := STDProdServ(cProductCode)
26
27 If !EMPTY(aProductCode)
28     lRet := aProductCode[05] == "2" //Produto não Vinculado (Avulso)
29 EndIf
30
31 Return lRet
32
33
34 //-----
35 /*/{Protheus.doc} STBIsPriceRange
36 Verifica se o produto serviço é precificado por faixa de preço
37
38 @param  cProductCode      Codigo do Produto
39 @return  lRet              Retorno da validação
40 @obs
41 @sample
42 */
43 //-----
44 Function STBIsPriceRange( cProductCode )
45
46 Local      lRet              := .F.              // Retorna Serviço Avulso?
47 Local      aProductCode     := {}               // Array do Produto de Serviço
48
49 Default    cProductCode     := ""               // Codigo do Produto
50
51 ParamType 0 Var cProductCode AS Character      Default ""
52

```

```

53 aProductCode := STDProdServ(cProductCode)
54
55 If !EMPTY(aProductCode)
56     lRet := aProductCode[04] == "1" //Tipo de Precificação 1=Faixa de Preço;2=Fixo
57 EndIf
58
59 Return lRet
60
61
62 //-----
63 /*{Protheus.doc} STBIsFinService
64 Verifica se o produto é do tipo serviço financeiro
65
66 @param    cProductCode      Codigo do Produto
67 @return   lRet              Retorno da validação
68 @obs
69 @sample
70 */
71 //-----
72 Function STBIsFinService( cProductCode )
73
74 Local      lRet              := .F.                // Retorna se é serviço financeiro
75 Local      aProduct          := {}                // Array do Produto
76 Local      cServType         := SuperGetMv("MV_LJTPSF",,"SF") // Define o tipo do produto de servico financeiro
77
78 Default    cProductCode      := ""                // Codigo do Produto
79
80 ParamType 0 Var cProductCode AS Character Default ""
81
82 aProduct   := STDFindItem( cProductCode )
83
84 If !EMPTY(aProduct)
85     lRet := aProduct[ITEM_TIP0] == cServType
86 EndIf
87
88 Return lRet
89
90 //-----
91 /*{Protheus.doc} Adm_Item_FinService
92 Classe para Armazenamento de Informacoes de Itens Servicos Financeiros
93
94 @param
95 @return
96 @obs
97 @sample
98 */
99 //-----
100 CLASS Adm_Item_FinService
101 /*
102     Declaracao das propriedades da Classe
103 */
104 DATA aRecords As Array //Propriedade para armazenamento das informacoes

```

```

105
106 /*
107     Declaracao dos Metodos da Classe
108 */
109 METHOD New() CONSTRUCTOR           //Metodo Construtor
110 METHOD SetServFin(nItemLine, cCodItem) //Metodo para inserir informacoes no objeto
111 METHOD GetServFin(nItemServFin)      //Metodo para pegar informacoes no objeto
112 METHOD ClearServFin()               //Metodo para limpar informacoes no objeto
113 METHOD DelServFin()                 //Metodo para marcar item como deletado no objeto
114 METHOD SelServFin()                 //Metodo para marcar item como selecionado no objeto
115
116 ENDCLASS
117
118 /*
119     Criacao do construtor para atribuir os valores default
120     para as propriedades e retorna Self
121 */
122 METHOD New() CLASS Adm_Item_FinService
123
124 Self:aRecords := {} //Inicializa propriedade com tipo array
125
126 Return Self
127
128 /*
129     Metodo para adicionar informacoes no objeto
130 */
131 METHOD SetServFin(cCodServProd, nPriceServFin, cCodItem, nItemLine, cTypeItem) CLASS Adm_Item_FinService
132
133 Local lHabilita := .F.
134
135 Default      cCodServProd      := ""    //Codigo do Produto Servico Financeiro
136 Default      nPriceServFin     := 0     //Valor do Produto Servico Financeiro
137 Default      cCodItem          := ""    //Codigo do Produto Vendido
138 Default      nItemLine         := 0     //Numero de Item do Produto Vendido
139 Default      cTypeItem         := ""    //Tipo do Item - Utilizado para importacao de Orcamento
140
141 ParamType 0 Var cCodServProd    AS Character    Default ""
142 ParamType 1 Var nPriceServFin   AS Numeric     Default 0
143 ParamType 2 Var cCodItem       AS Character    Default ""
144 ParamType 3 Var nItemLine      AS Numeric     Default 0
145 ParamType 4 Var cTypeItem      AS Character    Default ""
146
147 //Tratamento para importacao Servico Financeiro
148 If cTypeItem == "IMP"
149     lHabilita := .T.
150 Else
151     lHabilita := STBIsBulkServ(cCodServProd)
152 EndIf
153
154 //Verifica se pproduto Servico Financeiro
155 If STBIsFinService( cCodServProd )
156     //Verifica se produto ja existe no objeto

```

```

157 If aScan(Self:aRecords,{|xVar| xVar[1] == cCodServProd .AND. xVar[3] == nPriceServFin .AND. xVar[4] == cCodItem .AND.;
158     xVar[6] == nItemLine .AND. xVar[7] == cTypeItem}) == 0
159
160     //Adiciona item no objeto
161     aAdd(Self:aRecords, ARRAY(9))
162     Self:aRecords[Len(Self:aRecords), 1] := cCodServProd //Codigo do Produto Servico Financeiro
163     Self:aRecords[Len(Self:aRecords), 2] := STDDescProd(cCodServProd) //Descricao do Produto Servico Financeiro
164     Self:aRecords[Len(Self:aRecords), 3] := nPriceServFin //Valor do Servico Financeiro
165     Self:aRecords[Len(Self:aRecords), 4] := cCodItem //Codigo do Produto Vendido
166     Self:aRecords[Len(Self:aRecords), 5] := STDDescProd(cCodItem) //Descricao do Produto Vendido
167     Self:aRecords[Len(Self:aRecords), 6] := nItemLine //Numero do Item do Produto Vendido
168     Self:aRecords[Len(Self:aRecords), 7] := cTypeItem //Tipo do Item - Utilizado na importacao de orcamento
169     Self:aRecords[Len(Self:aRecords), 8] := .F. //Define se item esta deletado
170     Self:aRecords[Len(Self:aRecords), 9] := lHabilita //Define se item selecionado para finalizacao de venda
171 EndIf
172 EndIf
173
174 Return Nil
175
176 /*
177     Metodo para pegar informacoes do objeto
178     Retorna posicao do array de acordo com o Item vendido (nItemLine)
179 */
180 METHOD GetServFin(nItemLine) CLASS Adm_Item_FinService
181
182 Local aRet := {}
183 Local nI := 0
184
185 Default nItemLine := 0 //Numero de Item do Produto Vendido
186
187 ParamType 0 Var nItemLine AS Numeric Default 0
188
189 //Busca itens de servico conforme parametro
190 For nI := 1 To Len(Self:aRecords)
191
192     If nItemLine == 0 .OR. Self:aRecords[nI, 6] == nItemLine
193         aAdd( aRet, aClone(Self:aRecords[nI]) )
194     EndIf
195
196 Next nI
197
198 Return aRet
199
200 /*
201     Metodo para limpar informacoes do objeto
202     Retorna NIL
203 */
204 METHOD ClearServFin() CLASS Adm_Item_FinService
205
206 Self:aRecords := {} //Limpa propriedade com tipo array
207
208 Return Nil

```

```

209
210 /*
211     Metodo para marcar item como deletado no objeto
212 */
213 METHOD DelServFin(cCodServProd, cCodItem, nItemLine) CLASS Adm_Item_FinService
214
215 Local nI := 0 //Contador
216
217 Default      cCodServProd      := "" //Codigo do Produto Servico Financeiro
218 Default      cCodItem          := "" //Codigo do Produto Vendido
219 Default      nItemLine         := 0 //Numero de Item do Produto Vendido
220
221 ParamType 0 Var cCodServProd AS Character Default ""
222 ParamType 1 Var cCodItem AS Character Default ""
223 ParamType 2 Var nItemLine AS Numeric Default 0
224
225 /* Busca itens de servico conforme parametro */
226 For nI := 1 To Len(Self:aRecords)
227     /* Se item Servico Financeiro avulso, deleta o mesmo */
228     If STBIsFinService( cCodServProd )
229         If AllTrim(Self:aRecords[nI, 1]) == AllTrim(cCodServProd)
230             Self:aRecords[nI, 8] := .T.
231         EndIf
232     Else
233         /* Senao deleta os Servicos Financeiros vinculados */
234         If AllTrim(Self:aRecords[nI, 4]) == AllTrim(cCodItem) .AND. Self:aRecords[nI, 6] == nItemLine
235             Self:aRecords[nI, 8] := .T.
236         EndIf
237     EndIf
238 Next nI
239
240 Return Nil
241
242 /*
243     Metodo para marcar selecao de item
244 */
245 METHOD SelServFin(cCodServProd, cCodItem, nItemLine, nValServ) CLASS Adm_Item_FinService
246
247 Local nI := 0 //Contador
248
249 Default      cCodServProd      := "" //Codigo do Produto Servico Financeiro
250 Default      cCodItem          := "" //Codigo do Produto Vendido
251 Default      nItemLine         := 0 //Numero de Item do Produto Vendido
252
253 ParamType 0 Var cCodServProd AS Character Default ""
254 ParamType 1 Var cCodItem AS Character Default ""
255 ParamType 2 Var nItemLine AS Numeric Default 0
256
257 /* Busca itens de servico conforme parametro */
258 For nI := 1 To Len(Self:aRecords)
259     /* Seleciona Servicos Financeiros vinculados */
260     If AllTrim(Self:aRecords[nI, 1]) == AllTrim(cCodServProd) .AND. AllTrim(Self:aRecords[nI, 4]) == AllTrim(cCodItem) .AND.;

```

```

261         Self:aRecords[nI, 6] == nItemLine .AND. Self:aRecords[nI, 3] == nValServ
262
263         Self:aRecords[nI, 9] := .T.
264     EndIf
265 Next nI
266
267 Return Nil
268
269 //-----
270 /*{Protheus.doc} STBVldClient
271 Verifica se o cliente da venda possui criticas cadastrais
272
273 @param  aFinItens      Array com as informacoes de Servico Financeiro
274 @param  aCriticCli     Array que contera as criticas
275 @return
276 @obs
277 @sample
278 */
279 //-----
280 Function STBVldClient( aFinItens, aCriticCli )
281
282 Default aFinItens := {}
283 Default aCriticCli := {}
284
285 ParamType 0 Var aFinItens As Array Default {}
286 ParamType 1 Var aCriticCli As Array Default {}
287
288 /* Efetua validacao do cliente */
289 aCriticCli := aClone(STDVldClient(aFinItens))
290
291 Return Nil
292
293 //-----
294 /*{Protheus.doc} STBPermUser
295 Verifica se usuario tem permissao para vender Servicos Financeiros
296
297 @param
298 @return  lRet      Usuario possui permissao para vender Servico Financeiro
299 @obs
300 @sample
301 */
302 //-----
303 Function STBPermUser()
304
305 Local lRet := STFProFile(38)[1] //Retorno de permissao do usuario para Servico Financeiro
306
307 /* Se usuario sem permissao exibe mensagem */
308 If !lRet
309     If lUsaDisplay
310         // Inicia Evento
311         STFFireEvent(ProcName(0), "STDdisplay", {StatDisplay(), "2C" + STR0001}) //"Senha invalida ou acesso negado"
312     EndIf

```

```

313
314     STFMessage(ProcName(),"STOP",STR0002 + cUserName + STR0003) //"Atenção, " ##" sem permissão para vender Serviços Financeiros com crítica de cliente"
315     STFS showMessage(ProcName())
316 EndIf
317
318 Return lRet
319
320 //-----
321 /*{Protheus.doc} STBDelServFin
322 Deleta servico financeiro avulso ou atrelado do objeto
323
324 @param      cProdCode  Codigo do produto deletado
325 @param      nItem      Item do produto deletado
326 @param      lServFin    Servico Financeiro avulso
327 @return
328 @obs
329 @sample
330 */
331 //-----
332 Function STBDelServFin(cProdCode, nItem, lServFin)
333
334 Default cProdCode := "" //Codigo do Produto Vendido
335 Default nItem      := 0  //Numero de Item do Produto Vendido
336 Default lServFin    := .F. //Flag Servico Financeiro avulso
337
338 ParamType 0 Var cProdCode AS Character Default ""
339 ParamType 1 Var nItem     AS Numeric   Default 0
340 ParamType 2 Var lServFin  AS Logical   Default .F.
341
342 /* Se produto Servico Financeiro deleta o mesmo
343 Senao envia produto e item para delecao de servicos financeiros atrelados */
344 STWItemFin( 4 ,; //Tipo do Processo (1=Set - 2=Get - 3=Clear - 4=Del)
345             IIF(lServFin, cProdCode, "") ,; //Codigo Servico Financeiro
346             0 ,; //Valor do Servico Financeiro
347             IIF(lServFin, "", cProdCode) ,; //Codigo Produto Vendido
348             IIF(lServFin, 0, nItem) ,; //Item Produto Vendido
349             "" ) //Tipo Item - Usado para importacao de Orcamento
350
351 Return Nil
352
353 //-----
354 /* {Protheus.doc} STBSubNotFiscal
355 Adiciona valor no totalizador não fiscal
356
357 @param  nValue      Valor
358 @return Nil
359 @obs
360 @sample
361 */
362 //-----
363 Function STBSubNotFiscal( nValue )
364

```

```

365 Local oTotal := STFGetTot()      // Totalizador
366
367 Default nValue      := 0
368
369 ParamType 0 Var  nValue As Numeric  Default 0
370
371 oTotal:SetValue( "L1_NOTFISCAL" , oTotal:GetValue("L1_NOTFISCAL") - nValue )
372
373 Return Nil
374
375 //-----
376 /* {Protheus.doc} STBVldQtdProd
377 Valida quantidade do produto de cobertura no vinculo de Servico Financeiro
378
379 @param  nItemProd      Item do produto de cobertura
380 @return lRet           Permite continuar o vinculo
381 @obs
382 @sample
383 */
384 //-----
385 Function STBVldQtdProd(nItemProd)
386
387 Local lRet      := .T.           //Retorno da funcao
388 Local aLines    := FwSaveRows()  //Array de linhas
389 Local oModelCesta := STDGPBModel() //Model da cesta
390
391 /* Carrega Modelo Detalhes */
392 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
393
394 /* Posiciona na linha do produto cobertura*/
395 oModelCesta:GoLine(nItemProd)
396
397 /* Verifica quantidade do produto cobertura */
398 If oModelCesta:GetValue("L2_QUANT") > 1
399     lRet := .F.
400
401     STFMessage(ProcName(),"STOP", STR0005) //#"Quantidade do Produto de cobertura maior que 1, não permitido vincular"
402     STFShowMessage(ProcName())
403 EndIf
404
405 FwRestRows(aLines)
406
407 Return lRet

```

Componentes/FinancialService/STDFinService.prw

590 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STDFINSERVICE.CH"
5
6 //-----
7 /*{Protheus.doc} STDProdServ
8 Retorna array do produto serviço.
9
10 @param  cProductCode      Codigo do Produto
11 @param  cServiceCode      Codigo do Serviço
12 @return  aRet              Retorno do produto serviço
13 @obs
14 @sample
15 */
16 //-----
17 Function STDProdServ( cProductCode, cServiceCode )
18
19 Local      aArea           := GetArea()           // Guarda area atual
20 Local      aRet            := {}                  // Retorno do Serviço
21 Local      lSeek          := .F.                  // Encontrou o Serviço?
22
23 Default    cProductCode    := ""                  // Codigo do Produto
24 Default    cServiceCode    := ""                  // Codigo do Serviço
25
26 ParamType 0 Var cProductCode AS Character Default ""
27 ParamType 1 Var cServiceCode AS Character Default ""
28
29 If !EMPTY(cProductCode)
30
31     DbSelectArea("MG8")
32     DbSetOrder(2) //MG8_FILIAL+MG8_PRDSB1
33     lSeek := DbSeek ( xFilial("MG8") + cProductCode )
34
35 ElseIf !EMPTY(cServiceCode)
36
37     DbSelectArea("MG8")
38     DbSetOrder(1) //MG8_FILIAL+MG8_CODIGO
39     lSeek := DbSeek ( xFilial("MG8") + cServiceCode )
40
41 EndIf
42
43 If lSeek
44
45     Aadd( aRet , MG8->MG8_CODIGO           ) // [01]Cod. Servico
46     Aadd( aRet , MG8->MG8_DESCRI           ) // [02]Descricao
47     Aadd( aRet , MG8->MG8_PRDSB1           ) // [03]Prd.Serv. na venda
48     Aadd( aRet , MG8->MG8_TPPREC           ) // [04]Tipo de Precificação 1=Faixa de Preço;2=Fixo
49     Aadd( aRet , MG8->MG8_TPXPRD           ) // [05]Vinculado 1=Sim;2=Nao
50     Aadd( aRet , MG8->MG8_REGRA            ) // [06]Regra Cliente
51
52 EndIf

```

```

53
54 //Restaura area anterior
55 RestArea(aArea)
56
57 Return aRet
58
59 //-----
60 /*{Protheus.doc} STDServPrice
61 Retorna o valor do serviço baseado por Faixa de Preço
62
63 @param  cProdSerCode      Codigo do Produto Servico
64 @param  cProdSalCode      Codigo do Produto Vendido
65 @param  cGroupCode        Codigo do Grupo do Produto Vendido
66 @param  nProdPrice        Preco do Produto Vendido
67 @return nRet              Retorno do preço do produto servico
68 @obs
69 @sample
70 */
71 //-----
72 Function STDServPrice( cProdSerCode, cProdSalCode, cGroupCode, nProdPrice )
73
74 Local      aArea          := GetArea()           //Guarda area atual
75 Local      aAreaMBF       := MBF->(GetArea())    //Guarda area MBF atual
76 Local      aAreaMBL       := MBL->(GetArea())    //Guarda area MBL atual
77 Local      nRet           := 0                  //Retorno do preço do produto serviço
78 Local      lAchou         := .F.                //Flag se encontrou Faixa de Preço
79
80 Default    cProdSerCode   := ""                 //Codigo do Produto Servico
81 Default    cProdSalCode   := ""                 //Codigo do Produto Vendido
82 Default    cGroupCode     := ""                 //Codigo do Grupo do Produto da Venda
83 Default    nProdPrice     := 0                  //Preco do Produto da Venda
84
85 ParamType 0 Var cProdSerCode AS Character      Default ""
86 ParamType 1 Var cProdSalCode AS Character      Default ""
87 ParamType 2 Var cGroupCode  AS Character      Default ""
88 ParamType 3 Var nProdPrice  AS Numeric        Default 0
89
90 If !EMPTY(cProdSerCode)
91     //1 - Busca Faixa de Preço do Servico Financeiro pelo codigo do servico
92     MBF->(dbSetOrder(1)) //MBF_FILIAL + MBF_PRDGAR + MBF_PRODPR + MBF_GRUPO
93
94     If MBF->(dbSeek(xFilial("MBF") + cProdSerCode + cProdSalCode));
95         .AND. (dDataBase >= MBF->MBF_DTINI .AND. dDataBase <= MBF->MBF_DTFIM)
96
97         lAchou := .T.
98 Else //2 - Busca Faixa de Preço do Servico Financeiro pelo grupo
99     MBF->(dbSetOrder(5)) //MBF_FILIAL + MBF_GRUPO + MBF_PRODPR
100
101     If MBF->(dbSeek(xFilial("MBF") + cGroupCode))
102         While MBF->(EOF()) .AND. MBF->(MBF_FILIAL + MBF_GRUPO) == xFilial("MBF") + cGroupCode
103             If MBF->MBF_PRDGAR == cProdSerCode .AND. (dDataBase >= MBF->MBF_DTINI .AND. dDataBase <= MBF->MBF_DTFIM )
104                 lAchou := .T.

```

```

105             Exit
106         EndIf
107
108         MBF->(dbSkip())
109     EndDo
110 EndIf
111 EndIf
112
113 //Se encontrou Faixa de Preço, busca valor nos itens (MBF)
114 If lAchou
115     MBL->(dbSetOrder(1)) //MBL_FILIAL + MBL_CODIGO + MBL_ITEM
116
117     If MBL->(dbSeek(xFilial("MBL") + MBF->MBF_CODIGO))
118         While MBL->(EOF()) .AND. (MBL->MBL_FILIAL = xFilial("MBL")) .AND. (MBF->MBF_CODIGO == MBL->MBL_CODIGO)
119
120             //Armazena Valor do Serviço Financeiro
121             If nProdPrice >= MBL->MBL_VLINI .AND. nProdPrice <= MBL->MBL_VLFIM
122                 nRet := MBL->MBL_VALOR
123             EndIf
124
125             MBL->(dbSkip())
126         EndDo
127     EndIf
128 EndIf
129 EndIf
130
131 //Restaura areas
132 RestArea(aAreaMBF)
133 RestArea(aAreaMBL)
134 RestArea(aArea)
135
136 Return nRet
137
138 //-----
139 /*{Protheus.doc} STDProdInfo
140 Retorna o Preço/Grupo/Preço do Produto da Venda
141
142 @param   cProdCode      Código do Produto da Venda
143 @param   cGroupCode     Código do Grupo do Produto da Vena
144 @param   nProdPrice     Preço do Produto da Venda
145 @return  Nil
146 @obs
147 @sample
148 */
149 //-----
150 Function STDProdInfo( cProdCode, cGroupCode, nProdPrice )
151
152 Local aArea      := GetArea()           //Guarda area atual
153 Local aAreaSB1   := SB1->(GetArea())    //Guarda area atual
154 Local aLines     := FwSaveRows()        //Array de linhas
155 Local oModelCesta := STDGPBModel()      //Model da cesta
156

```

```

157 Default      cProdCode      := ""           //Codigo do Produto da Venda
158 Default      cGroupCode     := ""           //Codigo do Grupo do Produto da Venda
159 Default      nProdPrice     := 0            //Preco do Produto da Venda
160
161 ParamType 0 Var cProdCode    AS Character   Default ""
162 ParamType 1 Var cGroupCode  AS Character   Default ""
163 ParamType 2 Var nProdPrice  AS Numeric     Default 0
164
165 //Carrega Modelo Detalhes
166 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
167
168 //Armazena informacoes
169 cProdCode := oModelCesta:GetValue("L2_PRODUTO") //Codigo
170 cGroupCode := Posicione("SB1", 1, xFilial("SB1" + cProdCode), "B1_GRUP0") //Grupo
171 nProdPrice := STWFormPr(cProdCode) //Preco
172
173 //Restaura areas
174 FwRestRows(aLines)
175 RestArea(aAreaSB1)
176 RestArea(aArea)
177
178 Return Nil
179
180 //-----
181 /*{Protheus.doc} STDServItens
182 Busca todos os servicos Vinculados ao produto e armazena no objeto
183
184 @param cCodItem      Codigo do Produto Vendido
185 @param nItemLine     Item do Produto Vendido
186 @param cTypeItem     Tipo do Item - Importacao de Orcamentos
187 @return
188 @obs
189 @sample
190 */
191 //-----
192 Function STDServItens( cCodItem, nItemLine, cTypeItem )
193
194 Local aArea      := GetArea()           //Guarda area atual
195 Local aAreaSB1  := SB1->(GetArea())     //Guarda area SB1
196 Local cGroupCode := ""
197 Local nProdPrice := 0
198
199 Default cCodItem := "" //Codigo do Produto da Venda
200 Default nItemLine := 0 //Item do Produto da Venda
201 Default cTypeItem := "" //Tipo do Item - Importacao de Orcamentos
202
203 ParamType 0 Var cCodItem AS Character   Default ""
204 ParamType 1 Var nItemLine AS Numeric     Default 0
205 ParamType 2 Var cTypeItem AS Character   Default ""
206
207 If cTypeItem <> "IMP" //Se veio da importação, não insiro todos os serviços financeiros escolhido pelo cliente.
208 /*

```

```

209     Busca Grupo do Produto da Venda
210     */
211     DbSelectArea("SB1")
212     SB1->(DbSetOrder(1)) //B1_FILIAL+B1_COD
213     DbSeek( xFilial("SB1") + cCodItem )
214     cGroupCode := SB1->B1_GRUPO //Grupo
215
216     /*
217     Busca Preco do Produto da Venda
218     */
219     nProdPrice := STWFormPr( cCodItem )
220
221     //Valida se produto permite Servicos Financeiros Vinculados
222     If !EMPTY(cCodItem) .AND. Posicione("SB0", 1, xFilial("SB0") + Padr(cCodItem, TamSx3("B0_COD")[1]), "SB0->B0_SERVFIN") == "1"
223     /*
224     Busca Faixa de Preco do Servico Financeiro pelo codigo do produto
225     */
226     MBF->(dbSetOrder(4)) //MBF_FILIAL + MBF_PRODPR + MBF_GRUPO
227     MBL->(dbSetOrder(1)) //MBL_FILIAL + MBL_CODIGO + MBL_ITEM
228
229     If MBF->(dbSeek(xFilial("MBF") + cCodItem)) ;
230     .AND. (dDataBase >= MBF->MBF_DTINI .AND. dDataBase <= MBF->MBF_DTFIM);
231     .AND. STBIsFinService(MBF->MBF_PRDGAR)
232
233     While MBF->(!EOF()) .AND. MBF->(MBF_FILIAL + MBF_PRODPR) == xFilial("MBF") + cCodItem ;
234     .AND. (dDataBase >= MBF->MBF_DTINI .AND. dDataBase <= MBF->MBF_DTFIM)
235
236     If MBL->(dbSeek(xFilial("MBL") + MBF->MBF_CODIGO))
237     While MBL->(!EOF()) .AND. (MBL->MBL_FILIAL = xFilial("MBL")) .AND. (MBF->MBF_CODIGO == MBL->MBL_CODIGO)
238     If (MBL->MBL_FILIAL = xFilial("MBL")) .AND. (MBF->MBF_CODIGO == MBL->MBL_CODIGO);
239     .AND. nProdPrice >= MBL->MBL_VLINI .AND. nProdPrice <= MBL->MBL_VLFIM
240     /*
241     Armazena Itens Servico Financeiro
242     */
243     STWItemFin( 1 ; //Tipo do Processo (1=Set - 2=Get - 3=Clear)
244     MBF->MBF_PRDGAR ; //Codigo Servico Financeiro
245     MBL->MBL_VALOR ; //Valor do Servico Financeiro
246     cCodItem ; //Codigo Produto Vendido
247     nItemLine ; //Item Produto Vendido
248     cTypeItem ) //Tipo Item - Usado para importacao de Orcamento
249
250     EndIf
251     MBL->(dbSkip())
252     EndDo
253     EndIf
254
255     MBF->(dbSkip())
256     EndDo
257     EndIf
258
259     /*
260     Busca Faixa de Preco do Servico Financeiro pelo grupo

```

```

261 */
262 MBF->(dbSetOrder(5)) //MBF_FILIAL + MBF_GRUPO + MBF_PRODPR
263
264 If !Empty(cGroupCode) .AND. MBF->(dbSeek(xFilial("MBF") + cGroupCode)) ;
265 .AND. (dDataBase >= MBF->MBF_DTINI .AND. dDataBase <= MBF->MBF_DTFIM);
266 .AND. STBIsFinService(MBF->MBF_PRDGAR)
267
268 While MBF->(!EOF()) .AND. MBF->(MBF_FILIAL + MBF_GRUPO) == xFilial("MBF") + cGroupCode ;
269 .AND. (dDataBase >= MBF->MBF_DTINI .AND. dDataBase <= MBF->MBF_DTFIM)
270 If MBL->(dbSeek(xFilial("MBL") + MBF->MBF_CODIGO))
271 While MBL->(!EOF()) .AND. (MBL->MBL_FILIAL = xFilial("MBL")) .AND. (MBF->MBF_CODIGO == MBL->MBL_CODIGO)
272 If (MBL->MBL_FILIAL = xFilial("MBL")) .AND. (MBF->MBF_CODIGO == MBL->MBL_CODIGO);
273 .AND. nProdPrice >= MBL->MBL_VLINI .AND. nProdPrice <= MBL->MBL_VLFIM
274 /*
275 Armazena Itens Servico Financeiro
276 */
277 STWItemFin(1 ; //Tipo do Processo (1=Set - 2=Get - 3=Clear)
278 MBF->MBF_PRDGAR ; //Codigo Servico Financeiro
279 MBL->MBL_VALOR ; //Valor do Servico Financeiro
280 cCodItem ; //Codigo Produto Vendido
281 nItemLine ; //Item Produto Vendido
282 cTypeItem ) //Tipo Item - Usado para importacao de Orcamento
283
284 EndIf
285 MBL->(dbSkip())
286 EndDo
287 EndIf
288
289 MBF->(dbSkip())
290 EndDo
291 EndIf
292
293 EndIf
294
295 //Restaura areas
296 RestArea(aAreaSB1)
297 RestArea(aArea)
298
299 EndIf
300
301 Return Nil
302
303 //-----
304 /*{Protheus.doc} STDDescProd
305 Retorna a Descricao do Produto
306
307 @param cCodProd Codigo do Produto
308 @return cDescProd Descricao do Produto
309 @obs
310 @sample
311 */
312 //-----

```

```

313 Function STDDescProd(cCodProd)
314
315 Local aArea      := GetArea()    //Guarda area atual
316 Local cDescProd  := ""          //Descricao do produto
317
318 Default cCodProd := ""
319
320 ParamType 0 Var cCodProd AS Character Default ""
321
322 SB1->(dbSetOrder(1))
323
324 /*
325     Descricao do Produto
326 */
327 If !Empty(cCodProd) .AND. SB1->(dbSeek(xFilial("SB1") + cCodProd))
328     cDescProd := AllTrim(SB1->B1_DESC)
329 EndIf
330
331 /*
332     Restaura areas
333 */
334 RestArea(aArea)
335
336 Return cDescProd
337
338 //-----
339 /*{Protheus.doc} STDVldClient
340 Function para Buscar as criticas cadastrais do Cliente quando Item Servico Financeiro
341
342 @param aFinItens    Array com as informacoes de Servico Financeiro
343 @return aRet        Array com as criticas cadastrais do cliente
344 @obs
345 @sample
346 */
347 //-----
348 Function STDVldClient(aFinItens)
349
350 Local aArea      := GetArea()          //Guarda area atual
351 Local aAreaSA1   := SA1->(GetArea())    //Guarda area SA1
352 Local aAreaMG7   := MG7->(GetArea())    //Guarda area MG7
353 Local aAreaMG8   := MG8->(GetArea())    //Guarda area MG8
354 Local cMsgCritic := ""                //Mensagem de critica
355 Local aRet       := {}                 //Array com Retorno das criticas cadastrais do cliente
356 Local nI         := 0                  //Contador
357 Local xRet       := Nil                 //Retorno da Formula
358 Local cCustomer  := STDGPBasket("SL1", "L1_CLIENTE") //Codigo do Cliente
359 Local cStore     := STDGPBasket("SL1", "L1_LOJA")    //Loja do Cliente
360
361 Default aFinItens := {}
362
363 ParamType 0 Var aFinItens AS Array Default {}
364

```

```

365 SA1->(dbSetOrder(1)) //A1_FILIAL + A1_COD + A1_LOJA
366 MG7->(dbSetOrder(1)) //MG7_FILIAL + MG7_CODIGO + MG7_ITEM
367 MG8->(dbSetOrder(2)) //MG8_FILIAL + MG8_PRDSB1
368
369 /* Valida se os campos do cadastro de cliente estao preenchidos */
370 If SA1->(dbSeek(xFilial("SA1") + cCustomer + cStore))
371     For nI := 1 to Len(aFinItens)
372         /* Valida se item nao esta deletado e se existe regra de cliente */
373         If !aFinItens[nI][8] .AND. MG8->(dbSeek(xFilial("MG8") + aFinItens[nI][1])) .AND. MG7->(dbSeek(xFilial("MG7") + MG8->MG8_REGRA))
374             While MG7->(!EOF()) .AND. MG7->MG7_CODIGO == MG8->MG8_REGRA
375                 /* Valida Campos Obrigatorios */
376                 If !Empty(MG7->MG7_CAMPO)
377                     If Empty(&("SA1->" + MG7->MG7_CAMPO))
378                         /* Adiciona criticas de campos no array */
379                         If Empty(MG7->MG7_MSGPD)
380                             SX3->(DbSetOrder(2))
381                             SX3->(DbSeek(MG7->MG7_CAMPO))
382
383                             /* Armazena mensagem de critica */
384                             cMsgCritic := STR0001 + Alltrim(X3Titulo()) + " (" + Alltrim(MG7->MG7_CAMPO) + ") " + STR0002 //#"0 campo " ##"não esta preenchido."
385
386                             /* Adiciona mensagem no Array se nao for repetida */
387                             If aScan(aRet,{|x| x == cMsgCritic}) == 0
388                                 aAdd(aRet, cMsgCritic)
389                             EndIf
390                         Else
391                             /* Armazena mensagem de critica */
392                             cMsgCritic := Alltrim(MG7->MG7_MSGPD)
393
394                             /* Adiciona mensagem no Array se nao for repetida */
395                             If aScan(aRet,{|x| x == cMsgCritic}) == 0
396                                 aAdd(aRet, cMsgCritic)
397                             EndIf
398                         EndIf
399                     EndIf
400             EndIf
401
402             /* Valida Formulas */
403             If !Empty(MG7->MG7_FORMUL)
404                 xRet:= &(Alltrim(MG7->MG7_FORMUL))
405
406                 /* Valida se Formula eh valida */
407                 If ValType(xRet) <> "L"
408                     If Empty(MG7->MG7_MSGPD)
409                         /* Armazena mensagem de critica */
410                         cMsgCritic := STR0003 //"Erro ao executar a fórmula - Retorno deve ser Lógico."
411
412                         /* Adiciona mensagem no Array se nao for repetida */
413                         If aScan(aRet,{|x| x == cMsgCritic}) == 0
414                             aAdd(aRet, cMsgCritic)
415                         EndIf
416                     Else

```

```

417             /* Armazena mensagem de critica */
418             cMsgCritic := Alltrim(MG7->MG7_MSGPD)
419
420             /* Adiciona mensagem no Array se nao for repetida */
421             If aScan(aRet,{|x| x == cMsgCritic}) == 0
422                 aAdd(aRet, cMsgCritic)
423             EndIf
424         EndIf
425     Else
426         /* Adiciona criticas de formulas no array */
427         If !xRet
428             If Empty(MG7->MG7_MSGPD)
429                 /* Armazena mensagem de critica */
430                 cMsgCritic := STR0004 //"Validação de fórmula - Mensagem não preenchida"
431
432                 /* Adiciona mensagem no Array se nao for repetida */
433                 If aScan(aRet,{|x| x == cMsgCritic}) == 0
434                     aAdd(aRet, cMsgCritic)
435                 EndIf
436             Else
437                 /* Armazena mensagem de critica */
438                 cMsgCritic := Alltrim(MG7->MG7_MSGPD)
439
440                 /* Adiciona mensagem no Array se nao for repetida */
441                 If aScan(aRet,{|x| x == cMsgCritic}) == 0
442                     aAdd(aRet, cMsgCritic)
443                 EndIf
444             EndIf
445         EndIf
446     EndIf
447 EndIf
448
449 MG7->(dbSkip())
450 EndDo
451 EndIf
452 Next nI
453 EndIf
454
455 /* Restaura areas */
456 RestArea(aAreaSA1)
457 RestArea(aAreaMG7)
458 RestArea(aAreaMG8)
459 RestArea(aArea)
460
461 Return aRet
462
463 //-----
464 /*{Protheus.doc} STDPrintServ
465 Function para Buscar alimentar Rdmake dos Relatorios Gerenciais - Servico Financeiro
466
467 @param aSelIten      Array com o item selecionado de Servico Financeiro
468 @param aRelSF        Array que contera os itens selecionados para impressao

```

```

469 @return
470 @obs
471 @sample
472 /*/
473 //-----
474 Function STDPrintServ(aSelIten, aRelSF)
475
476 Local cImpRGer := SuperGetMV("MV_RELGART",, "U_LOJR600A") //Rdmake de Impressao
477 Local aArea := GetArea() //Guarda area atual
478 Local aAreaMG8 := MG8->(GetArea()) //Guarda area MG8
479
480 Default aSelIten := {}
481 Default aRelSF := {}
482
483 ParamType 0 Var aSelIten AS Array Default {}
484 ParamType 1 Var aRelSF AS Array Default {}
485
486 /* Busca Cadastro de Servicos Financeiros */
487 MG8->(dbSetOrder(2))
488
489 If MG8->(DbSeek(xFilial("MG8") + aSelIten[1]))
490 /* Armazena Rdmake */
491 If !Empty(MG8->MG8_RDMAKE)
492 cImpRGer := MG8->MG8_RDMAKE
493 EndIf
494
495 /* Array com informacoes para impressao */
496 aAdd(aRelSF,{aSelIten[4], aSelIten[5], Str(a410Arred(aSelIten[3] * 1,"L2_VLRITEM"), TamSx3("L2_VLRITEM")[1],;
497 TamSx3("L2_VLRITEM")[2]), ""/*SERIE*/, MG8->MG8_CODIGO, AllTrim(MG8->MG8_DESCRI), ""/*Vlr.SF*/, cImpRGer})
498 EndIf
499
500 /* Restaura areas */
501 RestArea(aAreaMG8)
502 RestArea(aArea)
503
504 Return Nil
505
506 //-----
507 /*/{Protheus.doc} STDUpdServFin
508 Function para atualizar campos de Servicos Financeiros vinculados a produtos
509
510 @param cItemPrd Item do produto vendido
511 @param cCodPrd Codigo do produto vendido
512 @return
513 @obs
514 @sample
515 /*/
516 //-----
517 Function STDUpdServFin(cItemPrd, cCodPrd)
518
519 Local aArea := GetArea() //Guarda area atual
520 Local aAreaSL2 := SL2->(GetArea()) //Guarda area SL2

```

```

521
522 Default cItemPrd := ""
523 Default cCodPrd := ""
524
525 ParamType 0 Var cItemPrd AS Character Default ""
526 ParamType 1 Var cCodPrd AS Character Default ""
527
528 /* Atualiza campos com produto de venda */
529 RecLock("SL2", .F.)
530 SL2->L2_ITEMCOB := cItemPrd
531 SL2->L2_PRDCOBE := cCodPrd
532 SL2->(MsUnlock())
533
534 /* Restaura areas */
535 RestArea(aAreaSL2)
536 RestArea(aArea)
537
538 Return Nil
539
540 //-----
541 /*{Protheus.doc} STDFinVinc
542 Function para Verificar se Produto Financeiro vinculado e sem amarracao
543
544 @param oMdlIte Objeto Cesta de produtos
545 @param nItem Item anterior ao produto
546 @return
547 @obs
548 @sample
549 */
550 //-----
551
552 Function STDFinVinc(oMdlIte, nItem)
553
554 Local aArea := GetArea() //Guarda area atual
555 Local aAreaSL2 := SL2->(GetArea()) //Guarda area SL2
556 Local aLines := FwSaveRows() //Array de linhas
557 Local nI := nItem - 1 //Contador do item anterior
558
559 Default oMdlIte := Nil
560 Default nItem := 0
561
562 ParamType 0 Var oMdlIte As Object Default Nil
563 ParamType 1 Var nItem AS Numeric Default 0
564
565 /* Verifica se item anterior e a cobertura do Servico Financeiro */
566 While nI > 0
567     oMdlIte:GoLine(nI)
568
569     /* Valida se produto cobertura nao e Servico Financeiro */
570     If !STBIsFinService(STDGPBasket("SL2", "L2_PRODUTO"))
571         RecLock("SL2", .F.)
572         SL2->L2_ITEMCOB := STDGPBasket("SL2", "L2_ITEM")

```

```
573         SL2->L2_PRDCOBE := STDGPBasket("SL2", "L2_PRODUTO")
574         MsUnlock()
575
576         Exit
577     EndIf
578
579     nI--
580 EndDo
581
582 /* Restaura posicao do item */
583 oMdlIte:GoLine(nItem)
584
585 /* Restaura areas */
586 FwRestRows(aLines)
587 RestArea(aAreaSL2)
588 RestArea(aArea)
589
590 Return Nil
```

Componentes/FinancialService/STWFinService.prw

335 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWFINSERVICE.CH"
5
6 Static oFinItens := Adm_Item_FinService():New() //Objeto para armazenamento de informacoes de Itens Servicos Financeiros
7
8 //-----
9 /*{Protheus.doc} STWValidService
10 Function Valida Item de Serviço.
11
12 @param nType          Tipo de Validação
13 @param aInfoItem      Array do Item
14 @param cProductCode   Código do Produto
15 @param cCliCode       Código do Cliente
16 @param cCliStore      Loja do Cliente
17 @return lRet          Validação do Item
18 @obs nType == 01      Valida se o produto serviço é Avulso (.T.) ou Vinculado (.F.)
19      nType == 02      Valida se o produto por Faixa de Preço
20      nType == 03      Valida se o cliente não for padrão para serviço financeiro
21 @sample
22 */
23 //-----
24 Function STWValidService( nType, aInfoItem, cProductCode, cCliCode,;
25                          cCliStore )
26
27 Local lRet := .F. // Retorno
28
29 Default nType := 1 // Tipo de Validação (1-Avulso?)
30 Default aInfoItem := {} // Array com informação do item
31 Default cProductCode := "" // Código do Produto
32 Default cCliCode := "" // Código do Cliente
33 Default cCliStore := "" // Loja do Cliente
34
35 ParamType 0 Var nType As Numeric Default 1
36 ParamType 1 Var aInfoItem As Array Default {}
37 ParamType 2 Var cProductCode As Character Default ""
38 ParamType 3 Var cCliCode As Character Default ""
39 ParamType 4 Var cCliStore As Character Default ""
40
41
42 Do Case
43
44 Case nType == 1 //Retorna se o produto serviço é avulso (.T.) ou vinculado (.F.)
45
46 If !EMPTY(aInfoItem) .AND. !EMPTY(aInfoItem[ITEM_CODIGO])
47 lRet := STBIsBulkServ(aInfoItem[ITEM_CODIGO])
48 EndIf
49
50 Case nType == 2 //Retorna se o produto serviço é por Faixa de Preço
51
52 If !EMPTY(aInfoItem) .AND. !EMPTY(aInfoItem[ITEM_CODIGO])

```

```

53         lRet := STBIsPriceRange(aInfoItem[ITEM_CODIGO])
54     EndIf
55
56     Case nType == 3 //Valida se o cliente não for padrão para serviço financeiro
57
58         lRet := !( AllTrim(cCliCode) == SuperGetMv("MV_CLIPAD") .AND. AllTrim(cCliStore) == SuperGetMV("MV_LOJAPAD") )
59
60     Otherwise
61
62         lRet := .F.
63
64 EndCase
65
66 Return lRet
67
68
69 //-----
70 /*{Protheus.doc} STWServPrice
71 Function Retorna preco de serviço quando faixa de preço.
72
73 @param  cItemCode      Produto
74 @param  cFil           Filial
75 @return nRet          Preço do serviço
76 @obs
77 @sample
78 */
79 //-----
80 Function STWServPrice( cItemCode, cFil )
81
82 Local      aInfoItem      := {}                      //Array do Produto de Serviço
83 Local      lFinServ       := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
84 Local      lContinua     := lFinServ                //Define se continua o processo
85 Local      cServType     := SuperGetMv("MV_LJTSPF",,"SF") // Define o tipo do produto de servico financeiro
86 Local      nRet          := 0                      //Retorna o preço da faixa de preço
87 Local      cProdCode     := ""                     //Codigo Produto da Venda
88 Local      cGroupCode    := ""                     //Grupo Produto da Venda
89 Local      nProdPrice    := 0                      //Preco Produto da Venda
90
91 Default   cItemCode      := ""
92 Default   cFil           := ""
93
94 ParamType 0 Var cItemCode As Character Default ""
95 ParamType 1 Var cFil     As Character Default ""
96
97 If lContinua
98
99     //Verifica se é produto serviço financeiro.
100     lContinua := Posicione("SB1", 1, xFilial("SB1" + cItemCode), "B1_TIP0") == cServType
101
102     //Verifica se o produto é Vinculado e por Faixa de Preço.
103     If lContinua
104         aInfoItem := STDProdServ(cItemCode)

```

```

105
106      //Vinculado?      Avulso (.T.) ou Vinculado (.F.)
107      lContinua      := !( STWValidService(1,aInfoItem) )
108
109      //Caso sim, busca preço pela faixa de preço do produto Vinculado.
110      //Mesmo quando precificação for Fixo, o preço virá da tabela de Faixa de Preço com registro unico (MBL).
111      If lContinua
112          //Busca codigo/grupo/Preço do produto da venda
113          STDProdInfo( @cProdCode, @cGroupCode, @nProdPrice )
114
115          nRet := STDServPrice( aInfoItem[3], cProdCode, cGroupCode, nProdPrice )
116      EndIf
117  EndIf
118
119 EndIf
120
121 Return nRet
122
123 //-----
124 /*{Protheus.doc} STWItemFin
125 Function Armazena, Retorna ou Limpa Item Servico Financeiro em Array
126
127 @param  nTipoProc      Tipo do Processo (1=Set - 2=Get - 3=Clear)
128 @param  cCodServFin     Produto Servico
129 @param  nPriceServFin   Valor do Produto Servico
130 @param  cCodItem        Produto da Venda
131 @param  nItemLine       Item do Produto da Venda
132 @param  cTypeItem       Tipo do Item - Usado na importacao de orcamento
133 @return lRet           Servico Financeiro Avulso
134 @obs
135 @sample
136 */
137 //-----
138 Function STWItemFin( nTipoProc, cCodServFin, nPriceServFin, cCodItem, nItemLine, cTypeItem )
139
140 Local xRet      := Nil      // Retorno da função
141
142 Default nTipoProc      := 1      //Tipo do Processo (1=Set - 2=Get - 3=Clear - 4=Del - 5=Sel)
143 Default cCodServFin     := ""     //Produto Servico
144 Default nPriceServFin   := 0      //Valor do Produto Servico
145 Default cCodItem        := ""     //Produto da Venda
146 Default nItemLine       := 0      //Item do Produto da Venda
147 Default cTypeItem       := ""     //Tipo do Item - Usado na importacao de orcamento
148
149 ParamType 0 Var nTipoProc      As Numeric Default 0
150 ParamType 1 Var cCodServFin     As Character Default ""
151 ParamType 2 Var nPriceServFin   As Numeric Default 0
152 ParamType 3 Var cCodItem        As Character Default ""
153 ParamType 4 Var nItemLine       As Numeric Default 0
154 ParamType 5 Var cTypeItem       As Character Default ""
155
156 /*

```

```

157   Armazena Item Servico Financeiro
158 */
159 If Valtype(oFinItens) <> "U" //Valida se objeto foi instanciado
160   If nTipoProc == 1
161     xRet := oFinItens:SetServFin(cCodServFin, nPriceServFin, cCodItem, nItemLine, cTypeItem)
162   ElseIf nTipoProc == 2
163     xRet := oFinItens:GetServFin(nItemLine)
164   ElseIf nTipoProc == 3
165     xRet := oFinItens:ClearServFin()
166   ElseIf nTipoProc == 4
167     xRet := oFinItens:DelServFin(cCodServFin, cCodItem, nItemLine)
168   ElseIf nTipoProc == 5
169     xRet := oFinItens:SelServFin(cCodServFin, cCodItem, nItemLine, nPriceServFin)
170   EndIf
171 EndIf
172
173 Return xRet
174
175 //-----
176 /*/{Protheus.doc} STWItemFin
177 Funcao para impressao de relatorios gerenciais Servico Financeiro
178
179 @param
180 @return
181 @obs
182 @sample
183 */
184 //-----
185 Function STWPrintServ()
186
187 Local aRelSF      := {}
188 Local nVias       := 2
189 Local nCont       := 0
190 Local cTexto      := ""
191 Local cImpRGer     := ""
192 Local aFinItens   := STWItemFin(2) //Tipo do Processo (1=Set - 2=Get - 3=Clear - 4=Del - 5=Sel)
193
194 If ExistFunc("STDPrintServ")
195   For nCont := 1 To Len(aFinItens)
196     /* Servico Financeiro habilitado e selecionado */
197     If !aFinItens[nCont][8] .And. aFinItens[nCont][9]
198       /* Alimenta Rdmake dos Relatorios */
199       STDPrintServ(aFinItens[nCont], @aRelSF)
200     EndIf
201   Next nCont
202 EndIf
203
204 /*
205   Efetua a impressao
206 */
207 For nCont := 1 To Len(aRelSF)
208   cImpRGer := aRelSF[nCont][8]

```

```

209
210   If FindFunction(cImpRGer)
211     If cImpRGer == SuperGetMV("MV_RELGART",,"U_LOJR600A")
212       cTexto := chr(16) + &cImpRGer.(aRelSF[nCont][1],aRelSF[nCont][2],aRelSF[nCont][3],aRelSF[nCont][4],;
213         aRelSF[nCont][5],aRelSF[nCont][6],aRelSF[nCont][7],SA1->A1_NOME,1,.T.,.T.)
214     Else
215       cTexto := chr(16) + &cImpRGer.(SL1->L1_NUM, Nil)
216     EndIf
217     //Relatorio da conferencia de caixa
218     STWManagReportPrint( cTexto ,nVias )
219   EndIf
220 Next nCont
221
222 Return Nil
223
224 //-----
225 /*{Protheus.doc} STWFindServ
226 Funcao para busca de Servicos Financeiros vinculados
227
228 @param1
229 @return
230 @obs
231 @sample
232 */
233 //-----
234 Function STWFindServ()
235
236 Local aFinItens      := {} //Array que contera os Servicos Financeiros vinculados
237 Local aCriticCli     := {} //Array que contera as criticas ao cadastro do cliente
238 Local lSTWNxtTelaIt := ExistFunc("STWNxtTelaIt")
239
240 //Verifica se existem Servicos Financeiros Vinculados
241 aFinItens := STWItemFin(2) //Tipo do Processo (1=Set - 2=Get - 3=Clear)
242
243 // Apresento tela para escolha de serviços quando existir serviços Vinculados (xVar[06]) e não deletados (xVar[08])
244 // Quando a posição 06 do array estiver 0, significa que não é serviço Vinculado á um produto.
245 If LEN(aFinItens) > 0 .AND. aScan(aFinItens,{|xVar| xVar[06] <> 0;
246   .AND. xVar[07] <> "IMP" .AND. xVar[08] == .F.}) > 0
247   STIExchangePanel( { || STIFinService(aFinItens, aCriticCli) } )
248 ElseIf LEN(aFinItens) > 0 .AND. aScan(aFinItens,{|xVar| xVar[06] == 0;
249   .AND. xVar[07] <> "IMP" .AND. xVar[08] == .F.}) > 0
250   /* Efetua validacao do cliente */
251   STBVldClient(aFinItens, aCriticCli)
252
253   If Len(aCriticCli) > 0
254     /* Servico Financeiro - Mensagens da validacao do cliente */
255     STIExchangePanel({|| STIFinClient(aCriticCli)})
256   Else
257     IIf(lSTWNxtTelaIt,STWNxtTelaIt(),STICallPayment())
258   EndIf
259 Else
260   IIf(lSTWNxtTelaIt,STWNxtTelaIt(),STICallPayment())

```

```

261 EndIf
262
263 Return Nil
264
265 //-----
266 /*{Protheus.doc} STWFindTot
267 Funcao para busca do Totalizador Nao Fiscal
268
269 @param
270 @return Totalizador Nao Fiscal Servicos Financeiros
271 @obs
272 @sample
273 */
274 //-----
275
276 Function STWFindTot()
277
278 Local cRet := ""           //Totalizador Nao Fiscal
279 Local aRet := {}          //Array com informacoes Nao Fiscais
280 Local aDados := {Space(30)} //Dados de retorno da funcao
281
282 //Verifica se utiliza TotvsApi
283 aRet := STFFireEvent(ProcName(0) ,;
284                     "STRetTypePrt",;
285                     aDados)
286
287 //Busca Totalizador Nao Fiscal
288 If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0 .AND. ValType(aDados[1]) == "C"
289     If aDados[1] == "TOTVSAPI" //TotvsApi
290         cRet := GetPvProfString("Microsiga", "TotalizadorPedido", "RECEBER", GetClientDir() + "TOTVSAPI.INI")
291     Else //SigaLoja
292         cRet := GetPvProfString("Microsiga", "TotalizadorPedido", "RECEBER", GetClientDir() + "SIGALOJA.INI")
293     EndIf
294 EndIf
295
296 Return cRet
297
298 //-----
299 /*{Protheus.doc} STWDescServ
300 Funcao para validar desconto no Total quando venda possui Servicos Financeiros
301
302 @param
303 @return Se Permite desconto no total
304 @obs
305 @sample
306 */
307 //-----
308
309 Function STWDescServ()
310
311 Local lRet := .T.           //Retorno da validacao
312 Local nI := 0              //Contador

```

```
313 Local aLines          := FwSaveRows() //Array de linhas
314 Local oModelCesta     := STDGPBModel() //Model da cesta
315 Local oModelItens := oModelCesta:GetModel("SL2DETAIL") //Model com os itens da cesta
316
317 /* Verifica se existem Servicos Financeiros na Cesta */
318 For nI := 1 To oModelItens:Length()
319     /* Posiciona na linha do produto cobertura*/
320     oModelItens:GoLine(nI)
321
322     /* Verifica quantidade do produto cobertura */
323     If STBIsFinService(oModelItens:GetValue("L2_PRODUTO")) .And. !oModelItens:IsDeleted()
324         lRet := .F.
325
326         STFMessage("STITotalDiscount","STOP", STR0001) //#"Não é possível desconto na venda pois contem Servicos Financeiros"
327         STFShowMessage("STITotalDiscount")
328
329         Exit
330     EndIf
331 Next nI
332
333 FwRestRows(aLines)
334
335 Return lRet
```

Componentes/FinishSale/STDFinishSale.prw

486 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWMVCDEF.CH"
4
5 Static lPDVOnline := ExistFunc("STFGetOnPdv") .AND. STFGetOnPdv() // Variável para definir se o totvs PDV está no modo online
6 //-----
7 /*{Protheus.doc} STDFinishSale
8 Componente de gravacao de dados no Model e persistencia do Model no banco de dados.
9 @param lEmitNFe, Indica se esta emitindo NF-e
10 @param lRetArea , Indica se a função STWVldRgNfe() retornou a area.
11 @return Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STDFinishSale(lEmitNFe, lRetArea)
17
18 Local lRMS := SuperGetMv("MV_LJ RMS",,.F.) //Integracao RMS ativada
19 Local cRetorno := "" //Retorno da data da impressora
20 Local lCentPDV := LjGetCPDV()[1] //Server é Central de Pdv -
21 //Permitido Somente quando o Totvs Pdv é Online
22
23 Default lEmitNFe := .F. // Se emite NF-e
24 Default lRetArea := .F. // Se teve area selecionada no retorno da função STWVldRgNfe()
25
26 If STDGPBasket("SL1","L1_TIPO") == "P" .And. Empty(STDGPBasket("SL1","L1_PEDRES")) .And. !Empty(STDGPBasket("SL1","L1_ORCRES"))
27     STDSPBasket("SL1","L1_TIPO" , "P")
28 Else
29     STDSPBasket("SL1","L1_TIPO" , "V")
30 EndIf
31 If SL1->(FieldPos("L1_TIMEATE")) > 0 .AND. ValType(STDGPBasket("SL1","L1_TIMEATE")) == 'N'
32     STDSPBasket("SL1","L1_TIMEATE" , Seconds() - STDGPBasket("SL1","L1_TIMEATE") )
33 EndIf
34
35 If STBGetNFCe()
36     STDSPBasket("SL1", "L1_HORA", SL1->L1_HORA)
37 ElseIf !STFGetCfg("lPafEcfc") .AND. !lRMS
38     STDSPBasket("SL1","L1_HORA" , Left(Time(),TamSX3("L1_HORA")[1]) )
39 EndIf
40
41 STDSPBasket("SL1","L1_VALPIS" , STBTaxRet(Nil,"NF_VALPIS"))
42 STDSPBasket("SL1","L1_VALCOFI" , STBTaxRet(Nil,"NF_VALCOF"))
43 STDSPBasket("SL1","L1_VALCSLL" , STBTaxRet(Nil,"NF_VALCSL"))
44
45 If ExistBlock("STSaleComp")
46     /*
47         P.E com objetivo de gravar alguma informação complementar na venda antes de subir para a retaguarda.
48         Sera executado no final da venda, depois que já transmitiu o documento e antes de gravar o L1_SITUA = 00
49     */
50     LjGrvLog( "L1_NUM: " + STDGPBasket('SL1','L1_NUM'), "Antes P.E STSaleComp | L1_SITUA: " + STDGPBasket('SL1','L1_SITUA') )
51     ExecBlock("STSaleComp",.F.,.F.,{STDGPBasket('SL1','L1_FILIAL'),STDGPBasket('SL1','L1_NUM'),STDGPBasket('SL1','L1_DOC'),STDGPBasket('SL1','L1_SERIE')}}) //Filial, número do
    orçamento, doc. e série

```

```

52     LjGrvLog( "L1_NUM: " + STDGPBasket('SL1','L1_NUM'), "Depois P.E STSaleComp | L1_SITUA: " + STDGPBasket('SL1','L1_SITUA') )
53 EndIf
54
55 If lPDVOnline
56     If lEmiteNFe .AND. !lCentPDV
57         STDSPBasket("SL1","L1_SITUA", "OK")
58     Elseif lCentPDV //Se Totvs Pdv está conectado no Central de Pdv L1_SITUA deve ser "CP" para que Central envie a venda para RET.
59         STDSPBasket("SL1","L1_SITUA", "CP")
60     Else
61         STDSPBasket("SL1","L1_SITUA", "RX")
62     EndIf
63 Else
64     STDSPBasket("SL1","L1_SITUA", "00")
65 EndIf
66 STDSPBasket("SL1","L1_CONFVEN" , "SSSSSSSSNSSS")
67
68 if !lEmiteNFe
69     STDSPBasket("SL1","L1_IMPRIME" , "1S")
70 Endif
71
72
73 If lRMS
74     nRet := STWPrinterStatus( '19', @cRetorno )
75     If nRet <> 1 .AND. !Empty(cRetorno)
76         STDSPBasket("SL1","L1_DTM0V" , CToD(cRetorno) )
77     Else
78         STDSPBasket("SL1","L1_DTM0V" , dDataBase )
79     EndIf
80     STDSPBasket("SL1","L1_DTFIM" , Date())
81     STDSPBasket("SL1","L1_HRFIM" , SubStr(Time(),1,8))
82 EndIf
83
84 // Doação para o Instituto Arredondar
85 If SL1->(FieldPos( "L1_VLRARR" )) > 0
86     STIInsArrTax(STDGPBasket("SL4","L4_FORMA"),STDGPBasket("SL4","L4_ADMINIS"))
87     STDSPBasket("SL1","L1_VLRARR", STBGetInsArr()) //Valor com desconto
88 EndIf
89
90 If Empty(STDGPBasket("SL1","L1_CONDPG"))
91     STDSCondPG("CN")
92 EndIf
93
94 //STDSPBasket("SL1","L1_TABELA" , ) TODO:
95 //STDSPBasket("SL1","L1_NUMORC" , )
96 //STDSPBasket("SL1","L1_TPORC" , )
97
98 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Faz gravação final da venda." ) //Gera LOG
99
100 /* Quando a variavel lRetArea está como .T. significa que é PDV online com importação de orçamento
101 com retira ou entrega, quando temos emissão de NF-e no PDV online processamos a venda pelo mini gravabatch,
102 com isso toda a venda já está processada neste momento, por esse motivo não chamamos a função STDSaveSale().
103 No caso da central com PDV online ele atualiza a situação para CP e o job processa a venda, por esse motivo deve ser chamado.

```

```

104 */
105 If !(lRetArea .And. lEmitaNFe .And. !LjGetCPDV())[1])
106     STDSaveSale()
107 EndIf
108
109 //Quando é PDV Online e for recebimento de mais de um orçamento, precisa atualizar os registros
110 //da SL1 originais. mais detalhes vide a propria rotina TDAltRegL1()
111 If (lPDVOnline .AND. STBIsImpOrc() .AND. ExistFunc("STDAltRegL1"),STDAltRegL1(),NIL)
112 */
113     Motivo de Desconto
114 */
115
116 STDUdpReason(STDGPBasket( "SL1", "L1_DOC" ),STDGPBasket( "SL1", "L1_SERIE" )) // -- Atualiza DOC apos emissão.
117 STDRecReason()
118
119 */
120     Motivo de Venda Perdida
121 */
122 STDRecLostSale()
123
124 */
125     Desconto na próxima venda
126 */
127 STDRecNextSaleDiscount()
128
129 */
130     Verifica se houve cancelamento de item na venda para ajustar a SLX
131 */
132 If ExistFunc('STDAjstSLX')
133     STDAjstSLX(SL1->L1_DOC)
134 EndIf
135
136 Return Nil
137
138 //-----
139 /*{Protheus.doc} STDSaveSale
140 Faz a persistencia dos dados nas tabelas SL1, SL2 e SL4.
141
142 @param
143 @return aSale[1]          Retorna Cabeçalho da Venda (SL1)
144 @return aSale[2]          Retorna Itens da Venda (SL2)
145 @return aSale[3]          Retorna Formas de Pagamento da venda (SL4)
146 @obs
147 @sample
148 */
149 //-----
150
151 Function STDSaveSale(nItemLine,lCancel,lBrinde)
152
153 Local aArea          := GetArea()
154 Local aRows           := FwSaveRows()
155 Local oModel          := STDGPBModel()

```

```

156 Local aSL1Fields      := oModel:GetModel("SL1MASTER"):GetStruct():GetFields()
157 Local aSL2Fields      := oModel:GetModel("SL2DETAIL"):GetStruct():GetFields()
158 Local aSL4Fields      := oModel:GetModel("SL4DETAIL"):GetStruct():GetFields()
159 Local cField          := ""
160 Local nX              := 0
161 Local nY              := 0
162 Local lIsPAF          := STBIsPAF()
163 Local lMD5SL1         := lIsPAF .AND. SL1->(ColumnPos("L1_PAFMD5")) > 0
164 Local lMD5SL2         := lIsPAF .AND. SL2->(ColumnPos("L2_PAFMD5")) > 0
165 Local lMD5SL4         := lIsPAF .AND. SL4->(ColumnPos("L4_PAFMD5")) > 0
166 Local cPAFMD5         := ""
167 Local uValor          := ""
168 Local cFieldSL4       := "" //Guarda Valor dos campos da SL4 na inclusao para Log
169 Local cTipouValor     := "" // Tipo da variavel uValor
170 Local cValorField     := "" // Valor da variavel uValor em caracter
171 Local lFoundSL2       := .F.
172
173 DEFAULT nItemLine := 0
174 DEFAULT lCancel    := .F.
175 DEFAULT lBrinde    := .F.
176
177
178 Begin Transaction
179
180   DbSelectArea("SL1")
181   SL1->(DbSetOrder(1))
182
183   LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Grava tabela SL1 | L1_SITUA: " + STDGPBasket('SL1','L1_SITUA') ) //Gera LOG
184
185   If SL1->( DbSeek(xFilial("SL1") + oModel:GetValue("SL1MASTER", "L1_NUM")) )
186     RecLock("SL1",.F.)
187   Else
188     RecLock("SL1",.T.)
189   EndIf
190
191   For nX := 1 To Len(aSL1Fields)
192     cField := aSL1Fields[nX,MODEL_FIELD_IDFIELD]
193     REPLACE &(cField) WITH STDGPBasket( "SL1" , cField )
194   Next nX
195
196   If lMD5SL1
197     cPAFMD5 := STBPAFMD5("SL1")
198     REPLACE SL1->L1_PAFMD5 WITH cPAFMD5
199   EndIf
200
201   SL1->(MsUnlock())
202   SL1->(ConfirmSX8())
203
204   LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Quantidade de itens no Model da tabela SL2", oModel:GetModel("SL2DETAIL"):Length() ) //Gera LOG
205
206
207   If nItemLine > 0

```

```

208 DbSelectArea("SL2")
209 SL2->(DbSetOrder(1)) //L2_FILIAL + L2_NUM + L2_ITEM + L2_PRODUTO
210 oModel:GetModel("SL2DETAIL"):GoLine(nItemLine)
211
212 lFoundSL2 := SL2->(DbSeek(xFilial("SL2") + oModel:GetValue("SL1MASTER" , "L1_NUM") + STDGPBasket( "SL2" , "L2_ITEM" )))
213
214 If !oModel:GetModel("SL2DETAIL"):IsDeleted() .Or. lIsPAF
215
216     If oModel:GetModel("SL2DETAIL"):IsDeleted()
217         SET DELETED OFF
218         lFoundSL2 := SL2->(DbSeek(xFilial("SL2") + oModel:GetValue("SL1MASTER" , "L1_NUM") + STDGPBasket( "SL2" , "L2_ITEM" )),.T.)
219         SET DELETED ON
220     EndIf
221
222     If lFoundSL2
223         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Tabela SL2 - Alteração do item", SL2->L2_ITEM ) //Gera LOG
224         /// Caso o item seja cancelado, Eh alterado o L2_SITUA para sinalizar que o mesmo esta deletado.
225         RecLock("SL2",.F.)//alteracao
226     Else
227         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Tabela SL2 - Inclusão de novo item", SL2->L2_ITEM ) //Gera LOG
228         RecLock("SL2",.T.)//Inclusao
229     EndIf
230
231     For nX := 1 To Len(aSL2Fields)
232         cField := aSL2Fields[nX,MODEL_FIELD_IDFIELD]
233         REPLACE &(cField) WITH STDGPBasket( "SL2" , cField )
234     Next nX
235
236     If lMD5SL2
237         cPAFMD5 := STBPAFMD5("SL2")
238         REPLACE L2_PAFMD5 WITH cPAFMD5
239     EndIf
240
241     SL2->(MsUnlock())
242
243     If lCancel .AND. lFoundSL2
244         RecLock("SL2",.F.)
245         SL2->( DbDelete() )
246         SL2->(MsUnlock())
247     EndIf
248 Else
249     If lCancel .AND. lFoundSL2
250         RecLock("SL2",.F.)
251         SL2->( DbDelete() )
252         SL2->(MsUnlock())
253     EndIf
254 EndIf
255
256 LjGrvLog("L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "L2_PRODUTO " , SL2->L2_PRODUTO) //Gera LOG
257
258 EndIf
259

```

```

260  /*/
261  Se L1_SITUA for igual a '10', significa que o SL2 ja foi salvo e nao sera alterado
262  /*/
263
264  If oModel:GetValue("SL1MASTER" , "L1_SITUA") $ "10|65"
265  LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "L1_SITUA 10|65") //Gera LOG
266  DbSelectArea("SL2")
267  SL2->(DbSetOrder(1))
268  SL2->( dbGoTop() )
269  For nX := 1 To oModel:GetModel("SL2DETAIL"):Length()
270  oModel:GetModel("SL2DETAIL"):GoLine(nX)
271
272  If oModel:GetModel("SL2DETAIL"):IsDeleted(nX) ;
273  .AND. SL2->( DbSeek(xFilial("SL2") + oModel:GetValue("SL1MASTER", "L1_NUM") + STDGPBasket("SL2", "L2_ITEM")) ) ;
274  .AND. STDGPBasket("SL2" , "L2_VENDIDO") == "N"
275
276  While SL2->( !Eof() ) .AND.( xFilial("SL2") + oModel:GetValue("SL1MASTER", "L1_NUM") + STDGPBasket("SL2", "L2_ITEM" ) == SL2->L2_FILIAL + SL2->L2_NUM +
SL2->L2_ITEM)
277
278  If SL2->L2_VENDIDO == "N"
279  RecLock("SL2",.F.)
280  SL2->( DbDelete() )
281  SL2->( MsUnlock() )
282  EndIf
283  SL2->(DbSkip())
284
285  End
286
287  EndIf
288  Next nX
289  EndIf
290
291  /*/
292  Se L1_SITUA for igual a '10', significa que o SL4 ja foi salvo e nao sera alterado
293  /*/
294
295  If oModel:GetValue("SL1MASTER" , "L1_SITUA") == "10" .AND. nItemLine == 0
296  DbSelectArea("SL4")
297  SL4->(DbSetOrder(1)) //L4_FILIAL + L4_NUM
298  If ExistFunc("STDDeLPay") .And. SL4->( DbSeek(xFilial("SL4") + oModel:GetValue("SL1MASTER", "L1_NUM")) ) // é recuperação de venda e já gravou L4
299  While !Eof() .AND. (L4_FILIAL + L4_NUM) == (xFilial("SL4") + oModel:GetValue("SL1MASTER" , "L1_NUM"))
300  STDDelPay(oModel:GetValue("SL1MASTER", "L1_NUM")) //deletar o registro da SL4 anterior
301  SL4->( DbSkip() )
302  EndDo
303  EndIf
304  LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Quantidade de itens no Model da tabela SL4", oModel:GetModel("SL4DETAIL"):Length() )
305  LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Quantidade de Campos no Model da tabela SL4: " + AllTrim(STR(Len(aSL4Fields))) )
306  If STDPBLength("SL4") > 0
307  DbSelectArea("SL4")
308  SL4->( DbSetOrder(1) )//L4_FILIAL+L4_NUM+L4_ORIGEM
309  For nX := 1 To STDPBLength("SL4")
310  oModel:GetModel("SL4DETAIL"):GoLine(nX)

```

```

311
312 //Verificação para não gravar linha em branco na SL4 (Gravava em branco quando utilizado NCC).
313 If !Empty(STDGPBasket( "SL4" , "L4_FORMA"))
314
315     cFieldSL4 := ""
316
317     If RecLock("SL4",.T.)
318         For nY := 1 To Len(aSL4Fields)
319             cField := aSL4Fields[nY,MODEL_FIELD_IDFIELD]
320             uValor := STDGPBasket( "SL4" , cField)
321             REPLACE SL4->&(cField) WITH uValor
322
323             cTipouValor := AllTrim( UPPER( ValType(uValor) ))
324             If cTipouValor == "U"
325                 cValorField := "Indefinido"
326             Else
327                 cValorField := AllTrim(cValToChar(uValor))
328             EndIf
329             cFieldSL4 += cField+"=" + cValorField + " | "
330         Next nY
331
332         If LMD5SL4
333             //REPLACE L4_DOC WITH oModel:GetValue("SL1MASTER" , "L1_DOC")
334             cPAFMD5 := STBPAFMD5("SL4")
335             REPLACE SL4->L4_PAFMD5 WITH cPAFMD5
336             cFieldSL4 += "L4_PAFMD5=" + cPAFMD5
337         EndIf
338         SL4->(MsUnlock())
339     Else
340         LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Falha no RecLock do alias SL4" )
341     EndIf
342 Else
343     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Forma de pagamento vazia L4_FORMA")
344 EndIf
345
346 Next nX
347 Else
348     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Não existem parcelas a gravar na Tabela SL4" )
349 EndIf
350
351 ElseIf oModel:GetValue("SL1MASTER" , "L1_SITUA") == "00" .AND. !Empty( oModel:GetValue("SL1MASTER" , "L1_CONTONF") )
352
353     DbSelectArea("SL4")
354     SL4->(DbSetOrder(1)) //L4_FILIAL + L4_NUM
355     If SL4->( DbSeek(xFilial("SL4") + oModel:GetValue("SL1MASTER", "L1_NUM")) )
356         While SL4->( !Eof() ) .AND. (L4_FILIAL + L4_NUM) == (xFilial("SL4") + oModel:GetValue("SL1MASTER" , "L1_NUM"))
357             If !Empty(L4_DATATEF)
358                 RecLock("SL4", .F.)
359
360                 If SL4->(FieldPos("L4_CONTONF")) > 0
361                     REPLACE L4_CONTONF With oModel:GetValue("SL1MASTER" , "L1_CONTONF")
362                 EndIf

```

```

363
364         If SL4->(FieldPos("L4_DOC")) > 0
365             REPLACE L4_DOC WITH STBRetCup()
366         EndIf
367
368         If LMD5SL4
369             cPAFMD5 := STBPAFMD5("SL4")
370             REPLACE L4_PAFMD5 WITH cPAFMD5
371         EndIf
372         SL4->(MsUnlock())
373     EndIf
374     SL4->( DbSkip() )
375 EndDo
376
377 EndIf
378 EndIf
379 End Transaction
380
381 RestArea(aArea)
382 FwRestRows(aRows)
383
384 // Limpa os vetores para melhor gerenciamento de memoria (Desaloca Memória)
385 aSize( aRows, 0 )
386 aRows := Nil
387
388 Return
389
390
391
392 //-----
393 /*{Protheus.doc} STDGrvMGX
394 Grava forma de pagamento e valor na tabela de controle para limite de Sangria.
395
396 @param
397 @return .T.
398 @obs
399 @sample
400 */
401 //-----
402
403 Function STDGrvMGX()
404
405 Local oModel          := STDGPBModel() // Objeto Model
406 Local nX              := 0 // Contador de registros SL4
407 Local nValor          := 0 // Valor atual do campo MGX_VALOR posicionado
408 Local aArea           := GetArea()
409
410 // Gravação da Tabela MGX para controle de Limite de Sangria
411 For nX := 1 To oModel:GetModel("SL4DETAIL"):Length()
412     oModel:GetModel("SL4DETAIL"):GoLine(nX)
413     DbSelectArea("MGX")
414     DbSetOrder(1) // MGX_FILIAL+MGX_FPAGTO+DTOS(MGX_DATA)

```

```

415   If DbSeek (xFilial("MGX") + Substr(oModel:GetValue("SL4DETAIL" , "L4_FORMA"),1,3))
416       Reclock("MGX", .F.) // Alteração
417       oModel:GetModel("SL4DETAIL"):GoLine(nX)
418       MGX->MGX_FPAGTO := oModel:GetValue("SL4DETAIL" , "L4_FORMA")
419       nValor := MGX->MGX_VALOR
420       MGX->MGX_VALOR := (nValor + oModel:GetValue("SL4DETAIL" , "L4_VALOR"))
421   Else
422       Reclock("MGX", .T.) // Inclusão
423       oModel:GetModel("SL4DETAIL"):GoLine(nX)
424       MGX->MGX_FPAGTO := oModel:GetValue("SL4DETAIL" , "L4_FORMA")
425       MGX->MGX_VALOR := oModel:GetValue("SL4DETAIL" , "L4_VALOR")
426   EndIf
427 Next nX
428 MsUnlock()
429
430 RestArea(aArea)
431
432 Return .T.
433
434 //-----
435 /*{Protheus.doc} STDNfCeSt
436 Alimenta as variáveis estáticas utilizadas na nfce
437
438 @param
439 @return .T.
440 @obs
441 @sample
442 */
443 //-----
444
445 Function STDNfCeSt()
446 Local oModel      := STDGPBModel() //Model da Venda
447 Local nX := 0      //Variavel Contadora
448 Local aNfCeICMST := {} //Array de Dados da NFce ST
449 Local aArea := GetArea() //WorkArea
450
451
452 DbSelectArea("SL2")
453 SL2->( DbSetOrder(1) )
454 SL2->( dbGoTop() )
455 For nX := 1 To oModel:GetModel("SL2DETAIL"):Length()
456     oModel:GetModel("SL2DETAIL"):GoLine(nX)
457     If !oModel:GetModel("SL2DETAIL"):IsDeleted(nX) .AND. ;
458         SL2->( DbSeek(xFilial("SL2") + oModel:GetValue("SL1MASTER", "L1_NUM") + STDGPBasket("SL2", "L2_ITEM")) )
459
460     While SL2->( !EoF() ) .AND. ;
461         xFilial("SL2") + oModel:GetValue("SL1MASTER", "L1_NUM") + STDGPBasket("SL2", "L2_ITEM") == SL2->L2_FILIAL + SL2->L2_NUM + SL2->L2_ITEM
462         aNfCeICMST := Array(2)
463
464         aNfCeICMST[1] := STDGPBasket( "SL2" , "L2_ITEM" )
465         aNfCeICMST[2] := {}
466

```

```
467      Aadd( aNFceICMST[2], STBTaxRet(nX, "IT_MARGEM" ) )
468      Aadd( aNFceICMST[2], STBTaxRet(nX, "IT_PREDST" ) )
469      Aadd( aNFceICMST[2], STBTaxRet(nX, "IT_BASESOL" ) )
470      Aadd( aNFceICMST[2], STBTaxRet(nX, "IT_ALIQSOL" ) )
471      Aadd( aNFceICMST[2], STBTaxRet(nX, "IT_VALSOL" ) )
472
473      //alimenta o array estatico (aICMSST) do LOJNFCE.PRW
474      If FindFunction("LjSetICMST")
475          LjSetICMST( aNFceICMST )
476      EndIf
477
478      SL2->( DbSkip() )
479  End
480
481  EndIf
482 Next nX
483
484 RestArea(aArea)
485
486 Return
```

Componentes/FinishSale/STDSendSale.PRW

318 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STDSendSale
6 Transmite a venda para retaguarda caso esteja online
7
8 @param      cL1NUM - Numero Orcamento
9
10 @return      Nil
11 @obs
12 @sample
13 */
14 //-----
15
16 Function STDSendSale(cL1_NUM,cEstacao,lStarJob,cEmp,cFil)
17
18 Local lRet      := .T.
19 Local lGravouOk := .F.
20 Local lEncontrou := .T. // Se encontrou registro
21 Local aSL1      := {} // array com dados da SL1
22 Local aSL2      := {} // array com dados da SL2
23 Local aSL4      := {} // array com dados da SL4
24 Local lHasConnect := .T. // Verifica se tem comunicacao
25 Local cStatusNum := ""
26 Local cNum       := ""
27 Local cNumSaleConf := ""
28 Local nCodRet    := 0
29 Local lComCPDV   := IIf( FindFunction("LjGetCPDV"), LjGetCPDV()[2] , .F. ) // Usa comunicacao com a central
30 Local lGrvRet    := .F. //Informa que gravou a venda com sucesso na retaguarda quando venda online e central de PDVs
31 Local lOk        := .T.
32
33 Default cL1_NUM := ""
34 Default cEstacao := "001"
35 Default lStarJob := .F.
36 Default cEmp     := cEmpAnt
37 Default cFil     := cFilAnt
38
39
40 If lStarJob
41     RpcSetType(3)
42     RpcSetEnv(cEmp,cFil,Nil,Nil,"SALE")
43 EndIf
44
45 aSL1 := STDSL1SEND(@lEncontrou, @cL1_NUM) //Recebe o Número do Orçamento gerado
46
47 If lEncontrou
48     aSL2 := STDSL2Create(@lEncontrou, cL1_NUM) //Passa o Número do Orçamento gerado
49 EndIf
50
51 If lEncontrou
52     aSL4 := STDSL4Create(@lEncontrou, cL1_NUM) //Passa o Numero do Orçamento gerado

```

```

53 EndIf
54
55 //Reserva registro caso JOB STWUPDATA seja iniciado
56 RecLock("SL1", .F.)
57 REPLACE SL1->L1_SITUA With "TX"
58 SL1->( MsUnlock() )
59
60 If lEncontrou
61
62     lGravouOk := STBRemoteExecute("GeraL1L2L4", {aSL1, aSL2, aSL4, cEstacao } , NIL, .F., @cStatusNum, , , @nCodRet)
63     CONOUT("STDSEndSale - L.60 - DEPOIS STBRemoteExecute(GeraL1L2L4) - lGravouOk: "+AllToChar(lGravouOk))
64     lHasConnect := !(nCodRet == -105 .OR. nCodRet == -107 .OR. nCodRet == -104)
65
66     If ValType( cStatusNum ) == "C" .AND. lHasConnect //tem comunic com a Ret
67         CONOUT("STDSEndSale - L.64 - lHasConnect: " +AllToChar(lHasConnect)+"cStatusNum: "+AllToChar(cStatusNum))
68         If Left( cStatusNum, 2 ) == "OK"
69             CONOUT("STDSEndSale - L.66 - cStatusNum: "+AllToChar(cStatusNum))
70             cNum := Substr( cStatusNum, 4, 6 )
71             CONOUT("STDSEndSale - L.68 - cNum: "+AllToChar(cNum))
72             If !Empty( cNum )
73                 lContinua := STBRemoteExecute("ConfL1L2L4", { cNum , cEstacao } , NIL, .F. , @cNumSaleConf )
74                 CONOUT("STDSEndSale - L.72 - DEPOIS STBRemoteExecute(ConfL1L2L4) - lContinua: "+AllToChar(lContinua))
75                 If AllTrim(cNum) == AllTrim(cNumSaleConf)
76                     CONOUT("STDSEndSale - L.74 - cNum: " +AllToChar(cNum)+"cNumSaleConf: "+AllToChar(cNumSaleConf))
77                     DbSelectArea("SL1")
78                     SL1->( DbSetOrder(1) ) //L1_FILIAL+L1_NUM
79                     If SL1->( DbSeek(xFilial("SL1")+cL1_NUM))
80                         RecLock("SL1", .F.)
81                         REPLACE SL1->L1_SITUA With "TX"
82                         REPLACE SL1->L1_NUMORIG With cNum
83                         SL1->( MsUnlock() )
84                     EndIf
85
86                     //Tratamento para os PDV conectados a Central de PDVs
87                     If lComCPDV
88                         cStatusNum := ""
89                         cNum := ""
90                         lGravouOk := STBRemoteExecute(" GeraL1L2L4", {aSL1, aSL2, aSL4, cEstacao } , NIL, .F., @cStatusNum, , , @nCodRet)
91                         CONOUT("STDSEndSale - L.88 - DEPOIS STBRemoteExecute(_GeraL1L2L4) NA CENTRAL - lGravouOk: "+AllToChar(lGravouOk))
92                         If ValType( cStatusNum ) == "C"
93                             CONOUT("STDSEndSale - L.90 - cStatusNum: "+AllToChar(cStatusNum))
94                             If Left( cStatusNum, 2 ) == "OK"
95                                 cNum := Substr( cStatusNum, 4, 6 )
96                                 lGrvRet := .T.
97                                 If !Empty( cNum )
98                                     STBRemoteExecute("_ConfL1L2L4", { cNum , cEstacao, lGrvRet } , NIL, .F. , @cNumSaleConf )
99                                     CONOUT("STDSEndSale - L.96 - DEPOIS STBRemoteExecute(_ConfL1L2L4) NA CENTRAL - cNumSaleConf: "+AllToChar(cNumSaleConf))
100                                 EndIf
101                             EndIf
102                         EndIf
103                     EndIf
104                 Else

```

```

105         lOk := .F.
106     EndIf
107     Else
108         lOk := .F.
109     EndIf
110     Else
111         lOk := .F.
112     EndIf
113     Else
114         lOk := .F.
115     EndIf
116 Else
117     lOk := .F.
118 EndIf
119
120 If !lOk
121     RecLock("SL1", .F.)
122     REPLACE SL1->L1_SITUA With "00"
123     SL1->( MsUnlock() )
124 EndIf
125
126 Return lRet
127
128 //-----
129 /*{Protheus.doc} STDSL1SEND
130 Busca Vendas para subir ao BackOffice
131
132 @param lEncontrou      Indica se encontrou a venda - Variavel usada por referencia
133
134 @return aSL1          array com dados da SL1
135 @obs
136 @sample
137 */
138 //-----
139 Function STDSL1SEND( lEncontrou, cL1_NUM )
140
141 Local aArea           := GetArea()      // Armazena alias corrente
142 Local nCampos         := 0              // Quantidade de campos do filtro atual
143 Local aSL1           := {}             // Array do Alias
144 Local i               := 0              // Contador
145
146 Default lEncontrou := .F.
147 Default cL1_NUM    := ""
148
149 lEncontrou := .F.
150
151 DbSelectArea("SL1")
152 SL1->( DbSetOrder(1) ) //L1_FILIAL+L1_NUM
153
154 lEncontrou := SL1->( DbSeek(xFilial("SL1")+cL1_NUM) )
155
156 If lEncontrou

```

```

157     cL1_NUM := SL1->L1_NUM
158
159     nCampos := FCount()
160     aSL1 := Array(nCampos)
161
162     For i := 1 To nCampos
163         aSL1[i] := {FieldName(i), FieldGet(i)}
164     Next i
165
166     // Salva origem da venda
167     If SL1->L1_OPERACA != "C"
168         aSL1[AScan(aSL1,{|x|x[1]=="L1_NUMORIG"})][2] := SL1->L1_NUM
169     EndIf
170
171 EndIf
172
173 RestArea(aArea)
174
175 Return aSL1
176
177 //-----
178 /*{Protheus.doc} STDSENDCLI
179 Transmite o cliente para retaguarda caso esteja online
180
181 @param      aSA1 - Dados do cliente enviado para execucao
182 @param      lSendCli - retorno se o cliente foi incluido com sucesso
183
184 @return      Nil
185 @obs
186 @sample
187 */
188 //-----
189
190 Function STDSENDCLI(aSA1,lSendCli,lStarJob,cEmp,cFil)
191
192 Local lHasConnect := .T. // Verifica se tem comunicacao
193 Local lContinua := .F.
194 Local lExistCli := .F.
195 Local lRetServer := .F.
196 Local lSendCli := .F.
197 Local nRecServer := 0
198 Local nX := 0
199 Local xRet := ""
200 Local aData := {}
201 Local lCentPDV := IIf( FindFunction("LjGetCPDV"), LjGetCPDV()[1] , .F. ) // Eh Central de PDV - CENTRAL TOTVSPDV
202
203 Default aSA1 := {}
204 Default lSendCli := .F.
205 Default lStarJob := .F.
206 Default cEmp := cEmpAnt
207 Default cFil := cFilAnt
208

```

```

209 If lStarJob
210     RpcSetType(3)
211     RpcSetEnv(cEmp,cFil,Nil,Nil,"CLI")
212 Endif
213
214 If Len(aSA1) > 0
215     lContinua := STBRemoteExecute( "STDPesqCli" , { xFilial("SA1"), aSA1[1][2], aSA1[2][2] } , NIL, .F. , @lExistCli )
216 EndIf
217
218 If lContinua .AND. !lExistCli
219
220     If !lCentPDV
221         DbSelectArea("SA1")
222         SA1->(DbSetOrder(1)) //A1_FILIAL+A1_COD+A1_LOJA
223
224         If SA1->(DbSeek(xFilial("SA1")+aSA1[1][2]+aSA1[2][2]))
225
226             For nX := 1 To SA1->(FCount())
227                 If AllTrim(Fieldname( nX )) == 'A1_SITUA'
228                     AAdd( aData , { FieldName( nX ), '' } )
229                 Else
230                     AAdd( aData , { FieldName( nX ), FieldGet( nX ) } )
231                 EndIf
232             Next nX
233
234         EndIf
235     Else
236         aData := aClone(aSA1)
237     EndIf
238
239     If Len(aData) > 0
240
241         lContinua := STBRemoteExecute( "STDRecXData" , { 'SA1' , aData } , NIL, .F. , @xRet )
242
243         //Trata o retorno xRet
244         If lContinua .AND. ValType(xRet) == "A"
245             lRetServer := xRet[1] == "OK"
246             nRecServer := xRet[2]
247         ElseIf lContinua .AND. ValType(xRet) == "L"
248             lRetServer := xRet
249         EndIf
250
251         If lContinua .AND. lRetServer
252             RecLock('SA1',.F.)
253             SA1->A1_SITUA := 'TX'
254             MsUnlock()
255             lSendCli := .T.
256         ElseIf lContinua .AND. !lRetServer
257             RecLock('SA1',.F.)
258             SA1->A1_SITUA := 'ER'
259             MsUnlock()
260         EndIf

```

```

261
262     EndIf
263
264 EndIf
265
266 Return
267
268 //-----
269 /*{Protheus.doc} STDSendRZ
270 Transmite o cliente para retaguarda caso esteja online
271
272 @param      aData - Dados da reducao Z enviada para retaguarda
273 @param      lSendRZ - retorno se a reducao Z foi incluida com sucesso
274
275 @return     Nil
276 @obs
277 @sample
278 */
279 //-----
280
281 Function STDSendRZ(aData,lSendRZ,lStarJob,cEmp,cFil)
282
283 Local lContinua      := .F.
284 Local lRetServer     := .F.
285 Local nRecServer     := 0
286 Local nX             := 0
287 Local xRet           := ""
288
289 Default aData        := {}
290 Default lSendRZ      := .F.
291 Default lStarJob     := .F.
292 Default cEmp         := cEmpAnt
293 Default cFil         := cFilAnt
294
295 If lStarJob
296     RpcSetType(3)
297     RpcSetEnv(cEmp,cFil,Nil,Nil,"REDZ")
298 Endif
299
300 If Len(aData) > 0
301
302     lContinua := STBRemoteExecute( "GeraSFI" , { aData } , NIL, .F. , @xRet )
303
304     //Trata o retorno xRet
305     If lContinua .AND. ValType(xRet) == "A"
306         lRetServer := xRet[1] == "OK"
307         nRecServer := xRet[2]
308     ElseIf lContinua .AND. ValType(xRet) == "L"
309         lRetServer := xRet
310     EndIf
311
312     If lContinua .AND. lRetServer

```

```
313         lSendRZ := .T.  
314     EndIf  
315  
316 EndIf  
317  
318 Return
```

Componentes/FinishSale/STWFinishSale.prw

808 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWFINISHSALE.CH"
4
5 Static cStDoc      := ""
6 Static cStSerie    := ""
7 Static cSiglaSat   := IIF( ExistFunc("LjSiglaSat"),LjSiglaSat(), "SAT" ) //Retorna sigla do equipamento que esta sendo utilizado
8 Static lPDVOnline  := ExistFunc("STFGetOnPdv") .AND. STFGetOnPdv() // Variável para definir se o totvs PDV está no modo online
9
10 /*/{Protheus.doc} STWFinishSale
11 Workflow de finalizacao de venda.
12 @type      function
13 @param     lPendSale, logico, indica se a venda pode ser finalizada novamente caso haja erro
14 @return    Nil
15 */
16 Function STWFinishSale(lPendSale)
17
18 Local nRet      := 0 // Retorno
19 Local lContinua := .T. // Controle de processo
20 Local nTotalNCCs := STDGetNCCs("2") // Valor total das NCCs
21 Local cDoc      := "" // Documento
22 Local cSerie    := "" // Serie do Documento Fiscal
23 Local cACli     := "" // Cliente
24 Local cALojaCli := "" // Loja do Cliente
25 Local cACPFcli  := "" // CPF/CNPJ do Cliente
26 Local aRet      := {}
27 Local aDados    := {"23", space(SLG->(TamSx3("LG_CRO")[1])) }
28 Local lArredondar := SuperGetMV( "MV_LJINSAR",, .F.) // Configuração para ativar doação para o Instituto Arredondar
29 Local nArredondar := 0 // Valor da doação para o Instituto Arredondar
30 Local lEmitNfce   := Iif(FindFunction("LjEmitNFce"), LjEmitNFce(), .F.) // Sinaliza se utiliza NFC-e
31 Local cKeyNfce    := ""
32 Local nRetNfce    := -1 //Sinaliza se transmitiu a NFCe
33 Local lPrintNFCE := .T. // Se imprime Danfe Nfc-e
34 Local lLimSang    := SuperGetMV( "MV_LJLISAN",, .F.) // Utiliza controle para limite de sangria
35 Local aNotas      := {}
36 Local nItem       := 0
37 Local nTotItSL2   := 0
38 Local lMobile     := STFGetCfg("lMobile", .F.) //Smart Client Mobile
39 Local cXML        := ""
40 Local lSAT        := STFGetCfg("lUseSAT",.F.)
41 Local cPass       := STFGetStat("CODSAT",,.F.) //Retorna o código de ativação do SAT
42 Local cDocSat     := "" //numero de documento da venda SAT
43 Local cSerieSat   := "" //numero de serie do equipamento SAT
44 Local lSaveOrc    := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento
45 Local lRet        := .F.
46 Local cAFormPg    := "" //Forma de pagamento
47 Local cMsgErro    := "" //Motivo da rejeição
48 Local lEmitNFE    := .F. //Valida se gerar NF-e
49 Local oCliModel   := STDGCLiModel() // Model do Cliente
50 Local lCliPadrao   := (AllTrim(oCliModel:GetValue("SA1MASTER","A1_COD"))+AllTrim(oCliModel:GetValue("SA1MASTER","A1_LOJA"))) == ( AllTrim( SuperGetMV("MV_CLIPAD", .F., "")) )
+ AllTrim( SuperGetMV("MV_LOJAPAD", .F., "")) )
51 Local aRetTSS     := {}

```

```

52 Local cRetTSS      := ""
53 Local cMsgTSS      := ""
54 Local cRetTrans     := ""
55 Local cMVL0JANF     := AllTrim( SuperGetMV("MV_LOJANF", .F. ,"UNI") )
56 Local nDocSai       := SuperGetMv("MV_LJLBNT", .F., 0)
57 Local lMVFISNOTA    := SuperGetMV("MV_FISNOTA", .F., .F.) .and. !Empty(cMVL0JANF) .and. nDocSai > 0
58 Local lLjDNota      := .F.
59 Local oRaas         := Nil
60 Local oFidelityC    := Nil
61 Local lLjGetModNf    := ExistFunc("LjGetModNf")
62 Local lSemRetXml     := .F. // Define se poderá ou não finalizar novamente a venda NFC-e que será no modo de contingencia.
63 Local lNaoGrvSLX     := .F. // Quando retorna do TSS "nota ja autorizada" mas a chave NFCE são diferentes então signifca q a nota e serie já foram utilizadas em outra venda
64                      // nesse caso não pode inutilizar nem gravar SLX.
65 Local lDelPg        := !(ExistFunc("STFGetOnPdv") .AND. STFGetOnPdv() .AND. !Empty(STDGPBasket("SL1" , "L1_NUMORIG")))
66 Local aDescPgto     := {}
67 Local aItensFid     := {}
68 Local aFormPgto     := {}
69 Local nX            := 0
70 Local aAreaSL4      := {}
71 Local aAreaSL1      := {}
72 Local lRetArea      := .F. // Retorna se teve area selecionada no retorno da função STWVldRgNfe()
73
74 Default lPendSale   := .F. //Em caso de erro, indica se a venda fica pendente para ser finalizada de novo
75
76 lMobile := ( ValType(lMobile) == "L" .AND. lMobile )
77
78 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Inicio - Workflow de finalizacao de venda." ) //Gera LOG
79
80 //tratamento para verificar se está no modo SAT
81 lSAT := IIf( ValType(lSAT) == "U", .F., lSAT)
82
83 // ?? TO D0: Cancelar cupom caso não haja item registrado.
84
85 //nRet := IFStatus(nHdleCF, "9", @cRet) // Verifico o Status do ECF
86 If nRet <> 0
87     // "Erro com a Impressora Fiscal. Operação não efetuada.", "Atenção"
88     HELP(' ',1,'FRT011')
89
90     lContinua := .F.
91 EndIf
92
93 If lContinua
94
95     // TO D0: Chamar funcao de rateio de desconto e acrescimo e Arredondamento da impressora - Verificar com o Rondon.
96
97     // TO D0: Monta FORMAS DE PAGAMENTO que serao enviadas para a impressora, com seus valores - Bruno
98
99     // TO D0: Atualizar totais do SL1 e Gravar L1_SITUA COM "09"
100
101     cDoc := STDGPBasket("SL1","L1_DOC")
102
103     If Empty(cDoc)

```

```

104      cDoc := STDGPBasket("SL1","L1_D0CPED")
105  EndIf
106
107  aRet := STFFireEvent(  ProcName(0) ,;      // Nome do processo
108                        "STPrinterStatus",; // Nome do evento
109                        aDados )
110
111  //Atualiza o numero de série
112  If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 1 .AND. !Empty(aDados[2])
113      //Verifica o numero de série
114      STBSerialt( cDoc, aDados[2] )
115  EndIf
116
117  // Doação para o Instituto Arredondar
118  If lArredondar
119
120      If SL1->(FieldPos( "L1_VLRARR" )) > 0
121          nArredondar := STBGetInsArr()
122          If nArredondar > 0
123              If ExistFunc("STBIANotFiscal")
124                  If !(lEmitNfce .OR. lSAT)      // Somente ECF. NFC-e ou SAT é após esta etapa
125                      cSerie      := STDGPBasket("SL1","L1_SERIE")
126                      cDoc        := STDGPBasket("SL1","L1_DOC")
127                      cACli       := STDGPBasket("SL1","L1_CLIENTE")
128                      cAlojaCli   := STDGPBasket("SL1","L1_LOJA")
129                      cACPFcli    := STDGPBasket("SL1","L1_CGCCLI")
130                      cAFormPg    := STDGPBasket("SL1","L1_FORMPG")
131
132                      //Imprime cupom de doação ao Instituto Arredondar
133                      STBIANotFiscal( nArredondar , cSerie      , cDoc        , cACli,;
134                                     cAlojaCli   , cACPFcli    , cAFormPg    )
135                  EndIf
136              EndIf
137          EndIf
138      EndIf
139  Endif
140
141  // Grava tabela MGX para controle de limite de Sangria
142  If lLimSang .AND. AliasIndic("MGX")
143      STDGrvMGX()
144  EndIf
145  // Verifica se haverá atualizacao de saldo de ShopCard
146  If STDGUpdShopCardFundsResult()
147      STDIncShopCardFunds()
148      STDRUpdShopCardFundsResult()
149  EndIf
150
151  // T0 D0: Verificar se o cupom foi encerrado e alterar o status do L1_SITUA.
152
153  // T0 D0: Informar a retaguarda o status atual do check-out.
154
155  // T0 D0: Exibir mensagem de finalizacao de cupom

```

```

156
157 If GetVersao(.F.) >= "12" .AND. !(lEmitNfce .OR. lSAT) //Impressão de Vale-Troca quando utilizada IMPRESSORA FISCAL
158     STBVTcupom(SL1->L1_DOC,.F.)
159 EndIf
160
161 //Versao Mobile Demonstrativa nao emite Cupom
162 If lMobile
163
164     //Se salva venda como orcamento
165     If lSaveOrc
166         STDFinishSale()
167     EndIf
168
169 EndIf
170
171 //Executa PE para saber se gera NFC-e/NF-e/SAT ou utiliza parametro
172 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "MV_LJLBNT: " + Str(nDocSai,1) )
173
174 //-- Chama a funcao LjNFFimVd, pois dentro dela executa o PE "LJ7087" e valida se pode gerar NF de acordo com as regras do Varejo
175 if lMVFISNOTA .and. !lCliPadrao
176     lEmitNfce := LjNFFimVd() //Resgata a opção selecionada na tela de Seleção de Documento de saída no inicio da venda.
177 endif
178
179 //Itens da Venda
180 For nX := 1 To STDPBLength("SL2")
181     aAdd(aItensFid, { nX ,;
182                     STDGPBasket("SL2", "L2_DESCRI" ,nX),;
183                     STDGPBasket("SL2", "L2_PRODUTO" ,nX),;
184                     STDGPBasket("SL2", "L2_QUANT" ,nX),;
185                     STDGPBasket("SL2", "L2_VLRITEM" ,nX) + STDGPBasket("SL2", "L2_VALDESC", nX),;
186                     STDGPBasket("SL2", "L2_VLRITEM" ,nX) })
187 Next nX
188
189 //Formas de Pagamento da Venda
190 aAreaSL4 := SL4->(GetArea())
191 DbSelectArea("SL4")
192 SL4->(DbSetOrder(1))
193 If DbSeek(xFilial("SL4") + SL1->L1_NUM)
194     While SL4->L4_FILIAL + SL4->L4_NUM == xFilial("SL4") + SL1->L1_NUM
195         aDescPgto := FWGetSX5("24", SL4->L4_FORMA)
196         aAdd(aFormPgto, { "",;
197                         aDescPgto[1][4],;
198                         SL1->L1_VLRTOT })
199     SL4->(DbSkip())
200 EndDo
201 EndIf
202 RestArea(aAreaSL4)
203
204 If lEmitNfce .and. STBExistItemFiscal(.F.) //usa NF-e e tem item fiscal
205     cSerie := PadR(If(SubStr(cMVLOJANF,1,1)=="&",&(SubStr(cMVLOJANF,2,Len(cMVLOJANF))),cMVLOJANF), len(SL1->L1_SERIE))
206     lLjDNota := LjxDNota( cSerie, 3, .F., 1, @aNotas,,,,,,,,,"SPED" ) // DOC/SERIE
207

```

```

208     if lLjDNota .and. Len(aNotas) >= 1 .AND. Len(aNotas[1]) >= 2
209
210         cDoc := aNotas[1][2]
211
212         STDSPBasket( "SL1", "L1_HORA" , Left(Time(), TamSX3("L1_HORA")[1]) )
213         STDSPBasket( "SL1", "L1_SERIE" , cSerie )
214         STDSPBasket( "SL1", "L1_DOC" , cDoc )
215         STDSPBasket( "SL1", "L1_NUMCFIS", cDoc ) // Numero cupom fiscal
216         STDSPBasket( "SL1", "L1_SITUA" , "55" ) //Atribuimos um L1_SITUA extra (55) antes da transmissão
217         STDSPBasket( "SL1", "L1_IMPNF" , .T. )
218         STDSPBasket( "SL1", "L1_IMPRIME", "2S" ) //Necessário pois se trata de uma venda fiscal (CUPOM/NFC-e)
219
220
221     nTotItSL2 := STDPBLength("SL2")
222     For nItem := 1 to nTotItSL2
223
224         If STDGPBasket("SL2","L2_FISCAL", nItem) .OR.;
225             (ExistFunc("STBIsGarEst") .AND. STBIsGarEst(STDGPBasket("SL2","L2_PRODUTO", nItem)))
226
227             STDSPBasket( "SL2", "L2_DOC", cDoc, nItem )
228             //persiste todos os dados da venda (SL1 e item do SL2)
229             STDSaveSale(nItem)
230         EndIf
231
232     Next nItem
233
234     STFCleanInterfaceMessage()
235     STFMessage(ProcName(), "ALERT", STR0006 + STDGPBasket("SL1","L1_NUM") + STR0007 + cDoc+"/"+cSerie) //#"Aguarde, Preparando Orc: " / #" Ref. NFe: "
236     STFSHOWMessage( ProcName() )
237
238     //Chama Mini-Grava-Batch para gerar SF2, SD2, SF3, SFT e CD2 para o NFSEFAZ
239     if STBGrvBatch(/*cAlias*/, /*lMultFil*/, /*cKey*/,.T. /*Executado pelo Totvs PDV?*/)
240         //Transmitir para a SEFAZ
241         LjAutoNFe(cDoc,;
242             cSerie,;
243             oCliModel:GetValue("SA1MASTER","A1_COD"),;
244             oCliModel:GetValue("SA1MASTER","A1_LOJA"))
245
246
247         aAreaSL1:= STWVldRgnNfe()
248         //Como a rotina nao retorna se deu problema, valida-se pelo L1_RETSSFZ
249         aRetTSS := StrToKarr(SL1->L1_RETSSFZ,"|")
250         if len(aRetTSS) > 2
251             cRetTSS := aRetTSS[02]
252             cMsgTSS := aRetTSS[03]
253         else
254             cRetTSS := aRetTSS[01]
255             cMsgTSS := iif( len(aRetTSS) > 1, aRetTSS[02], "" )
256         endif
257
258         If lLjGetModNf .AND. LjGetModNf() == 7
259             nRetNfce := 1

```

```

260         cKeyNFCe := SL1->L1_KEYNFCe
261     Else
262         //-- Quando Autorizado conclui a venda com sucesso
263         cRetTrans := LjRetStatusSEFAZ(cRetTSS, cMsgTSS)
264         If ValType(cRetTrans) == "C" .and. cRetTrans $ "A"
265             nRetNfce := 1
266             cKeyNFCe := SL1->L1_KEYNFCe
267         EndIf
268     EndIf
269
270     STFCleanInterfaceMessage()
271
272     If Len(aAreaSL1)
273         lRetArea := .T.
274     Endif
275 endif
276
277 endif
278
279 If nRetNfce <> 1 .And. lPDVOnline .And. !LjGetCPDV()[1]
280     MsgAlert("", " IMPORTANTE :"+ CRLF+ CRLF+ CRLF;
281         + "A Nota : " + STDGPBasket("SL1","L1_DOC") + CRLF;
282         + "Serie : " + STDGPBasket("SL1","L1_SERIE") + CRLF + CRLF;
283         + "Não foi transmitida."+ CRLF + CRLF;
284         + "Será necessario realizar a retransmissão dessa nota !"+ CRLF + CRLF;
285         + "Acessar a opção: " + CRLF + CRLF;
286         + "Menu F2 > Retransmissão NF-e.")
287     nRetNfce := 1
288     cKeyNFCe := SL1->L1_KEYNFCe
289 EndIf
290
291 If nRetNfce == 1 .AND. nArredondar > 0 .AND. ExistFunc("STBIANotFiscal") //Instituto Arredondar
292
293     cSerie      := STDGPBasket("SL1","L1_SERIE")
294     cDoc        := STDGPBasket("SL1","L1_DOC")
295     cACli       := STDGPBasket("SL1","L1_CLIENTE")
296     cALojaCli   := STDGPBasket("SL1","L1_LOJA")
297     cACPFCli    := STDGPBasket("SL1","L1_CGCCLI")
298     cAFormPg    := STDGPBasket("SL1","L1_FORMPG")
299
300     //Imprime cupom de doação ao Instituto Arredondar - NFC-e
301     STBIANotFiscal( nArredondar , cSerie , cDoc ,cACli,;
302         cALojaCli , cACPFCli , cAFormPg )
303 EndIf
304
305 STBVTcupom() //Impressão de Vale-Troca quando utilizada IMPRESSORA NÃO-FISCAL
306
307 ---
308 // Situações nRetNfce:
309 // 1 -> NFC-e/NFe processada com sucesso
310 // 0 -> NFC-e/NFe enviada para o TSS sem Rejeicao,
311 // mas não houve sucesso na comunicação TSS->SEFAZ

```

```

312      //-1      -> NFC-e/NFe rejeitada
313      //--
314
315      //reseta o objeto referente a NFC-e
316      LjNFCeFree()
317
318      If nRetNfce == 1
319          //a chave da NFC-e nao deve ser gravada em caso de rejeição, já que a nota será inutilizada
320          STDSPBasket("SL1", "L1_KEYNFCE", cKeyNfce)
321
322          If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
323
324              // -- Finaliza BonusHub
325              oRaas := STBGetRaas()
326
327              If Valtype(oRaas) == "0" .And. oRaas:ServiceIsActive("TFC")
328                  oFidelityC := oRaas:GetFidelityCore()
329
330                  If oFidelityC:ChoseToUse()
331                      // -- Desativa Botões na tela
332                      STIBtnDeActivate()
333                      oFidelityC:Finalization(STFGetStat("CODIGO"),;
334                          POSICIONE( "SA3", 1, xfilial("SA3") + STDGPBasket('SL1','L1_VEND'), "A3_NOME" ),;
335                          cKeyNfce;;
336                          STDPBLength("SL2"),;
337                          STDGPBasket('SL1','L1_VLRLIQ'),;
338                          aItensFid;;
339                          aFormPgto)
340
341                      // -- Ativa Botões na tela
342                      STIBtnActivate()
343
344                      oFidelityC:Clean()
345                      STDSPBasket( "SL1" , "L1_FIDCORE",.T.)
346                  EndIf
347              Endif
348          Endif
349
350          EndIf
351
352          STDFinishSale(lEmiteNFe, lRetArea)
353          lRet := .T.
354      Else
355          /*
356          Pre-Requisitos para a inutilizacao atraves da SLX
357          */
358          lPendSale := STBChkInut()
359
360          If lPendSale
361              // realiza a inutilizacao da NFC-e atraves da SLX
362              LjGrvLog( NIL, "Antes da chamada da função STWInuNFCE - Inutilização atraves da SLX" )
363              lPendSale := STWInuNFCE(STDGPBasket("SL1","L1_DOC")      , STDGPBasket("SL1","L1_SERIE")      , STDGPBasket("SL1","L1_PDV")      , ;

```

```

364          STDGPBasket("SL1","L1_OPERADO"), STDGPBasket("SL1","L1_ESTACA0") , cMsgErro      ,;
365          STDGPBasket("SL1","L1_NUM")      )
366      LjGrvLog( NIL, "Depois da chamada da função STWInuNFCE",lPendSale)
367
368      // resetamos e apagamos os campos referentes aos Pagamentos que ja foram realizados(SL1/SL4)
369      If lPendSale .AND. lDelPg
370          lPendSale := STWDelPay( STDGPBasket("SL1","L1_NUM") )
371      EndIf
372  EndIf
373
374  // Faz a exclusao/inutilizacao pelo metodo antigo
375  If !lPendSale
376      STWCancelSale(.T.,,cDoc, "L1_NUM",,)
377  EndIf
378
379
380  If !IsBlind()
381      STFMessage("STWFinisale3", "POPUP", STR0003)
382      STFShowMessage("STWFinisale3")
383  Else
384      Conout(STR0003) // "Venda não Efetuada devido falha de comunicação"
385  EndIf
386
387      LjGrvLog( NIL,STR0003)
388  EndIf
389
390  ElseIf lSAT .AND. STBExistItemFiscal(.F.) //usa SAT e tem item fiscal
391
392      STDSPBasket( "SL1", "L1_SITUA", "10" ) //Para possivel restauração de venda
393
394      cXML := LjSATXml() // Gera XML
395
396      aRet := LJSATComando({"12","EnviarDadosVenda",LJSATnSessao(),cPass,cXML})
397
398      If Len(aRet) > 2 .And. Val(aRet[2]) == 6000 //retorno de sucesso
399          If FindFunction("LJSATRetDoc")
400              aSATDoc := LJSATRetDoc(Decode64(aRet[5]),aRet) //retorna o doc e serie gerado no SAT
401
402              cDoc := cDocSat := aSATDoc[1]
403              cSerieSat      := aSATDoc[2]
404
405              STDSPBasket( "SL1", "L1_DOC"      , cDoc )
406              STDSPBasket( "SL1", "L1_ESPECIE"  , "SATCE" )
407              STDSPBasket( "SL1", "L1_SERSAT"   , cSerieSat )
408              STDSPBasket( "SL1", "L1_KEYNFCE"  , SubStr(aRet[7],4,Len(aRet[7])) )
409              STDSPBasket( "SL1", "L1_HORA"     , Left(Time(),TamSX3("L1_HORA")[1]) )
410              STDSPBasket( "SL1", "L1_SITUA"    , "T3" ) // Pois a venda já foi transmitida para sefaz e retornou a numeração para o sistema.
411              STDSaveSale()
412
413              LjGrvLog( SL1->L1_NUM, "SAT - Numeração do Doc e Serie do SAT", aSATDoc )
414
415          EndIf

```

```

416
417   STFMessagem("STIPayment","STOP", cSiglaSat + " - Transmitido com sucesso" ) //"SAT - Transmitido com sucesso"
418   STFSHOWMESSAGE("STIPayment")
419
420   LojSATImprimir(Decode64(aRet[5]), cXML, SL1->L1_SERIE, @cDocSat, @cSerieSat, ,aSATDoc) //impressao do cupom
421
422   cDoc := cDocSat
423
424   STDSPBasket( "SL1", "L1_DOC", cDoc )
425   STDSPBasket( "SL1", "L1_NUMCFIS", cDoc )    // Numero cupom fiscal
426   STDSPBasket( "SL1", "L1_SERSAT", cSerieSat )
427   STDSPBasket( "SL1", "L1_ESPECIE", "SATCE" )
428   STDSPBasket( "SL1", "L1_KEYNFCE", SubStr(aRet[7],4,Len(aRet[7])) )
429   STDSPBasket( "SL1", "L1_SITUA" , "58" )    //Atribuimos um L1_SITUA extra (58) SAT
430
431   nTotItSL2 := STDPBLength("SL2")
432
433   For nItem := 1 to nTotItSL2
434     If STDGPBasket("SL2","L2_FISCAL", nItem) .OR.;
435       (ExistFunc("STBIsGarEst") .AND. STBIsGarEst(STDGPBasket("SL2","L2_PRODUTO", nItem)))
436       STDSPBasket( "SL2", "L2_DOC", cDoc, nItem )
437       STDSaveSale(nItem)
438     EndIf
439   Next
440
441   If nArredondar > 0 .AND. ExistFunc("STBIANotFiscal")
442     cSerie      := STDGPBasket("SL1","L1_SERIE")
443     cDoc        := STDGPBasket("SL1","L1_DOC")
444     cACli       := STDGPBasket("SL1","L1_CLIENTE")
445     cLojaCli    := STDGPBasket("SL1","L1_LOJA")
446     cACPFcli    := STDGPBasket("SL1","L1_CGCCLI")
447     cAFormPg    := STDGPBasket("SL1","L1_FORMPG")
448     //Imprime cupom de doação ao Instituto Arredondar - SAT
449     STBIANotFiscal(nArredondar,cSerie,cDoc,cACli,;
450                   cAlojaCli,cACPFcli,cAFormPg)
451   EndIf
452
453   If GetVersao(.F.) >= "12" //Impressão de Vale-Troca quando utilizada IMPRESSORA NÃO-FISCAL
454     STBVTcupom(SL1->L1_DOC,.T.)
455   EndIf
456
457   If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
458
459     // -- Finaliza BonusHub
460     oRaas := STBGetRaas()
461
462     If ValType(oRaas) == "O" .And. oRaas:ServiceIsActive("TFC")
463       oFidelityC := oRaas:GetFidelityCore()
464
465       If oFidelityC:ChoseToUse()
466         // -- Desativa Botões na tela
467         STIBtnDeActivate()

```

```

468         oFidelityC:Finalization(STFGetStat("CODIGO"),;
469                                POSICIONE( "SA3", 1, xfilial("SA3") + STDGPBasket('SL1','L1_VEND'), "A3_NOME" ),;
470                                cSerie + cDoc;;
471                                STDPBLength("SL2"),;
472                                STDGPBasket('SL1','L1_VLRLIQ'),;
473                                aItensFid;;
474                                aFormPgto)
475
476         // -- Ativa Botões na tela
477         STIBtnActivate()
478
479         oFidelityC:Clean()
480         STDSPBasket( "SL1" , "L1_FIDCORE",.T.)
481     EndIf
482
483 Endif
484
485 EndIf
486
487 STDFinishSale()
488
489 lRet := .T.
490 Else
491     STWCancelSale(.T.,,cDoc, "L1_NUM",,) //cancela venda
492
493     If Len(aRet) > 3 .And. ExistFunc("LjGrvLogSAT")
494         //Log SAT
495         LjGrvLogSAT(;
496                 /*serie*/,;
497                 /*numero da venda*/,;
498                 /*chave da venda*/,;
499                 cXML;;
500                 /*xml retorno*/,;
501                 "VENDA",;
502                 "ERRO",;
503                 IIf( Len(aRet) > 1 , aRet[2], "aRetSAT[2] = NIL") + "-" + IIf( Len(aRet) > 3 , cValToChar(DecodeUTF8(aRet[4])), "aRetSAT[4] = NIL");
504             )
505
506         /*
507             aRet[2] - Codigo
508             aRet[4] - Descrição
509         */
510         //Não foi colocado em STFMESSAGE porque a mensagem pode ser grande e não será exibida por completo
511         STFMessage(ProcName(),"STOP",STR0005 + IIf( Len(aRet) > 1 , aRet[2], "aRetSAT[2] = NIL") + "-" + IIf( Len(aRet) > 3 , cValToChar(DecodeUTF8(aRet[4])),
"aRetSAT[4] = NIL")) //#"Venda não realizada: " // Código de erro retornado pelo SAT
512         STFShowMessage(ProcName())
513
514     EndIf
515
516 EndIf
517
518 ElseIf lEmitNFCe .AND. STBExistItemFiscal(.F.) .AND. !lSaveOrc

```

```

519
520 //
521 // obtem a serie e numeracao da NFC-e
522 cSerie := STFGetStation("SERIE")
523
524 LjxDNota( cSerie, 3, .F., 1, @aNotas,,,,,,,,,"NFCE" ) // DOC/SERIE
525 If Len(aNotas) >= 1 .AND. Len(aNotas[1]) >= 2
526     cDoc := aNotas[1][2]
527 EndIf
528
529 STDSPBasket( "SL1", "L1_HORA" , Left(Time(), TamSX3("L1_HORA")[1]) )
530 STDSPBasket( "SL1", "L1_SERIE" , cSerie )
531 STDSPBasket( "SL1", "L1_DOC" , cDoc )
532 STDSPBasket( "SL1", "L1_NUMCFIS", cDoc ) // Numero cupom fiscal
533 STDSPBasket( "SL1", "L1_SITUA", "65" ) //Atribuímos um L1_SITUA extra (65) antes da transmissão
534 STDSPBasket( "SL1", "L1_IMPRIME", "1S" ) //Necessário pois se trata de uma venda fiscal (CUPOM/NFC-e)
535
536 nTotItSL2 := STDPBLength("SL2")
537 For nItem := 1 to nTotItSL2
538     If STDGPBasket("SL2","L2_FISCAL", nItem) .OR.;
539     (ExistFunc("STBIsGarEst") .AND. STBIsGarEst(STDGPBasket("SL2","L2_PRODUTO", nItem)))
540     STDSPBasket( "SL2", "L2_DOC", cDoc, nItem )
541     //persiste todos os dados da venda (SL1 e item do SL2)
542     STDSaveSale(nItem)
543 EndIf
544 Next nItem
545
546 STFCleanInterfaceMessage()
547
548 LjMsgRun( STR0002+ " " + SL1->L1_NUM + " " + STR0004 + " " + cDoc,,;
549 {|| nRetNfce := LjNFCeGera(SL1->L1_FILIAL,SL1->L1_NUM, @cKeyNfce,,lPrintNFCE, @cMsgErro,,,,@lSemRetXml, @lNaoGrvSLX)} ) // "Aguarde... Processando NFC-e Orcamento:
" " - Doc.: "
550
551 If nRetNfce == 1 .AND. nArredondar > 0 .AND. ExistFunc("STBIANotFiscal") //Instituto Arredondar
552     cSerie := STDGPBasket("SL1","L1_SERIE")
553     cDoc := STDGPBasket("SL1","L1_DOC")
554     cACli := STDGPBasket("SL1","L1_CLIENTE")
555     cALojaCli := STDGPBasket("SL1","L1_LOJA")
556     cACPFCli := STDGPBasket("SL1","L1_CGCCLI")
557     cAFormPg := STDGPBasket("SL1","L1_FORMPG")
558
559     //Imprime cupom de doação ao Instituto Arredondar - NFC-e
560     STBIANotFiscal( nArredondar , cSerie , cDoc ,cACli,;
561     cALojaCli , cACPFCli , cAFormPg )
562 EndIf
563
564 If nRetNfce == 1 .AND. GetVersao(.F.) >= "12" //Impressão de Vale-Troca quando utilizada IMPRESSORA NÃO-FISCAL
565     STBVTcupom(SL1->L1_DOC,.T.)
566 EndIf
567
568 //
569 //---
```

```

570 // Situações nRetNfce:
571 // 1 -> NFC-e/NFe processada com sucesso
572 // 0 -> NFC-e/NFe enviada para o TSS sem Rejeicao,
573 // mas não houve sucesso na comunicação TSS->SEFAZ
574 //-1 -> NFC-e/NFe rejeitada
575 //---
576
577 //reseta o objeto referente a NFC-e
578 LjNFCeFree()
579
580 If nRetNfce == 1
581 //a chave da NFC-e nao deve ser gravada em caso de rejeição, já que a nota será inutilizada
582 STDSPBasket("SL1", "L1_KEYNFC", cKeyNfce)
583
584 If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
585
586 // -- Finaliza BonusHub
587 oRaas := STBGetRaas()
588
589 If Valtype(oRaas) == "0" .And. oRaas:ServiceIsActive("TFC")
590 oFidelityC := oRaas:GetFidelityCore()
591
592 If oFidelityC:ChoseToUse()
593 // -- Desativa Botões na tela
594 STIBtnDeActivate()
595 oFidelityC:Finalization(STFGetStat("CODIGO"),;
596 POSICIONE( "SA3", 1, xfilial("SA3") + STDGPBasket('SL1','L1_VEND'), "A3_NOME" ),;
597 cKeyNfce,;
598 STDPBLength("SL2"),;
599 STDGPBasket('SL1','L1_VLRLIQ'),;
600 aItensFid,;
601 aFormPgto)
602
603 // -- Ativa Botões na tela
604 STIBtnActivate()
605
606 oFidelityC:Clean()
607 STDSPBasket( "SL1" , "L1_FIDCORE",.T.)
608 EndIf
609
610 Endif
611
612 EndIf
613
614 STDFinishSale()
615 lRet := .T.
616 lEnviaContg := .F.
617 Else
618 /*
619 Pre-Requisitos para a inutilizacao atraves da SLX
620 */
621

```

```

622      If ExistFunc("STBChkInut") //STBCANCELSALE.PRW
623
624          //verifica se possui todos os requisitos
625
626          lPendSale := STBChkInut()
627
628          If lPendSale .AND. !lNaoGrvSLX
629              // realiza a inutilizacao da NFC-e atraves da SLX
630              LjGrvLog( NIL, "Antes da chamada da função STWInuNFCE - Inutilização através da SLX" )
631              lPendSale := STWInuNFCE(STDGPBasket("SL1","L1_DOC")      , STDGPBasket("SL1","L1_SERIE")      , STDGPBasket("SL1","L1_PDV")      , ;
632                          STDGPBasket("SL1","L1_OPERADO") , STDGPBasket("SL1","L1_ESTACA0")      , cMsgErro      , ;
633                          STDGPBasket("SL1","L1_NUM")      )
634              LjGrvLog( NIL, "Depois da chamada da função STWInuNFCE",lPendSale)
635
636              // resetamos e apagamos os campos referentes aos Pagamentos que ja foram realizados(SL1/SL4)
637              If lPendSale .AND. lDelPg
638                  lPendSale := STWDelPay( STDGPBasket("SL1","L1_NUM") )
639              EndIf
640          EndIf
641      EndIf
642
643      // Faz a exclusao/inutilizacao pelo metodo antigo
644      If !lPendSale
645          STWCancelSale(.T.,,cDoc, "L1_NUM",,)
646      EndIf
647
648
649      If !IsBlind()
650          If lSemRetXml .OR. lNaoGrvSLX
651              STFMessage("STWFinisale3", "POPUP", STR0008)//Finalize a Venda novamente através do botão Finalizar pagamento CTRL+P!
652              STFSHOWMessage("STWFinisale3")
653          Else
654              STFMessage("STWFinisale3", "POPUP", STR0003)//Venda não Efetuada devido falha de comunicação com SEFAZ
655              STFSHOWMessage("STWFinisale3")
656          Endif
657      Else
658          Conout(STR0003) //Venda não Efetuada devido falha de comunicação com SEFAZ
659      EndIf
660
661      LjGrvLog( NIL,STR0003)
662
663      EndIf
664      //Se nao for NF-e, NFC-e ou SAT
665      Else
666
667          If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
668
669              // -- Finaliza BonusHub
670              oRaas := STBGetRaas()
671
672              If Valtype(oRaas) == "0" .And. oRaas:ServiceIsActive("TFC")
673                  oFidelityC := oRaas:GetFidelityCore()

```

```

674
675         If oFidelityC:ChoseToUse()
676             // -- Desativa Botões na tela
677             STIBtnDeActivate()
678             oFidelityC:Finalization(STFGetStat("CODIGO"), POSICIONE( "SA3", 1, xfilial("SA3") + STDGPBasket('SL1','L1_VEND'), "A3_NOME" ),,;
679                                     STDGPBasket("SL1","L1_SERIE") + STDGPBasket("SL1","L1_DOC"),STDPBLength("SL2"),STDGPBasket('SL1','L1_VLRLIQ'))
680             // -- Ativa Botões na tela
681             STIBtnActivate()
682
683             oFidelityC:Clean()
684             STDSPBasket( "SL1" , "L1_FIDCORE",.T.)
685         EndIf
686
687     Endif
688
689 EndIf
690
691     STDFinishSale()
692     lRet := .T.
693 EndIf
694
695 If !lPendSale .AND. nTotalNCCs > 0
696     STWBxNCC()
697 EndIf
698
699 If !lPendSale .AND. lRet
700     STBBaixaVP()
701 Endif
702
703
704 EndIf
705
706 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Venda ficara pendente?", lPendSale )
707 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Fim - Workflow de finalizacao de venda.", lRet )
708
709 If !lPendSale
710     //Gravo o doc e a série antes do STFRestart(). Útil para o Ponto de Entrada STFinishSale() em STBPayment.prw.
711     STWSetDocSerie(STDGPBasket( "SL1", "L1_DOC" ),STDGPBasket( "SL1", "L1_SERIE" ))
712     STFRestart()
713 EndIf
714
715
716 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Proxima venda será: " + STDGPBasket('SL1','L1_NUM') )
717
718 Return lRet
719
720
721 //-----
722 /*{Protheus.doc} STWFinishValid
723 Função criada para validar se o fonte esta atualizado por conta da troca
724 de impressão do comprovante TEF
725 @param

```

```

726 @return Nil
727 @obs
728 @sample
729 */
730 //-----
731 Function STWFinishValid
732 Return
733
734
735 //-----
736 /*{Protheus.doc} STWSetDocSerie
737 Função criada para armazenar o último número do cupom fiscal antes do STFRestart()
738
739 @param cDoc,cSerie
740 @return Nil
741 @obs
742 @sample
743 */
744 //-----
745 Function STWSetDocSerie(cDoc,cSerie)
746
747 cStDoc := cDoc
748 cStSerie := cSerie
749
750 Return nil
751
752
753 //-----
754 /*{Protheus.doc} STWGetDocSerie
755 Função criada para ler o último número do cupom fiscal e Série antes do STFRestart()
756
757 @param Nil
758 @return {cStDoc,cStSerie}
759 @obs
760 @sample
761 */
762 //-----
763 Function STWGetDocSerie()
764
765 Return( {cStDoc,cStSerie} )
766
767 /*{Protheus.doc} STWvldRgNfe
768 Rotina criada para ajustar o ponteiro da SL1 no registro do Item Retira
769 quando foi emitido NF-e na situação seguinte:
770 -Importação de Orçamento com reserva (retira posterior, entrega);
771 -Venda de item durante a importação de orçamentos acima;
772 -Emissão de NF-e
773 Nessa situação, o ponteiro da SL1 fica no registro do Orçamento pai,
774 porém a NF-e foi emitida pela SL1 referente aos itens Retira ou item lançado no PDV.
775 No trecho da rotina STWFinishSale(), quando busca o L1_RETSFZ e L1_KEYNFCE ocorre erro
776 justamente pq no SL1 do registro Pai não tem esses valores.
777 Criada essa rotina para mudar o ponteiro para o registro de SL1 que foi gerado pelos itens retira.

```

```

778
779     @type Static Function
780     @param param_name, param_type, param_descr
781     @return return_var, return_type, return_description
782 */
783 Static Function STWVldRgNfe()
784 Local lRet      := .F. // Seta .T. caso for mudado o ponteiro, nesse caso precisa dar RestArea() depois
785 Local aArea     := {}
786 Local cSl1Num   := SL1->L1_NUM
787
788 If lPDVOnline .AND. STBIsImpOrc()
789     aArea := SL1->(GetArea())
790     DbSelectArea("SL1")
791     SL1->(DbSetOrder(14))/L1_FILIAL+L1_ORCRES
792     If SL1->( DbSeek(xFilial("SL1") + cSl1Num))
793         While !lRet .AND. !Eof() .AND. (xFilial()+cSl1Num) == (SL1->L1_FILIAL+SL1->L1_ORCRES)
794             lRet:= !Empty(SL1->L1_RETSFZ) .AND. !Empty(SL1->L1_KEYNFCE) .AND. SL1->L1_SITUA == "OK"
795             If !lRet
796                 SL1->(dbSkip())
797             Endif
798         EndDo
799     Endif
800
801     If !lRet
802         RestArea(aArea)
803         aArea:={}
804     Endif
805
806 Endif
807
808 Return aArea

```

Componentes/FlowControl/STBFlowControl.prw

59 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4
5 //-----
6 /*{Protheus.doc} STBAddFlow
7 Inclui registro para controle de fluxo
8
9 @return Nil
10 @obs
11 @sample
12 */
13 //-----
14 Function STBAddFlow( cFlowAction,aParams )
15
16 Local cStation := PADR(STFGetStation("CODIGO"),TamSX3("LI_ESTACA0")[1]) // Estacao atual, utilizado na gravacao da SLI de controle de fluxo
17 Local cFunction := ProcName(1) // Rotina que sera controlada
18 Local cParams := ""
19 Local nX := 0
20 Local cAuxLIMsg := ""
21 Local nCountRep := 1 //Qtde de vezes que reprocessou o fluxo(FlowAction)
22 Local aFlowAction := {}
23
24 Default cFlowAction := ""
25 Default aParams := []
26
27 LjGrvLog( "Controle de Fluxo", "Entrou controle de fluxo cFlowAction:",cFlowAction)
28
29 //Verifica se ja tentou reprocessar mesma etapa do fluxo, limita qtde de tentativas de reprocessar mesmo fluxo para evitar looping infinito e travamento do PDV
30 aFlowAction := StrTokArr( cFlowAction, "|" ) //IdAction|Qtde
31 If Len(aFlowAction) > 1
32     nCountRep := Val(aFlowAction[2]) + 1 //incrementa contador
33     cFlowAction := aFlowAction[1] //ajusta nome do fluxo removendo informacao de Qtde
34 EndIf
35
36 //Monta parametros para execucao da Function
37 For nX := 1 To Len(aParams)
38     cParams += IIF( ValType(aParams[nX]) == "C",""+aParams[nX]+"" , cValToChar(aParams[nX]) ) + IIF( nX < Len(aParams),"|","")
39 Next nX
40
41 //Quando qtde de vezes que tentou reprocessar o mesmo fluxo atingir 3(se necessario, criar param para definir a qtde maxima), gera log e nao tenta novamente, para evitar
    looping infinito
42 If nCountRep >= 6
43     LjGrvLog( "Controle de Fluxo", "Abortou reprocessamento de Fluxo. Motivo: Atingiu a quantidade de tentativas(5)",cFlowAction)
44     cFlowAction := ""
45 EndIf
46
47 If !Empty(cFlowAction)
48     cAuxLIMsg := cFunction + "|" + cFlowAction + "|" + Str(nCountRep) + "|" + cParams
49 Else
50     cAuxLIMsg := "OK" //Rotina utilizada para popular SLI(STFSLICreate) não considera LI_MSG vazio, por esse motivo é adicionado texto padrao "OK"
51 EndIf

```

```
52
53 LjGrvLog( "Controle de Fluxo", "Sera adicionado comando tabela SLI(Tipo: FLW) - LI_MSG:",cAuxLIMsg)
54
55 If !STFSLICreate( cStation, "FLW", cAuxLIMsg , "SOBREPOE" )
56     LjGrvLog( "Controle de Fluxo", "Nao foi possivel incluir controle de fluxo(SLI)",cFlowAction)
57 EndIf
58
59 Return Nil
```

Componentes/FlowControl/STDFlowControl.prw

26 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4
5 //-----
6 /*{Protheus.doc} STDRetFlow
7 Inclui registro para controle de fluxo
8
9 @return Nil
10 @obs
11 @sample
12 */
13 //-----
14 Function STDRetFlow( )
15
16 Local cStation := PADR(STFGetStation("CODIGO"),TamSX3("LI_ESTACA0")[1]) // Estacao atual, utilizado na gravacao da SLI de controle de fluxo
17 Local aRet      := {}
18
19 DbSelectArea("SLI")
20 SLI->(DbSetOrder( 1 )) //LI_FILIAL+LI_ESTACA0+LI_TIPO
21 If SLI->(DbSeek( xFilial("SLI") + cStation + "FLW" )) .AND. !Empty(SLI->LI_MSG) .AND. (SLI->LI_MSG <> "OK")
22     LjGrvLog( "Controle de Fluxo", "Localizou comando (LI_TIPO = FLW) para Reprocessar na SLI(LI_MSG:)",SLI->LI_MSG)
23     aRet := StrTokArr( SLI->LI_MSG, "|" ) //Function|IdAction|Params
24 EndIf
25
26 Return aRet

```

Componentes/FlowControl/STWFlowControl.prw

113 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STWFlowControl
6 Verifica se existe acao pendente para encerrar o fluxo de rotina, quando ocorre a saida inesperada do Sistema (Queda de energia)
7 Obs.: Necessario para controle do processamento de algumas rotinas. Motivo: nao existe controle de transacao para ambiente Codebase e nas interacoes com perifericos
8 @return Nil
9 @obs
10 @sample
11 */
12 //-----
13 Function STWFlowControl( )
14
15 Local aAuxLIMsg := {}
16 Local cFunction := ""
17 Local cIdAction := ""
18 Local nX      := 0
19 Local aGetFunc := {}
20 Local nPosFunc := 0
21
22
23 aAuxLIMsg := STDRetFlow() //Retorna fluxo pendente de execucao
24
25 If Len(aAuxLIMsg) >= 3
26   cFunction := aAuxLIMsg[1]
27   cIdAction := aAuxLIMsg[2] + "|" + aAuxLIMsg[3]
28
29   //Verifica qual posicao do parametro de controle de fluxo
30   If FindFunction('GetFuncPrm')
31     aGetFunc := GetFuncPrm(cFunction)
32     nPosFunc := aScan(aGetFunc,{|x| Upper(AllTrim(x)) == "CFLOWACTION"})
33   EndIf
34
35   cFunction += "("
36   For nX := 4 To Len(aAuxLIMsg)
37     If (nX-2 <> nPosFunc) .AND. (nX-2 <> nPosFunc + 1)
38       cFunction += IIF( ValType(aAuxLIMsg[nX]) == "C",aAuxLIMsg[nX], cValToChar(aAuxLIMsg[nX]) )
39     Else
40       cFunction += "'" + cIdAction + "'"
41     EndIf
42
43     If nX < Len(aAuxLIMsg)
44       cFunction += ","
45     EndIf
46
47   Next nX
48   cFunction += ")"
49
50   LjMsgRun("Em 5 segundos será reprocessado a rotina: ["+aAuxLIMsg[1]+"] etapa: [" + aAuxLIMsg[2] + "] tentativa: "+AllTrim(aAuxLIMsg[3])+ " de 5","Reprocessamento de Rotina",{|| Sleep(5000)})
51

```

```

52 //MacroExecuta Rotina para continuar o fluxo interrompido
53 &(cFunction)
54 EndIf
55
56
57 Return Nil
58
59
60 //-----
61 /*/{Protheus.doc} STWFlwCheck
62 Verifica se existe acao pendente para encerrar o fluxo de rotina, quando ocorre a saida inesperada do Sistema (Queda de energia)
63
64 @return lRet          Retorna se a venda está em andamento ou não
65 @obs
66 @sample
67 */
68 //-----
69 Function STWFlwCheck(cFlowAction, aParams)
70
71 Local lRet := .T.
72
73 Default cFlowAction := ""
74 Default aParams := []
75
76 Do Case
77
78     Case cFlowAction == "STBCSCancCupPrint"
79         lRet := CancCupPrint(aParams)
80
81 EndCase
82
83 Return lRet
84
85
86 //-----
87 /*/{Protheus.doc} STWFlwCheck
88 Verifica se existe acao pendente para encerrar o fluxo de rotina, quando ocorre a saida inesperada do Sistema (Queda de energia)
89
90 @return lRet          Retorna se a venda está em andamento ou não
91 @obs
92 @sample
93 */
94 //-----
95 Static Function CancCupPrint(aParams)
96
97 Local cDoc          := ""
98 Local nTamDocEcf    := 6          //Tamnho do Numero do Cupom ( 6 ou 9 digitos)
99 Local lRet          := .F.
100
101 Default aParams := []
102
103 //Verificar se solicitacao de cancelamento do ultimo cupom foi realizada pelo ECF

```

```
104 //Regra: Somente entra nesse fluxo caso tenha enviado o comando de cancelamento,
105 //sendo assim, se o numero do proximo cupom for igual o cupom que deseja cancelar +1, sinalizar que conseguiu realizar o comando.
106
107 cDoc          := STBRetCup()
108 nTamDocEcf    := STBLenReceipt(cDoc)
109 cDoc          := StrZero(Val(cDoc)-1,nTamDocEcf)
110
111 lRet := aParams[2] == cDoc
112
113 Return lRet
```

Componentes/FormPrice/STBGetSetPrice.prw

38 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static nPrice := 0
5
6 //-----
7 /*{Protheus.doc} STBSetPric
8 Seta o preço do item
9
10 @param      nValue - Preço
11 @return      Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STBSetPric(nValue)
17
18 Default nValue := 0
19
20 ParamType 0 Var nValue As Numeric Default 0
21
22 nPrice := nValue
23
24 Return Nil
25
26
27 //-----
28 /*{Protheus.doc} STBGetPric
29 Retorna o preço do item na variavel static
30
31 @param
32 @return      nPrice - Preço
33 @obs
34 @sample
35 */
36 //-----
37 Function STBGetPric()
38 Return nPrice

```

Componentes/FormPrice/STBValueExtendedWarranty.prw

57 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static nPercItem      := 0
5 Static nPricItem      := 0
6 Static nWarraPercItem := 0
7 Static nReturnPrice   := 0
8
9 //-----
10 /*{Protheus.doc} STBValExtW
11 Calcula valor da garantia
12
13 @param nPercentItem      Porcentagem do item
14 @param nPriceItem        Preco do item
15 @param nWarrantyPercentItem Porcentagem da garantia
16 @return nReturnPrice Preco
17 @obs
18 @sample
19 */
20 //-----
21 Function STBValExtW( nPercentItem, nPriceItem, nWarrantyPercentItem )
22
23 Default nPercentItem      := 0
24 Default nPriceItem        := 0
25 Default nWarrantyPercentItem := 0
26
27 ParamType 0 Var      nPercentItem      As Numeric      Default 0
28 ParamType 1 Var      nPriceItem        As Numeric      Default 0
29 ParamType 2 Var      nWarrantyPercentItem As Numeric      Default 0
30
31 nPercItem      := nPercentItem
32 nPricItem      := nPriceItem
33 nWarraPercItem := nWarrantyPercentItem
34
35 STBValWar()
36
37 Return nReturnPrice
38
39
40 //-----
41 /*{Protheus.doc} STBValWar
42 Calcula o valor do preco de garantia estendida.
43
44 @param
45 @return nReturnPrice Preco
46 @obs
47 @sample
48 */
49 //-----
50 Function STBValWar()
51
52 Local nValueRisco := 0 //Valor

```

```
53
54 nValueRisco      := a410arred(nPricItem + (nPricItem * nPercItem / 100), "L2_VRUNIT")
55 nReturnPrice     := a410arred(nValueRisco * nWarraPercItem / 100 + (nPricItem * nPercItem / 100), "L2_VRUNIT")
56
57 Return nReturnPrice
```

Componentes/FormPrice/STBValueTablePrice.prw

178 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cTablePrice := "" //Codigo da tabela de preco
5 Static lMvConfCli := SuperGetMV("MV_CONFCLI",,"N") == "S"
6
7 //-----
8 /*{Protheus.doc} ValueTablePrice
9 Retorna o preco do item conforme esta na tabela, considerando quando retornar
10 o maior ou menor preco conforme o parametro MV_LJRETVL
11 @param cItemCode   Codigo do item
12 @param cCustomer   cliente
13 @param cFil        Filial
14 @param cStore      Loja
15 @param nMoeda      Moeda
16 @return            nReturnPrice  Preco
17 @obs
18 @sample
19 */
20 //-----
21 Function STBValTbPr( cItemCode, cCustomer, cFil, cStore ,nMoeda , nQtde)
22
23 Local nX                := 0                //Variavel para o laço For
24 Local nRet              := 0                //Verifica se foi retornado o valor do item
25 Local aValues           := {}              //Armazena temporariamente os valores do item
26 Local aArea            := GetArea()        //Salva a area
27 Local oModel            := STDGetData()     //Retorna o model
28 Local oRecords         := oModel:GetModel("GridStr") //Retorna os registros do grid
29 Local nReturnPrice     := 0                //Retorna o resultado da funcao
30 Local cTabPad          := Pad(SuperGetMv("MV_TABPAD"),TamSx3("DA0_CODTAB")[1]) // Parametro da tabela de preco padrao
31 Local lCenVen          := SuperGetMv("MV_LJCNVDA") //Cenario de vendas
32 Local aTabsPrecos      := {}              //tabelas de preço
33 Local nPos             := 0
34 Local cParam           := STBParam()[1]    //Retorna o valor dos parametros para preco e se Retorna preco maior ou menor
35 Local cTabCli          := ""              //Preço de tabela do cliente
36 Local oCliModel        := STDGCLiModel()   //Model com as informacoes do cliente
37 Local cMvLjRetVl      := SuperGetMV("MV_LJRETVL",,"3") //1=Retorna o menor preco de uma tabela | 2=Retorna o maior preco de uma tabela | 3=Considera preco da tabela
                        configurada no parametro MV_TABPAD
38
39 Default cItemCode      := ""
40 Default cCustomer      := ""
41 Default cFil           := ""
42 Default cStore         := ""
43 Default nMoeda         := 0
44 Default nQtde          := 1
45
46 If lCenVen .And. FindFunction("STDTabsPre")
47
48   If lMvConfCli .AND. !Empty(oCliModel:GetValue("SA1MASTER","A1_TABELA"))
49     cTabCli := oCliModel:GetValue("SA1MASTER","A1_TABELA")
50   Endif
51

```

```

52  aTabsPrecos := STDTabsPre(cItemCode, cMvLjRetVl, cTabCli)
53
54  If cMvLjRetVl == "3" .AND. Len( aTabsPrecos ) > 0 .AND. aTabsPrecos[1][1] == "-999"
55
56      nReturnPrice := -999 // Tabela fora de vigencia
57
58  Else
59
60      For nX := 1 To Len(aTabsPrecos)
61
62          nRet := MaTabPrVen(aTabsPrecos[nX][1], cItemCode, nQtde, cCustomer,;
63                          cStore, nMoeda, /*dDataVld*/, /*nTipo*/,/*lExec*/)
64
65          If nRet > 0
66              aAdd( aValues,{aTabsPrecos[nX][1], nRet} )
67          EndIf
68
69      Next nX
70
71      CoNout('Cenario de vendas ativo, tabela de preco: ' + AllTrim(cTabPad))
72
73  EndIf
74
75 Else
76  //Apos verificar a variavel estatica, analisa se existe mais de
77  //uma tabela de preco ativa. Agora, verifica no MaTabPrVen
78  //se existe preco para esse produto
79  For nX := 1 To oRecords:LenGth()
80      oRecords:GoLine(nX)
81      nRet := MaTabPrVen( oRecords:GetValue("DA0_CODTAB") , cItemCode , nQtde, cCustomer,;
82                      cStore, nMoeda, /*dDataVld*/, /*nTipo*/,/*lExec*/)
83
84      //Armazena no array aValores os possiveis valores para ser
85      //praticado para um produto. Esse array sera utilizado para
86      //avaliar, caso o produto pertenca a mais de uma tabela de
87      //preco, para saber qual sera praticada.
88      If nRet > 0
89          aAdd( aValues,{ oRecords:GetValue("DA0_CODTAB") ,nRet} )
90      EndIf
91  Next nX
92 EndIf
93
94 //Sendo o aValores maior que um, significa que existe mais de
95 //um preco possivel para um unico produto na venda
96
97 If Len(aValues) > 0 .AND. nReturnPrice <> -999
98
99     //Retornando os valores, ordena do menor para o maior para
100    //ser tomada a decisao posteriormente
101
102    ASort(aValues,,,{|x,y| x[2]< y[2]})
103

```

```

104 //Verifica o parametro MV_LJRETVL
105 If cParam == "1" //utilizara o menor preco encontrado
106     nReturnPrice := aValues[1][2]
107     STBSetTblPr(aValues[1][1])
108 ElseIf cParam == "2" //utilizara o maior valor encontrado
109     nReturnPrice := aValues[Len(aValues)][2]
110     STBSetTblPr(aValues[Len(aValues)][1])
111 elseif cParam == "3" //utilizará a tbl de preço informada no parâmetro "MV_TABPAD"
112     If !Empty(cTabCli)
113         nPos := Ascan(aValues,{|x| x[1]== cTabCli})
114     Endif
115     //caso retorne 0, garante o preço de tab padrão.
116     If nPos == 0
117         nPos := Ascan(aValues,{|x| x[1]== cTabPad})
118     Endif
119     If ( nPos <> 0 )
120         nReturnPrice := aValues[nPos,2]
121         STBSetTblPr(aValues[nPos,1])
122     Endif
123 EndIf
124 EndIf
125
126 RestArea(aArea)
127
128 Return nReturnPrice
129
130
131 //-----
132 /*{Protheus.doc} STBParam
133 Retorna o valor dos parametros para preco do SB0 e se Retorna preco maior ou menor
134
135 @param      Nil
136 @return     cMvLjRetVl - Informa se retorna o maior ou menor preco
137 @return     lLjCnVB0 - Retorna preço da SB0 na ausencia do preço do Produto
138 @obs
139 @sample
140 */
141 //-----
142 Function STBParam()
143
144 Local cMvLjRetVl      := SuperGetMv("MV_LJRETVL",,"1")
145 Local lLjCnVB0       := SuperGetMv("MV_LJCnVB0",,.F.)
146
147 Return ({cMvLjRetVl, lLjCnVB0})
148
149 //-----
150 /*{Protheus.doc} STBSetTblPr
151 Seta o codigo da tabela de preco na variavel
152
153 @param      Nil
154 @return
155 @obs

```

//Parametro que verifica se deve retornar o maior ou menor preco
//Retorna preço da SB0 na ausência do preço do Produto na DA0 e DA1

```
156 @sample
157 /*/
158 //-----
159 Function STBSetTblPr(cTblPr)
160
161 Default cTblPr := ""
162
163 cTablePrice := cTblPr
164
165 Return .T.
166
167 //-----
168 /*/{Protheus.doc} STBGetTblPr
169 Captura o codigo da tabela de preco
170
171 @param      Nil
172 @return     cTablePrice - Retorna o codigo da tabela de preco
173 @obs
174 @sample
175 /*/
176 //-----
177 Function STBGetTblPr()
178 Return cTablePrice
```

Componentes/FormPrice/STDTablePrice.prw

373 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDTABLEPRICE.CH"
4
5 Static oCache      // Objeto de Cache
6 Static oNewModel   //Objeto para instanciar o model
7 Static oStructDA0  := Nil
8 Static oStructMst  := Nil
9
10 //-----
11 /*/{Protheus.doc} STDGetMod
12 Cria e retorna estrutura de dados do Model
13
14 @param      Nil
15 @return     oModel - Retorno o Modelo de dados
16 @obs
17 @sample
18 */
19 //-----
20 Function STDGetMod()
21
22 Local oModel      := Nil // Model
23 Local oMasterStr  := Nil
24 Local oGridStr    := Nil
25
26 //Monta estrutura de tabela de cabecario
27 oMasterStr := STDCrMsStr()
28
29 //Monta estrutura de tabela de Itens
30 oGridStr := STDCrGrStr()
31
32 //Instacia Objeto
33 oModel := MPFormModel():New( 'DA0', /*bPreValidacao*/, /*bPosValidacao*/, /*{ |oMdl| xFRT80MVCGR(oMdl)}*/, /*bCancel*/ )
34
35 oModel:AddFields( "MasterStr"      , /*cOwner*/, oMasterStr )
36 oModel:AddGrid("GridStr"          , "MasterStr", oGridStr, /*B*/ , /*bLinePost*/, /*bPreVal*/, /*bPosVal*/, /*BLoad*/ )
37 oModel:SetRelation("GridStr"       , { { "DA0_FILIAL", 'xFilial( "DA0" )' }, { "DA0_CODTAB", "DA0_CODTAB" } }, 'DA0_FILIAL + DA0_CODTAB' )
38
39 Return oModel
40
41
42 //-----
43 /*/{Protheus.doc} STDCrMsStr
44 Cria estrutura do Model Master
45
46 @param      Nil
47 @return     oStruct - Retorna a estrutura Master
48 @obs
49 @sample
50 */
51 //-----
52 Function STDCrMsStr()

```

```

53
54 Local cX2Unico      := "DA0_FILIAL+DA0_CODTAB" //X2_UNICO da tabela DA0
55
56 If ValType(oStructMst) = '0'
57     oStructMst:Deactivate()
58     oStructMst:Activate()
59 Else
60
61     oStructMst := FWFormModelStruct():New() // Estrutura
62
63     If ExistFunc('FWX2Unico')
64         cX2Unico := FWX2Unico("DA0")
65     EndIf
66
67     oStructMst:AddTable(
68         FWX2CHAVE()
69         StrTokArr( cX2Unico, '+' )
70         FWX2Nome("DA0") )
71
72     oStructMst:AddField(
73         "FILIAL"
74         "FILIAL"
75         "DA0_FILIAL"
76         "C"
77         FWSizeFilial()
78         0
79         Nil
80         Nil
81         Nil
82         Nil
83         Nil
84         Nil
85         Nil
86     )
87
88     oStructMst:AddField(
89         "CODIGO"
90         "CODIGO"
91         "DA0_CODTAB"
92         "C"
93         TamSX3("DA0_CODTAB")[1]
94         0
95         Nil
96         Nil
97         Nil
98         Nil
99         Nil
100        Nil
101        Nil
102    )
103 EndIf
104

```

```

105 Return oStructMst
106
107
108 //-----
109 /*{Protheus.doc} CreateGridStruct
110 Cria estrutura do Grid
111
112 @param      Nil
113 @return      oStruct - Retorna a estrutura do GRID
114 @obs
115 @sample
116 */
117 //-----
118 Function STDCrGrStr()
119
120 Local cAlias      := "DA0"                // Alias que criara estrutura
121 Local aArea       := Nil
122 Local nCount      := 0
123 Local cX2Unico    := "DA0_FILIAL+DA0_CODTAB" //X2_UNICO da tabela DA0
124
125 If ValType(oStructDA0) = '0'
126     oStructDA0:Deactivate()
127     oStructDA0:Activate()
128 Else
129     oStructDA0 := FWFormModelStruct():New() // Estrutura
130
131     aArea := GetArea() // Guarda area
132
133     If ExistFunc('FWX2Unico')
134         cX2Unico := FWX2Unico(cAlias)
135     EndIf
136
137     oStructDA0:AddTable(
138         FWX2CHAVE()                ;
139         StrTokArr( cX2Unico, '+' ) , ; // [01] Alias da tabela
140         FWX2Nome(cAlias) )         ; // [02] Array com os campos que correspondem a primary key
141                                     // [03] Descrição da tabela
142
143     //Carrega informacoes de campos
144     DbSelectArea("SX3")
145     SX3->(DbSetOrder(1))
146     SX3->(DbSeek( cAlias ))
147
148     While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == cAlias
149         nCount++
150         If nCount > 1000
151             Conout('STDCrGrStr - Count>1000',nCount)
152         EndIf
153
154         oStructDA0:AddField(
155             AllTrim( X3Titulo() ) , ; // [01] Titulo do campo
156             AllTrim( X3Descric() ) , ; // [02] ToolTip do campo
157             AllTrim( SX3->X3_CAMPO ) , ; // [03] Id do Field

```

```

157          SX3->X3_TIPO          , ; // [04] Tipo do campo
158          SX3->X3_TAMANHO       , ; // [05] Tamanho do campo
159          SX3->X3_DECIMAL        , ; // [06] Decimal do campo
160          Nil                    , ; // [07] Code-block de validação do campo
161          Nil                    , ; // [08] Code-block de validação When do campo
162          StrTokArr( AllTrim( X3CBox() ),',' ) , ; // [09] Lista de valores permitido do campo
163          Nil                    , ; // [10] Indica se o campo tem preenchimento obrigatório
164          Nil                    , ; // [11] Code-block de inicializacao do campo
165          Nil                    , ; // [12] Indica se trata-se de um campo chave
166          Nil                    , ; // [13] Indica se o campo pode receber valor em uma operação de update.
167          (SX3->X3_CONTEXT == 'V' )          , ; // [14] Indica se o campo é virtual
168      SX3->(DbSkip())
169      End
170
171      RestArea(aArea)
172  EndIf
173
174  Return oStructDA0
175
176
177  //-----
178  /*{Protheus.doc} GetAllData
179  Inclui os dados no model e retorna
180
181  @param
182  @return      oNewModel - Retorna Model
183  @obs
184  @sample
185  */
186  //-----
187  Function STDGetData()
188
189  Local nX      := 0          //Variavel de loop
190  Local lFirstLine := .T.     //Verifica se esta incluindo a primeira linha
191
192  oNewModel := STDGetMod()
193  oNewModel:SetOperation( 3 )
194  oNewModel:SetDescription(STR0001) //"Tabela de preco"
195  oNewModel:Activate()
196
197  DbSelectArea("DA0")
198  DbSetOrder(1)
199  DA0->(DbGoTop())
200
201  // Dados do master
202  oModelMaster := oNewModel:GetModel("MasterStr")
203  oModelMaster:SetValue("DA0_FILIAL" , DA0->DA0_FILIAL)
204  oModelMaster:SetValue("DA0_CODTAB" , DA0->DA0_CODTAB)
205
206  oModelGrid := oNewModel:GetModel("GridStr")
207  aTestStr   := oModelGrid:GetStruct()
208

```

```

209 // Carrega todas as tabelas de preco
210 While !DA0->(EOF()) .AND. (DA0_FILIAL == xFilial("DA0")) .AND. (DA0->DA0_ATIVO == "1");
211     .AND. (dDataBase >= DA0->DA0_DATDE) .AND. (dDataBase <= If(Empty(DA0->DA0_DATATE),dDataBase,DA0->DA0_DATATE))
212
213
214 //Avalia efetivamente se existe tabela de preco ativa para a data e hora atual
215 //considerando se o DA0_TPHORA e igual a "1" ou "2"
216 If (DA0->DA0_TPHORA == "1"
217     (SubtHoras(dDataBase,Time(),If(Empty(DA0->DA0_DATATE),dDataBase,DA0->DA0_DATATE),DA0->DA0_HORATE) >= 0 .AND. ; .AND. ;
218     SubtHoras(DA0->DA0_DATDE,DA0->DA0_HORADE,dDataBase,Time()) >= 0)) .OR. ;
219     (DA0->DA0_TPHORA == "2" .AND. ; .AND. ;
220     (dDataBase >= DA0->DA0_DATDE .And. dDataBase <= If(Empty(DA0->DA0_DATATE),dDataBase,DA0->DA0_DATATE) .AND. ;
221     (SubStr(Time(),1,5) >= DA0->DA0_HORADE .And. SubStr(Time(),1,5) <= DA0->DA0_HORATE)))
222
223 //Se entrou a(s) tabela(s) o model
224 If !lFirstLine
225     nLine := oModelGrid:AddLine()
226 Else
227     lFirstLine := .F.
228 EndIf
229
230 For nX := 1 To Len(aTestStr:aFields)
231     oModelGrid:SetValue(aTestStr:aFields[nX][3], &("DA0->" + aTestStr:aFields[nX][3]))
232 Next nX
233
234 EndIf
235
236 DA0->(DbSkip())
237
238 End
239
240 Return oNewModel
241
242 //-----
243 /*{Protheus.doc} SubtHoras
244 Calcula o Numero de Horas entre dois tempos.
245 @param Funcao duplicada para evitar de colocar o fonte inteiro
246 @return numero de horas
247 */
248 //-----
249 Static Function SubtHoras(dDataIni,cHoraIni,dDataFim,cHoraFim,lHrCont)
250
251 Local nDias := dDataFim - dDataIni
252 Local nHoras:= HoraToInt(cHoraFim)-HoraToInt(cHoraIni)
253 //Horario Continuo. Ex. 10/10/2010 08:00 até 11/10/2010 17:00 como 32 horas. Quando .F. Considera apenas 8 hs dia 10 + 8hs dia 11 = 16hs
254 Default lHrCont := .T. //Considera horario continuo.
255
256 If nDias > 0 .AND. lHrCont == .F.
257     nHoras := nHoras+(nHoras * nDias)
258 Else
259     nHoras := nHoras+(nDias*24)
260 EndIf

```

```

261
262 Return(nHoras)
263
264 /*{Protheus.doc} HoraToInt
265 Calcula o Numero de Horas entre dois tempos.
266 @param      Funcao duplicada para evitar de colocar o fonte inteiro
267 @return     numero de horas
268 */
269 //-----
270 Static Function HoraToInt(cHora,nDigitos)
271
272 Local nHoras
273 Local nMinutos
274 Local nExtDigit
275
276 nExtDigit := If( ValType( nDigitos ) == "N", nDigitos - 2, 0 )
277
278 nHoras    := Val(SubStr(cHora,1,2 + nExtDigit ))
279 nMinutos  := Val(SubStr(cHora,4 + nExtDigit,2))/60
280
281 Return(nHoras+nMinutos)
282
283
284 //-----
285 /*{Protheus.doc} STDTabsPre
286 Funcao para recuperar as tabelas de preço do produto.
287 @param cItemCode      , caractere, Produto a ser consultado.
288 @param cMvLjRetVl     , caractere, Valor da tabela de Preço
289 @param cTabCli        , caractere, Tabela de Preço do Cliente, caso exista
290 @return  Tabelas DA1 (Tabelas de peço, Preço de Venda), onde o produto exista
291 */
292 //-----
293 Function STDTabsPre(cItemCode,cMvLjRetVl, cTabCli)
294
295 Local aAreaDA1 := {} //Guarda area
296 Local aAreaDA0 := {} //Guarda area
297 Local aRet     := {} //retorno
298 Local cSeek    := "" //Seek de busca DA0
299 Local cTabPad  := "" //Tabela de preco padrao
300 Local dValidTab := Date() // Data limite de validade da tabela de preço
301
302 Default cItemCode := " "
303 Default cMvLjRetVl := SuperGetMV("MV_LJRETVL",,"3") // 1=Retorna o menor preco de uma tabela | 2=Retorna o maior preco de uma tabela | 3=Considera preco da tabela
               configurada no parametro MV_TABPAD
304 Default cTabCli := ""
305
306 //Quando configurado para retornar preco da MV_TABPAD(cMvLjRetVl=3),
307 //nao realiza a busca na DA1, apenas retorna o conteudo do TABPAD
308 If cMvLjRetVl == "3"
309
310     IF !Empty(cTabCli)
311         cTabPad := cTabCli //tabela de preço do cliente

```

```

312 Else
313     cTabPad := Padr(AllTrim(SuperGetMv("MV_TABPAD",,"")),TamSX3("DA0_CODTAB")[1]) // Tabela de preco padrao
314 EndIf
315
316 If Len(cTabPad) > 1
317
318     dValidTab := GetAdvFVal( "DA0" , "DA0_DATATE" , xFILIAL("DA0") + cTabPad, 1 , 0 )
319
320     If !Empty(dValidTab) .And. dValidTab < Date()
321         LjGrvLog( cItemCode, "Tabela de preço fora de vigência. Verifique o código da tabela contido no parâmetro MV_TABPAD | Tabela de preço" , cTabPad )
322         AAdd(aRet,{"-999", 0})
323     Else
324         aAdd(aRet,{cTabPad, 0})
325     EndIf
326
327 EndIf
328
329 Else
330     aAreaDA1 := DA1->(GetArea())
331     aAreaDA0 := DA0->(GetArea())
332
333     DbSelectArea ("DA0")
334     DA0->(DbSetOrder(1)) //DA0_FILIAL+DA0_CODTAB
335
336     cSeek := xFilial("DA0")
337     LjGrvLog( cItemCode, "Ira pesquisar tabelas da DA0 onde o produto possui cadastro na DA1(MV_LJRETVL <> 3)")
338
339     If DA0->(DbSeek(cSeek))
340         While DA0->(!Eof()) .AND. DA0->DA0_FILIAL == cSeek
341             If DA0->DA0_ATIVO == "1"//percorre todas as tabelas cadastradas ativas/vigentes (Regra padrao fica centralizada na rotina MaTabPrVen), nesse ponto é realizado
validacao basica
342                 If (dDataBase >= DA0->DA0_DATDE) .AND. (dDataBase <= If(Empty(DA0->DA0_DATATE),dDataBase,DA0->DA0_DATATE))
343                     If (DA0->DA0_TPHORA == "1"
344                         (SubtHoras(dDataBase,Time(),If(Empty(DA0->DA0_DATATE),dDataBase,DA0->DA0_DATATE),DA0->DA0_HORATE) >= 0
345                         SubtHoras(DA0->DA0_DATDE,DA0->DA0_HORADE,dDataBase,Time()) >= 0))
346                         (DA0->DA0_TPHORA == "2"
347                         (dDataBase >= DA0->DA0_DATDE .And. dDataBase <= If(Empty(DA0->DA0_DATATE),dDataBase,DA0->DA0_DATATE)
348                         (SubStr(Time(),1,5) >= DA0->DA0_HORADE .And. SubStr(Time(),1,5) <= DA0->DA0_HORATE)))
349                     DA1->(DbSetOrder(1)) //DA1_FILIAL+DA1_CODTAB+DA1_CODPRO+DA1_INDLOT+DA1_ITEM
350                     If DA1->(DbSeek(DA0->DA0_FILIAL+DA0->DA0_CODTAB+cItemCode)) //Verifica se item cadastrado na tabela
351                         While DA1->(!Eof()) .And. xFilial("DA1") + DA0->DA0_CODTAB + Alltrim(cItemCode) == DA1->DA1_FILIAL + DA1->DA1_CODTAB + Alltrim(DA1->DA1_CODPRO)
352                             If DA1->DA1_DATVIG <= Date() .And. DA1->DA1_ATIVO == "1"
353                                 aAdd(aRet, {DA0->DA0_CODTAB, DA1->DA1_PRCVEN})
354                             Exit
355                         Endif
356                     DA1->(dbSkip())
357                 End
358             EndIf
359         EndIf
360     EndIf
361 EndIf
362 DA0->(DbSkip())

```

```
363         EndDo
364     EndIf
365
366     LjGrvLog( cItemCode, "Tabelas encontradas com item na DA1:",aRet)
367
368     RestArea(aAreaDA1)
369     RestArea(aAreaDA0)
370
371 EndIf
372
373 Return(aRet)
```

Componentes/FormPrice/STDValuePriceDefault.prw

162 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cTabPad := SuperGetMv("MV_TABPAD",,"1") //Define qual coluna da SB0 sera o preco default
5 Static lTabPad := (SB0->(FieldPos("B0_PRV" + cTabPad)) > 0) //Verifica se o campo existe na SB0
6
7 //-----
8 /*{Protheus.doc} ValuePriceDefault
9 Função responsável em localizar o preço do produto na SB0 conforme o parametro
10 MV_TABPAD
11
12 @param      cItemCode  Código do Item
13 @return     nReturnPrice - Preço
14 @obs
15 @sample
16 */
17 //-----
18 Function STDValPrDf( cItemCode )
19
20 Local nReturnPrice := 0 //Retorno da funcao
21
22 Default cItemCode := ""
23
24 ParamType 0 Var      cItemCode  As Character      Default      ""
25
26 DbSelectArea( "SB0" )
27
28 If lTabPad
29
30     If DbSeek(xFilial("SB0") + cItemCode)
31         nReturnPrice := &("SB0->B0_PRV" + cTabPad)
32     EndIf
33
34 EndIf
35
36 Return nReturnPrice
37
38
39 //-----
40 /*{Protheus.doc} STBPrecoB1
41 Retorna o preço do item da tabela SB1
42
43 @param      cItemCode  Código do Item
44 @return     nReturnPrice - Preço
45 @obs
46 @sample
47 */
48 //-----
49 Function STDPrecoB1( cItemCode )
50
51 Local aArea      := GetArea() //Salva a area
52 Local nReturnPrice := 0 //Retorno da funcao

```

```

53
54 Default cItemCode := ""
55
56 ParamType 0 Var      cItemCode  As Character      Default      ""
57
58 DbSelectArea      ( "SB1" )
59 DbSetOrder( 1 ) //B1_FILIAL + B1_COD
60
61 If DbSeek(xFilial("SB1") + cItemCode)
62     nReturnPrice := SB1->B1_Prv1
63 EndIf
64
65 RestArea(aArea)
66
67 Return nReturnPrice
68
69
70 //-----
71 /*/{Protheus.doc} STBChgPrDf
72 Função responsável em retornar os 9 preços de um determinado produto
73
74 @param      cItemCode - Código do item
75 @return     aPrices - Retorna uma lista de preços
76 @obs
77 @sample
78 */
79 //-----
80 Function STDChgPrDf( cItemCode )
81
82 Local aPrices      := {}                //Variavel para retorno dos preços
83 Local nI           := 1                //Variavel de loop
84 Local aArea := GetArea()                //Salva a area
85
86 Default cItemCode := ""
87
88 ParamType 0 Var      cItemCode  As Character      Default      ""
89
90 DbSelectArea( "SB0" )
91 DbSetOrder( 1 ) //B0_FILIAL + B0_COD
92
93 If DbSeek(xFilial("SB0") + cItemCode)
94     For nI := 1 to 9
95
96         /*/If lLJ7044
97             xRet := U_LJ7044(nX)
98             If ValType(xRet) == "L"
99                 If !xRet
100                     Loop
101                 Endif
102             Endif
103         Endif/*/
104

```

```

105      //Checar se o campo da tabela de precos esta 'usado' e o nivel do usuario permite ver o campo
106      DbSelectArea( "SX3" )
107      DbSetOrder( 2 )
108
109      If DbSeek(PadR("B0_PRV" + Str(nI,1,0),10," ") ) .AND. X3Uso(SX3->X3_USADO) .AND. cNivel >= SX3->X3_NIVEL
110
111          If Empty(SB0->&("B0_DATA" + Str(nI,1,0))) .OR. SB0->&("B0_DATA" + Str(nI,1,0)) >= dDatabase
112
113              aAdd( aPrices, { StrZero(nI,2,0), SB0->&("B0_PRV" + Str(nI,1,0)) } )
114
115          EndIf
116
117      EndIf
118
119  Next nI
120 EndIf
121
122 RestArea(aArea)
123
124 Return aPrices
125
126
127 //-----
128 /*{Protheus.doc} AddCurrency
129 Funcao responsavel em adicionar a coluna moeda quando for localizado
130
131 @param      aPrices Recebe os valores da SB0
132 @param      cItemCode  Codigo do Item
133 @return     aPrices - valores da SB0
134 @obs
135 @sample
136 */
137 //-----
138 Function STDAcCurre( aPrices, cItemCode )
139 Local nI          := 1          //Variavel de loop
140 Local aArea := GetArea()      //Salva a area
141
142 Default aPrices      := {}
143 Default cItemCode    := ""
144
145 ParamType 0 Var  aPrices  As Array  Default  {}
146 ParamType 1 Var  cItemCode As Character Default  ""
147
148
149 DbSelectArea( "SB0" )
150 DbSetOrder( 1 ) //B0_FILIAL + B0_COD
151
152 If DbSeek(xFilial("SB0") + cItemCode)
153
154     For nI := 1 To Len(aPrices)
155         AAdd(aPrices[nI],Capital(SuperGetMV("MV_MOEDA" + LTrim(Str(Max{SB0->&("B0_MOEDA" + Str(nI,1,0)),1}))))
156     Next nI

```

```
157  
158 EndIf  
159  
160 RestArea(aArea)  
161  
162 Return aPrices
```

Componentes/FormPrice/STWFormPrice.prw

80 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protoheus.doc} STWFormPr
6 Workflow do componente formacao de preco
7
8 1o. Pesquisa o preco nas tabelas de preco DA0 e DA1
9 2o. Caso nao retorna o preco na primeira opcao, pesquisa o preco na SB0
10 3o. Por ultimo, caso nao retorne o preco em nenhuma das opcoes, pesquisa na SB1
11
12 @param      cItemCode Codigo do item
13 @param      cCustomer Codigo do cliente
14 @param      cFil       Codigo da filial
15 @param      cStore     Codigo da loja
16 @param      nMoeda     Moeda
17 @return     nRet - Preco
18 @obs
19 @sample
20 */
21 //-----
22 Function STWFormPr( cItemCode, cCustomer, cFil, cStore, nMoeda, nQtde )
23
24 Local nRet      := 0      //Retorno do preco
25 Local lFinServ  := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
26 Local lCenVen   := SuperGetMV("MV_LJCNVDA",,.F.) //Cenario de vendas
27 Local aAreaSB1  := SB1->(GetArea()) // Guarda area atual do SB1
28
29 Default cItemCode := ""
30 Default cCustomer := ""
31 Default cFil      := ""
32 Default cStore    := ""
33 Default nMoeda    := 0
34 Default nQtde     := 1
35
36 ParamType 0 Var cItemCode As Character Default ""
37 ParamType 1 Var cCustomer As Character Default ""
38 ParamType 2 Var cFil      As Character Default ""
39 ParamType 3 Var cStore    As Character Default ""
40 ParamType 4 Var nMoeda    As Numeric Default 0
41 ParamType 5 Var nQtde     As Numeric Default 1
42
43
44 cItemCode := PadR(cItemCode,TamSx3("B1_COD")[1])
45
46 If STBGetPric() == 0
47
48     /* Tratamento Servico Financeiro */
49     If lFinServ
50         //Verifica se produto serviço é com Vinculado. Se for fixo, seu preço será do processo normal ( DA0-DA1 / SB0 / SB1 ).
51         nRet := STWServPrice( cItemCode, cFil )
52     EndIf

```

```
53
54   If nRet == 0
55       If lCenVen
56           nRet := STBValTbPr(      cItemCode, cCustomer, cFil, cStore,nMoeda, nQtde )
57       EndIf
58
59       If nRet == 0
60
61           nRet := STDValPrDf( cItemCode )
62
63           If nRet == 0
64               nRet := STDPrecioB1( cItemCode )
65           EndIf
66       EndIf
67   EndIf
68 Else
69     nRet := STBGetPric()
70     STBSetPric(0)
71 EndIf
72
73 If nRet <> -999
74     //Faz arredondamento do preço
75     nRet := STBRound(nRet) //Padrao de decimais "MV_CENT"+ Moeda corrente
76 EndIf
77
78 RestArea(aAreaSB1)
79
80 Return nRet
```

Componentes/General/STDGeneralFunctions.prw

462 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "AUTODEF.CH"
3 #INCLUDE "STDGENERALFUNCTIONS.CH"
4
5 /*/
6
7     Fonte contendo funcoes genericas utilizadas pelo Totvs PDV.
8
9 /*/
10
11
12 //-----
13 /*{Protheus.doc} STDExistChav
14 Funcao semelhante a ExistChav, porem esta ao inves de interagir com o usuario atraves da janela do help, o faz via componente de mensagem.
15
16 @param
17 @return lRet          Retorno .T. se nao houver registro com a mesma chave unica na base.
18 @obs
19 @sample
20 /*/
21 //-----
22 Function STDExistChav(cAliasArq,cKey,nInd,cHelp,nFAtuCli)
23 Local aArea := GetArea()
24 Local lRet := .T.
25
26 DEFAULT cAliasArq := "SA1"
27 DEFAULT cKey      := ""
28 DEFAULT nInd      := 1
29 DEFAULT cHelp     := "Já existe registro com esta informação."
30 DEFAULT nFAtuCli  := 2    // Força a atualizacao do cliente
31
32 DbSelectArea(cAliasArq)
33 (cAliasArq)->(DbSetOrder(nInd))
34 If DbSeek(xFilial(cAliasArq)+cKey)
35     lRet := .F.
36 EndIf
37
38 If !lRet .AND. nFAtuCli == 2
39     STFMessage(ProcName(),"STOP",cHelp)
40     STFShowMessage(ProcName())
41 Else
42     STFCleanInterfaceMessage()
43 EndIf
44
45 RestArea(aArea)
46
47 Return lRet
48
49 //-----
50 /*{Protheus.doc} STDUF
51 Funcao que retorna um array com as unidades federativas.
52

```

```

53 @param
54 @return lRet          Retorno .T. se nao houver registro com a mesma chave unica na base.
55 @obs
56 @sample
57 /*/
58 //-----
59 Function STDUF()
60 Local aArea      := GetArea()                // Area atual
61 Local lEndFis    := SuperGetMv("MV_SPEDEND",, .F.) // Se estiver como F refere-se ao endereço de Cobrança se estiver T ao endereço de Entrega.
62 Local cEstSM0    := UPPER(If(!lEndFis, SM0->M0_ESTCOB, SM0->M0_ESTENT)) // Indica qual estado sera considerado, se o de entrega ou cobrança
63 Local aRet       := {{""},{}}               // Array Default, array de retorno
64
65 DbSelectArea("CC2")
66 CC2->(DbSetOrder(1))
67 DbSeek(xFilial("CC2"))
68
69 // Estado atual do sistema baseado no parametro MV_SPEDEND
70 aAdd(aRet, {cEstSM0,{}})
71
72 While CC2->CC2_FILIAL == xFilial("CC2") .AND. !EOF()
73
74     If Alltrim(CC2->CC2_EST) <> Alltrim(cEstSM0)
75         aAdd(aRet,{CC2->CC2_EST,{}})
76         nPosEst := Len(aRet)
77     Else
78         // 0 estado do atual do sistema sempre será 2.
79         nPosEst := 2
80     EndIf
81
82     While CC2->CC2_FILIAL + CC2_EST == xFilial("CC2") + aRet[nPosEst][1] .AND. !EOF()
83         aAdd(aRet[nPosEst][2],{CC2_CODMUN,CC2_MUN})
84         CC2->(DbSkip())
85     End
86
87 End
88
89 RestArea(aArea)
90
91 Return aRet
92 //-----
93 /*{Protheus.doc} STFIsFiscalPrint
94 Seta se utiliza impressora fiscal na configuração e retorna se pode entrar no sistema
95
96 @param
97 @return lRet          Retorna se pode entrar no sistema
98 @obs
99 @sample
100 /*/
101 //-----
102 Function STFIsFiscalPrint()
103
104 Local lRet           := .F.                // Retorna se pode entrar no sistema

```

```

105 Local cModelPrint      := STFFGetStation("IMPFISC") // Modelo impressora estação
106 Local aDados           := {}                      // Retorno evento
107
108 /*
109     Utiliza impressora fiscal se:
110     1 - Brasil
111     2 - Usuário fiscal
112     3 - Modelo da impressora cadastrada for fiscal
113 */
114
115 If cPaisLoc == "BRA"
116
117     aDados := STFFireEvent(
118         ProcName(0)           ,;      // Nome do processo
119         "STExistEquip"       ,;      // Nome do evento
120         {EQUIP_IMPFISCAL     ,;      // 01 - Tipo
121         cModelPrint          ,;      // 02 - Modelo
122         Nil                  ,;      // lCommuOk
123         .F.                  })      // Define se abre comunicação
124
125     /*
126     Impressora Fiscal
127     */
128     If Len(aDados) > 0 .AND. aDados[1]
129         If STFFProfile(3)[1]
130             STFFSetCfg( "lUseECF" , .T. )
131             lRet := .T.
132         Else
133             HELP(" ",1,"FRT002")          // "Usuário 1 para usar impressora fiscal.", "Atenção"
134             lRet := .F.
135         EndIf
136     /*
137     Impressora nao fiscal
138     */
139     Else
140         If STFFProfile(3)[1]
141             HELP(" ",1,STR0001)          // "Usuário fiscal não pode utilizar impressora não fiscal.", "Atenção"
142             lRet := .F.
143         Else
144             STFFSetCfg( "lUseECF" , .F. )
145             lRet := .T.
146         EndIf
147     Else
148         STFFSetCfg( "lUseECF" , .F. )
149         lRet := .T.
150     EndIf
151
152 Return(lRet)
153
154 //-----
155 /*{Protheus.doc} STDFisGetEnd
156 Retorna a estrutura do endereço passado

```

```

157
158 @param
159 @return lRet          Retorna a estrutura do endereco passado
160 @obs
161 @sample
162 /**
163 //-----
164
165 Function STDFisGetEnd(cEndereco, cUF)
166 Local nVirgula      := Rat(",",cEndereco) // Insere a Virgula antes do Endereço
167 Local cNumero       := "" // Caracter do Numero
168 Local nNumero       := 0 // Numero
169 Local lEndNfe       := If(FunName()=="SPEDNFE", .T., .F.) // Se fo SPEDNFE
170 Local cEnderec      := "" // Endereço
171 Local cCompl        := "" // Complemento
172 Local cComplemen    := "" // Complemento auxiliar
173 Local lExterior     := .F. // Exterior
174 Local cEndAlte      := "" // Endereço alterado
175
176 Default cUF         := "" // UF
177 Default cEndereco   := "" // Endereço recebido
178
179 If ExistBlock('FISATEND')
180     cEndAlte := ExecBlock('FISATEND',.F.,.F.,{ cEndereco, cUF })
181     If !Empty(cEndAlte)
182         cEndereco := cEndAlte
183     EndIf
184 EndIf
185 lExterior := (cUF == "EX")
186
187 cNumero := If(!lExterior, AllTrim(SubStr(cEndereco,nVirgula+1)), Left(cEndereco, nVirgula-1))
188 nNumero := NoRound(Val(cNumero),3)
189 cCompl := If(!lExterior, AllTrim(SubStr(cEndereco,nVirgula+1)), Left(cEndereco, nVirgula-1))
190 cComplemen := ""
191
192 If lEndNfe
193     If nNumero != 0
194         If !lExterior
195             cEnderec := PadR(SubStr(cEndereco, 1, nVirgula-1), 60)
196         Else
197             cEnderec := PadR(LTrim(SubStr(cEndereco, nVirgula+1)), 60)
198         EndIf
199     Else
200         cEnderec := PadR(cEndereco, 60)
201     EndIf
202 Else
203     If nNumero != 0
204         If !lExterior
205             cEnderec := PadR(SubStr(cEndereco,1,nVirgula-1),34)
206         Else
207             cEnderec := PadR(LTrim(SubStr(cEndereco, nVirgula+1)), 34)
208         EndIf

```

```

209 Else
210     cEnderec := PadR(SubStr(cEndereco,1,nVirgula-1),34)
211 EndIf
212 EndIf
213
214 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
215 //³Quando nao ha virgula no endereco procura-se o caracter branco³
216 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
217 If ( nVirgula == 0 )
218     nVirgula := Rat(" ",AllTrim(cEndereco))
219     cEnderec := RTrim(cEndereco)
220     cCompl := "" //NAO TEM COMO PEGAR O COMPLEMENTO, JAH QUE UTILIZO O ULTIMO ESPACO A DIREITO PARA SEPARAR O LOGRADOURO DO NUMERO.
221     cNumero := AllTrim(SubStr(cEndereco,nVirgula+1))
222     nNumero := Val(cNumero)
223     If lEndNfe == .F.
224         lEnderec := PadR(If(nNumero!=0,SubStr(cEndereco,1,nVirgula-1),cEndereco),34)
225     Else
226         lEnderec := PadR(If(nNumero!=0,SubStr(cEndereco,1,nVirgula-1),cEndereco),60)
227     EndIf
228 EndIf
229 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
230 //³Quando o numero é numerico, obtem-se o complemento³
231 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
232 If nNumero <> 0
233     If At(" ",AllTrim(cCompl)) > 0
234         cComplemen := Alltrim(SubStr(cCompl,At(" ",AllTrim(cCompl))+1))
235     Endif
236 EndIf
237 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
238 //³Para o numero caracter extrai o complemento.³
239 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
240 cNumero := StrTran(cNumero,cComplemen,"")
241
242 Return({cEnderec,nNumero,cNumero,cComplemen})
243
244
245 //-----
246 /*{Protheus.doc} STDFisGetTel
247 Retorna a estrutura do telefone passado
248
249 @param
250 @return lRet          Retorna a estrutura do telefone passado
251 @obs
252 @sample
253 */
254 //-----
255 Function STDFisGetTel(cTelefone,cArea,cPais)
256
257 Local nX      := 0 // Variável pra Loop
258 Local nCount := 0 // Contador
259 Local cAux   := "" // Auxiliar
260 Local cNumero := "" // Numero

```

```

261 Local lFone    := .T. // Lógica do Fone
262 Local lArea    := .F.  // Lógica da Area
263 Local lPais    := .F. // Lógica do País
264
265 DEFAULT cArea := "" // Area
266 DEFAULT cPais := "" // Pais
267 DEFAULT cTelefone := "" // Telefone
268
269
270 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
271 //³Verifico o que deve ser extraído do numero do telefone      ³
272 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
273 lArea := Empty(cArea)
274 lPais := Empty(cPais) .And. lArea
275 cTelefone := AllTrim(cTelefone)
276
277 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
278 //³Obtenho o código de pais/area e telefone do Telefone      ³
279 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
280
281 For nX := Len(cTelefone) To 1 Step -1
282     nCount++
283     cAux := SubStr(cTelefone,nX,1)
284     If cAux >= "0" .And. cAux <= "9"
285         Do Case
286             Case lFone
287                 cNumero := cAux + cNumero
288             Case lArea
289                 cArea := cAux + cArea
290             Case lPais
291                 cPais := cAux + cPais
292             EndCase
293             If (nCount == 9)
294                 lFone := .F.
295             Endif
296     Else
297         Do Case
298             Case lFone
299                 If Len(cNumero) > 5
300                     lFone := .F.
301                 EndIf
302             Case lArea
303                 If !Empty(cArea)
304                     lArea := .F.
305                 EndIf
306             EndCase
307         EndIf
308 Next nX
309
310 Return({Val(cPais),Val(cArea),Val(cNumero)})
311
312

```

```

313 //-----
314 /*{Protheus.doc} STDAsc2Hex
315 Converte um texto com caracteres ASCII em texto com caracte-
316 res HEXA.
317
318 @param
319 @return lRet
320 @obs
321 @sample
322 */
323 //-----
324 Function STDAsc2Hex(cString)
325
326 Local nX      := 1 // Contador
327 Local cResult := "" // Resultado
328 Local nVal    := 0 // Valor
329
330 Default cString := ""
331
332 For nX := 1 to Len(cString)
333     nVal := Asc(SubStr(cString,nX,1))
334     cResult += STDDec2Hex(nVal)
335 Next nX
336
337 cResult := Lower(cResult)
338
339 Return(cResult)
340
341
342 //-----
343 /*{Protheus.doc} STDDec2Hex
344 Converte um numero decimal ate' 255 para hexadecimal
345
346 @param
347 @return lRet
348 @obs
349 @sample
350 */
351 //-----
352 Function STDDec2Hex(nVal)
353 Local cString := "0123456789ABCDEF"
354
355 Default nVal := ""
356
357 Return(Substr(cString,Int(nVal/16)+1,1)+Substr(cString,nVal-(Int(nVal/16)*16)+1,1))
358
359
360 //-----
361 /*{Protheus.doc} STDQueryDB
362 Executa query no banco de dados da Retaguarda.
363 @param aFields - Campos para montagem da Query.
364 @param aTables - Tabelas para montagem da Query.

```

```

365 @param      cWhere      - Cláusula "WHERE" para montagem da Query.
366 @param      cOrderBy    - Cláusula "ORDER BY" para montagem da Query.
367 @param      nLimitRegs  - Quantidade de registros que se deseja limitar a pesquisa da query.
368 @return      aRet        - Array com o resultado da query executada.
369 @obs
370 @sample
371 /*/
372 //-----
373 Function STDQueryDB( aFields, aTables, cWhere, cOrderBy, nLimitRegs )
374 Local aRet      := {}
375 Local aArea     := GetArea()
376 Local cAliasQry := ""
377 Local cQuery    := ""
378 Local cEndQuery := ""
379 Local nY        := 0
380 Local nX        := 1
381 Local cFields   := ""
382 Local cTables   := ""
383 Local lExecQuery := .T.
384 Local cSGBD     := AllTrim(Upper(TcGetDb())) //Tipo do banco de dados em uso
385
386 Default aFields := {}
387 Default aTables := {}
388 Default cWhere  := ""
389 Default cOrderBy := ""
390 Default nLimitRegs := 0
391
392 For nX:=1 To Len(aFields)
393     cFields += aFields[nX]+", "
394 Next nX
395 cFields := Left(cFields,Len(cFields)-1) //Retira a última vírgula
396
397 For nX:=1 To Len(aTables)
398     cTables += RetSQLName(aTables[nX]) + " " + aTables[nX] + " ,"
399 Next nX
400 cTables := Left(cTables,Len(cTables)-1) //Retira a última vírgula
401
402 If Empty(cFields) .Or. Empty(cTables) .Or. Empty(cWhere)
403     lExecQuery := .F.
404 EndIf
405
406 If lExecQuery
407
408     cAliasQry := GetNextAlias()
409
410     If !Empty(cOrderBy)
411         cOrderBy := " ORDER BY " + cOrderBy
412     EndIf
413
414     //Tratamento para trazer uma quantidade limitada de registros
415     If nLimitRegs > 0
416

```

```

417     If "MSSQL" $ cSGBD //Microsoft SQL Server
418         cFields := " TOP " + AllTrim(Str(nLimitRegs)) + " " + cFields
419     ElseIf "ORACLE" $ cSGBD //Oracle
420         cWhere += " AND ROWNUM <= " + AllTrim(Str(nLimitRegs))
421     ElseIf "DB2" $ cSGBD //DB2
422         cEndQuery := "FETCH FIRST " + AllTrim(Str(nLimitRegs)) + " ROWS ONLY"
423     ElseIf "INFORMIX" $ cSGBD //Informix
424         cFields := "FIRST " + AllTrim(Str(nLimitRegs)) + " " + cFields
425     ElseIf "SYBASE" $ cSGBD //Sybase
426         cFields := " TOP " + AllTrim(Str(nLimitRegs)) + " " + cFields
427     ElseIf "POSTGRES" $ cSGBD //PostgreSQL
428         cEndQuery := " LIMIT " + AllTrim(Str(nLimitRegs))
429     ElseIf "MYSQL" $ cSGBD //MySQL
430         cEndQuery := " LIMIT " + AllTrim(Str(nLimitRegs))
431     EndIf
432
433 EndIf
434
435 //Montagem da query
436 cQuery := "SELECT " + cFields
437 cQuery += " FROM " + cTables
438 cQuery += " WHERE " + cWhere
439 cQuery += cOrderBy
440 cQuery += cEndQuery
441 cQuery := ChangeQuery(cQuery)
442 DbUseArea(.T., "TOPCONN", TcGenQry(, cQuery), cAliasQry, .T., .T.)
443
444 nX := 0
445 While (cAliasQry) -> (!EOF())
446     aAdd(aRet, {})
447     nX++
448
449     For nY := 1 To Len(aFields)
450         aAdd(aRet[nX], &((cAliasQry) -> (aFields[nY])))
451     Next nY
452
453     (cAliasQry) -> (DbSkip())
454 End
455
456 (cAliasQry) -> (DbCloseArea())
457
458 EndIf
459
460 RestArea(aArea)
461
462 Return aRet

```

Componentes/GiftCard/STBValePresente.prw

130 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 Static aVlrVP := {}
4 Static aCodVP := {}
5
6 //-----
7 /*{Protheus.doc} STBGetMinMaxVP
8 Retorna o valor minimo e maximo do vale presente
9
10 @param cCodProduto  codigo do produto
11 @return aRes valor minino e maximo vale presente
12 @obs
13 @sample
14 */
15 //-----
16 Function STBGetMinMaxVP(cCodProduto)
17
18 Local aRes := {}
19 Local lRet := .F.
20
21 Default cCodProduto := 0
22
23 lRet := STBRemoteExecute("STDGetMinMaxVP" ,{cCodProduto}, NIL,.F., @aRes)
24
25 Return aRes
26
27 //-----
28 /*{Protheus.doc} STBSetVlrVP
29 Adiciona à um array os valores dos vales presentes vendidos venda variavel
30
31 @param cCodVP  codigo vale presente
32 @param nValor  valor do vale presente
33 @return aRes valores vales presentes
34 @obs
35 @sample
36 */
37 //-----
38 Function STBSetVlrVP(cCodVP,nValor)
39
40 Default cCodVP := ""
41 Default nValor := 0
42
43 If !Empty(cCodVP)
44     AAdd(aVlrVP, {cCodVP, nValor})
45 Else
46     aVlrVP := {}
47 EndIf
48
49 Return aVlrVP
50
51 //-----
52 /*{Protheus.doc} STBGetVlrVP

```

```

53 Recupera array com os valores dos vales presentes vendidos
54
55 @param
56 @return aVlrVP valores dos vales presentes
57 @obs
58 @sample
59 /*/
60 //-----
61 Function STBGetVlrVP()
62 Return aVlrVP
63
64 //-----
65 /*{Protheus.doc} STBIsGiftCard
66 Verifica se o produto é vale presente.
67
68 @param
69 @return lRet .T. -> Produto Vale presente / .F. -> Produto não é vale presente
70 @obs
71 @sample
72 /*/
73 //-----
74 Function STBIsGiftCard(cCodProd)
75 Return FindFunction("STDGGiftCard") .And. STDGGiftCard(cCodProd) == "1" //1-Sim /2->Nao
76
77 //-----
78 /*{Protheus.doc} STBGetCodVP
79 Recupera codigo do vale presente registrado
80
81 @param
82 @return aCodVP codigo do vale presente
83 @obs
84 @sample
85 /*/
86 //-----
87 Function STBGetCodVP()
88 Return aCodVP
89
90 //-----
91 /*{Protheus.doc} STBSetCodVP
92 Seta codigo do vale presente registrado
93
94 @param cCodigo codigo do vale presente
95 @return
96 @obs
97 @sample
98 /*/
99 //-----
100 Function STBSetCodVP(cCodigo)
101
102 Default cCodigo := ""
103
104 If !Empty(cCodigo)

```

```
105      aAdd(aCodVP , cCodigo)
106 Else
107      aCodVP := {}
108 EndIf
109
110 Return
111
112 //-----
113 /*{Protheus.doc} STBExistVP
114 Verifica se existe vale presente na venda
115
116 @param
117 @return
118 @obs
119 @sample
120 */
121 //-----
122 Function STBExistVP()
123
124 Local lRet := .F.
125
126 If STDExistVP()
127     lRet := .T.
128 EndIf
129
130 Return lRet
```

Componentes/GiftCard/STDValePresente.prw

282 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 //-----
4 /*{Protheus.doc} STDEstorVP
5 Estorna valor do vale presente
6
7 @param cVale  Código do vale presente
8 @param nValor Valor do vale presente
9 @return lRet
10 @obs
11 @sample
12 */
13 //-----
14 Function STDEstorVP(cVale, nValor)
15
16 Local aArea      := GetArea()    // Guarda area corrente
17 Local lRet       := .F.         // Retorno
18
19 Default cVale   := ""
20
21 cVale := AllTrim(cVale)
22
23 DbSelectArea("MDD")
24 DbSetOrder(1) //MDD_FILIAL+MDD_CODIGO
25
26 If DbSeek(xFilial("MDD")+cVale)
27   RecLock( "MDD", .F. )
28
29   If SuperGetMV("MV_LJBXPAR",,.F.)
30     MDD->MDD_STATUS := '5'
31     MDD->MDD_SALDO := MDD->MDD_SALDO + nValor
32   Else
33     MDD->MDD_STATUS := '2'
34   EndIf
35
36   MsUnlock()
37   lRet := .T.
38 EndIf
39
40 RestArea(aArea)
41
42 Return lRet
43
44 //-----
45 /*{Protheus.doc} STDEstorVP
46 Valida e chama a função para executar o estorno do vale presente
47
48 @param cNumSale  Código da venda
49 @return lRet
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STDExecEstorVP(cNumSale)
55
56 Local lRet := .F. // Retorno
57
58 DbSelectArea("SL4")
59 DbSetOrder(1) //L4_FILIAL + L4_NUM
60
61 DbSeek( xFilial("SL4") + cNumSale )
62
63 While !EOF() .AND. SL4->L4_FILIAL + SL4->L4_NUM == xFilial("SL4") + cNumSale
64
65     If AllTrim(SL4->L4_FORMA) == 'VP'
66         lRet := STBRemoteExecute("STDEstorVP" ,{SL4->L4_CODVP,SL4->L4_VALOR}, NIL,.F.)
67     EndIf
68
69     SL4->(dbSkip())
70
71 End
72
73 Return lRet
74
75 //-----
76 /*{Protheus.doc} STDGetMinMaxVP
77 Retorna um array contendo o valor minimo e maximo de venda do vale presente
78
79 @param cCodProduto  codigo do produto
80 @return aRes valor minino e maximo vale presente
81 @obs
82 @sample
83 */
84 //-----
85 Function STDGetMinMaxVP(cCodProduto)
86
87 Local aRes      := {}
88 Local lMDDTipo  := MDD->(ColumnPos("MDD_TIPO"))>0
89
90 Default cCodProduto := ""
91
92 DbSelectArea("MDD")
93 DbSetOrder(1) //MDD_FILIAL + MDD_CODIGO + MDD_SERIR + MDD_DOCR + MDD_PDVR
94
95 If DbSeek( xFilial("MDD") + cCodProduto )
96     aRes := {MDD->MDD_VALDE, MDD->MDD_VALATE, iIf(lMDDTipo,MDD->MDD_TIPO,"P"), MDD->MDD_PROD}
97 EndIf
98
99 Return aRes
100
101 //-----
102 /*{Protheus.doc} STDGGiftCard
103 Retorna o conteudo do campo B1_VALEPRE que indica se o produto é vale presente
104

```

```

105 @param cCodProduto - código do produto
106 @return cRet conteúdo do campo B1_VALEPRE ("1" -Sim / "2" - Não
107 @obs
108 @sample
109 */
110 //-----
111 Function STDGGiftCard(cCodProd)
112 Local cRet := ""
113 Local aArea := SB1->(GetArea())
114
115 SB1->(DBSetOrder(1)) //FILIAL + CODIGO
116 If SB1->(DBseek(xFilial("SB1") + cCodProd))
117     cRet := AllTrim(SB1->B1_VALEPRE) //1-Sim /2->Nao
118 EndIf
119
120 RestArea(aArea)
121
122 Return cRet
123
124 //-----
125 /*{Protheus.doc} STDVldVP
126 Valida se o Vale Presente já foi adicionado no carrinho
127
128 @param cCodVP Código do vale presente
129 @return lRet
130 @obs
131 @sample
132 */
133 //-----
134 Function STDVldVP(cCodVP,cTipoVp)
135
136 Local cNum := STDGPBasket("SL1","L1_NUM") //numero da venda
137 Local lRet := .F.
138
139 Default cCodVP := ""
140 Default cTipoVp := "P"
141
142 If cCodVP <> "" .And. !(cTipoVp == "C")
143
144     DbSelectArea("SL2")
145     DbSetOrder(1) //L2_FILIAL + L2_NUM + L2_ITEM + L2_PRODUTO
146
147     /*Busca por todos produtos da venda*/
148     If DbSeek( xFilial("SL2") + cNum )
149         While !EOF()
150             /*Valida se o vale presente informado ja foi registrado*/
151             If SL2->L2_VALEPRE == cCodVP
152                 lRet := .T.
153             EndIf
154             DbSkip()
155         EndDo
156     EndIf

```

```

157
158 EndIf
159
160 Return lRet
161
162 //-----
163 /*{Protheus.doc} STDIsVP
164 Verifica se o produto é vale presente
165
166 @param   nCodProd Código do produto
167 @return  lRet
168 @obs
169 @sample
170 */
171 //-----
172 Function STDIsVP(cCodProd)
173
174 Local lRet := .F.
175
176 Default cCodProd := 0
177
178 If !Empty(cCodProd)
179
180     DbSelectArea("SB1")
181     DbSetOrder(1) //B1_FILIAL + B1_COD
182
183     If DbSeek(xFilial("SB1") + cCodProd)
184
185         If SB1->B1_VALEPRE == "1"
186             lRet := .T.
187         EndIf
188     Else
189         lRet := .F.
190     EndIf
191 Else
192     lRet := .F.
193 EndIf
194
195 Return lRet
196
197 //-----
198 /*{Protheus.doc} STDExistVP
199 Verifica se existe vale presente na venda
200
201 @param
202 @return
203 @obs
204 @sample
205 */
206 //-----
207 Function STDExistVP()
208

```

```

209 Local cNum      := STDGPBasket("SL1","L1_NUM") //numero da venda
210 Local lRet      := .F.
211 Local cSL2Fil    := ""
212
213 DbSelectArea("SL2")
214 DbSetOrder(1)    //L2_FILIAL + L2_NUM + L2_ITEM + L2_PRODUTO
215 cSL2Fil := xFilial("SL2")
216 If SL2->(DbSeek( cSL2Fil + cNum ))
217     While SL2->(!EOF()) .And. cSL2Fil + cNum == SL2->(L2_FILIAL+L2_NUM)
218         If !Empty(SL2->L2_VALEPRE)
219             lRet := .T.
220         EndIf
221         DbSkip()
222     EndDo
223 EndIf
224
225 Return lRet
226
227 /*{Protheus.doc} STDGDVpre
228 Seleciona os vales presentes na tabela MDD e atualiza o objeto oSay com os dados
229
230 @param  cStVP, caracter, Status do vale presente
231
232 @Type Function
233 @Return aRet, Array, Array com os dados da consulta
234
235 */
236 Function STDGDVpre(cStVP)
237 Local aArea      := GetArea()
238 Local cQuery     := ""
239 Local aRet       := {}
240 Local cAlias     := GetNextAlias()
241 Default cStVP   := "1" //Status de vale a ser vendido
242
243 cQuery := "SELECT MDD_CODIGO, MDD_VALOR, MDD_PROD, MDD_DTINI, MDD_DTFIM "
244 cQuery += "FROM " + RetSQLName("MDD") + " MDD "
245 cQuery += "WHERE "
246 cQuery += "MDD_FILIAL =' " + xFilial("MDD") + "' AND "
247 cQuery += "MDD_STATUS =' " + cStVP + "' AND "
248 cQuery += "MDD.D_E_L_E_T_ = ' ' "
249
250 cQuery += " ORDER BY MDD_CODIGO "
251
252 cQuery := ChangeQuery(cQuery)
253
254 dbUseArea(.T.,"TOPCONN",TcGenQry(,cQuery),cAlias,.T.,.T.)
255
256 DbSelectArea(cAlias)
257 DbGoTop()
258
259 While !(cAlias)->(Eof())
260     cDtvig := Substr((cAlias)->MDD_DTINI,7,2) + "/" +;

```

```
261          Substr((cAlias)->MDD_DTINI,5,2) + "/" +;  
262          Substr((cAlias)->MDD_DTINI,1,4)  
263  
264      cDtfin := Substr((cAlias)->MDD_DTFIM,7,2) + "/" +;  
265              Substr((cAlias)->MDD_DTFIM,5,2) + "/" +;  
266              Substr((cAlias)->MDD_DTFIM,1,4)  
267  
268      Aadd(aRet, {(cAlias)->MDD_CODIGO,;  
269                  Alltrim(STR((cAlias)->MDD_VALOR,7,2)),;  
270                  AllTrim((cAlias)->MDD_PROD),;  
271                  cDtvig,;  
272                  cDtfin})  
273  
274      (cAlias)->(DbSkip())  
275 End  
276  
277 (cAlias)->(dbCloseArea())  
278  
279  
280 RestArea(aArea)  
281  
282 Return aRet
```